

第4章 贪心法

(Greedy Approach)

什么是贪心法

贪心法的**基本思路**是在对问题求解时总是做出在当前看来是最好的选择，也就是说贪心法不从整体最优上加以考虑，所做出的仅是在某种意义上的局部最优解。

人们通常希望找到整体最优解，所以采用贪心法**需要证明**设计的算法确实是整体最优解或求解了它要解决的问题。

贪心算法的正确性，就是要证明按贪心准则求得的解是全局最优解。

贪心算法不能对所有问题都得到全局最优解。

贪心法求解的问题应具有的性质

1. 贪心选择性质

所谓贪心选择性质是指所求问题的整体最优解可以通过一系列局部最优的选择，即贪心选择来达到。

也就是说，贪心法仅在当前状态下做出最好选择，即局部最优选择，然后再去求解做出这个选择后产生的相应子问题的解。

通常以自顶向下的方式进行，每次选择后将问题转化为规模更小的子问题。

该性质是贪心法使用成功的保障，否则得到的是近优解。

2. 最优子结构性质

如果一个问题的最优解包含其子问题的最优解，则称此问题具有**最优子结构性质**。

问题的最优子结构性质是该问题可用动态规划算法或贪心法求解的**关键特征**。

并不是所有具有最优子结构性质的问题都可以采用贪心策略。

往往可以利用最优子结构性质来证明贪心选择性质。

贪心算法虽不能保证得到最优结果，但对于一些除了“穷举”方法外没有有效算法的问题，用贪心算法往往能很快地得出较好的结果，此方法还是很实用的。

与动态规划方法的比较

- 动态规划方法可用的条件

- 优化子结构
- 子问题重叠性
- 子问题空间小

- 贪心方法可用的条件

- 优化子结构
- 贪心选择性

活动选择问题

输入: $S = \{1, 2, \dots, n\}$ 为 n 项活动的集合, s_i, f_i 分别为活动 i 的开始和结束时间.

活动 i 与 j 相容 $\Leftrightarrow s_i \geq f_j$ 或 $s_j \geq f_i$.

求: 最大的两两相容的活动集 A

输入实例:

i	1	2	3	4	5	6	7	8	9	10
s_i	1	3	2	5	4	5	6	8	8	2
f_i	4	5	6	7	9	9	10	11	12	13

解: $\{1, 4, 8\}$

贪心算法

挑选过程是多步判断，每步依据某种“短视”的策略进行活动选择，选择时注意满足相容性条件.

策略1： 开始时间早的优先

排序使 $s_1 \leq s_2 \leq \dots \leq s_n$ ，从前向后挑选

策略2： 占用时间少的优先

排序使得 $f_1 - s_1 \leq f_2 - s_2 \leq \dots \leq f_n - s_n$ ，
从前向后挑选

策略3： 结束早的优先

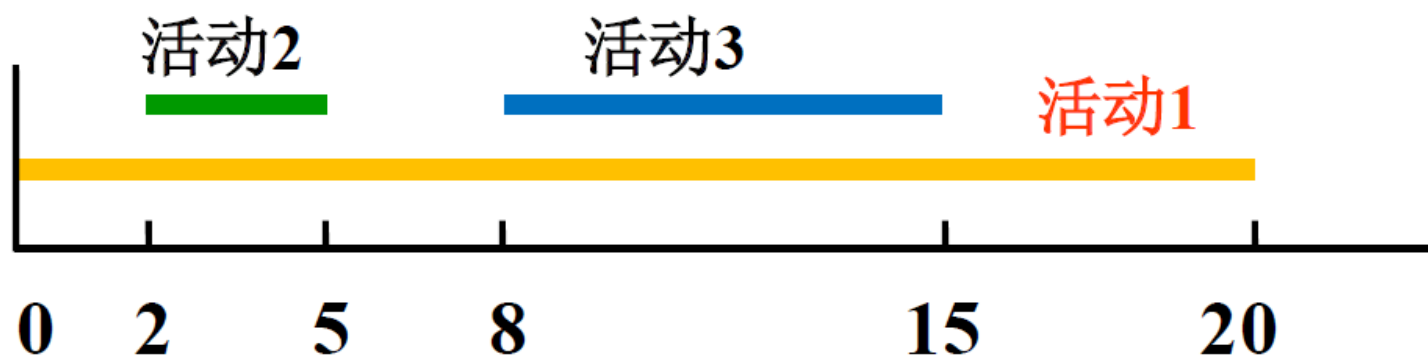
排序使 $f_1 \leq f_2 \leq \dots \leq f_n$ ，从前向后挑选

策略1的反例

策略1：开始早的优先

反例： $S = \{1, 2, 3\}$

$s_1=0, f_1=20, s_2=2, f_2=5, s_3=8, f_3=15$

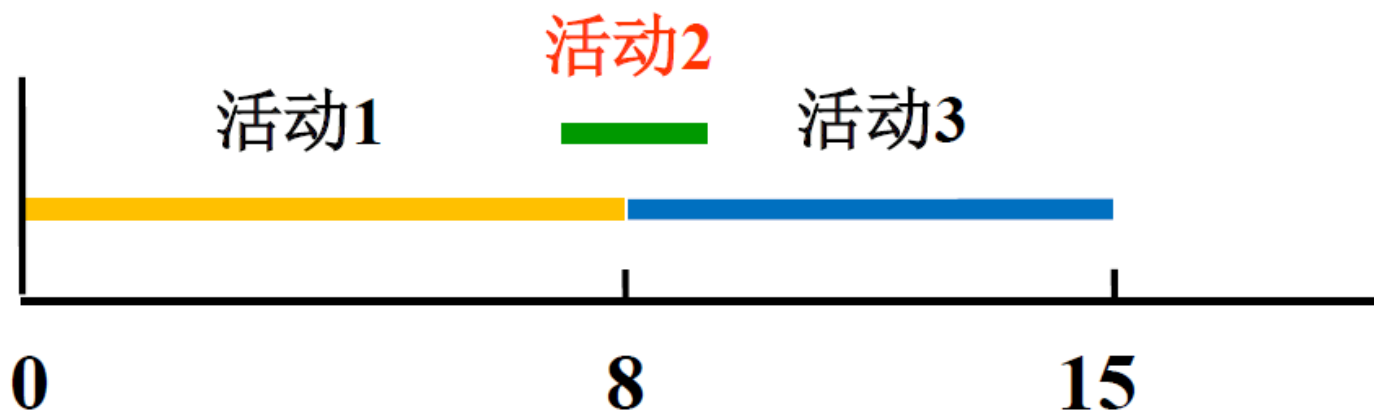


策略2的反例

策略2： 占时少的优先

反例： $S = \{ 1, 2, 3 \}$

$$s_1=0, f_1=8, s_2=7, f_2=9, s_3=8, f_3=15$$



策略3伪码

算法 Greedy Select

输入：活动集 S , s_i, f_i ,

$i = 1, 2, \dots, n, f_1 \leq \dots \leq f_n$

输出： $A \subseteq S$, 选中的活动子集

1. $n \leftarrow \text{length}[S]$

2. $A \leftarrow \{1\}$

已选入的
最后标号

3. $j \leftarrow 1$

4. for $i \leftarrow 2$ to n do

5. if $s_i \geq f_j$

判相容

6. then $A \leftarrow A \cup \{i\}$

7. $j \leftarrow i$

8. return A

完成时间 $t = \max \{f_k: k \in A\}$

运行实例

输入: $S = \{ 1, 2, \dots, 10 \}$

i	1	2	3	4	5	6	7	8	9	10
s_i	1	3	0	5	3	5	6	8	8	2
f_i	4	5	6	7	8	9	10	11	12	13

解: $A = \{1, 4, 8\}, t = 11$

时间复杂度

$$O(n \log n) + O(n) = O(n \log n)$$



如何证明该算法对所有的实例
都得到正确的解?

一个数学归纳法的例子

例：证明对于任何自然数 n ,

$$1+2+\dots+n = n(n+1)/2$$

证 $n=1$, 左边=1, 右边= $1 \times (1+1)/2=1$

假设对任意自然数 n 等式成立, 则

$$\begin{aligned} & 1+2+\dots+(n+1) \\ &= (1+2+\dots+n) + (n+1) \\ &= n(n+1)/2 + (n+1) \\ &= (n+1)(n/2+1) \\ &= (n+1)(n+2)/2 \end{aligned}$$

归纳假
设代入

第一数学归纳法

适合证明涉及自然数的命题 $P(n)$

归纳基础: 证明 $P(1)$ 为真 (或 $P(0)$ 为真).

归纳步骤: 若对所有 n 有 $P(n)$ 为真, 证明
 $P(n+1)$ 为真

$$\forall n, P(n) \rightarrow P(n+1)$$

$$P(1)$$

$$n=1, P(1) \Rightarrow P(2)$$

$$n=2, P(2) \Rightarrow P(3)$$

...

第二数学归纳法

适合证明涉及自然数的命题 $P(n)$

归纳基础: 证明 $P(1)$ 为真 (或 $P(0)$ 为真).

归纳步骤: 若对所有小于 n 的 k 有 $P(k)$ 真,
证明 $P(n)$ 为真

$$\forall k (k < n \wedge P(k)) \rightarrow P(n)$$

$$P(1)$$

$$n=2, \quad P(1) \Rightarrow P(2)$$

$$n=3, \quad P(1) \wedge P(2) \Rightarrow P(3)$$

...

两种归纳法的区别

归纳基础一样

$P(1)$ 为真

归纳步骤不同

证明逻辑

归纳法1: $P(1) \Rightarrow P(2) \Rightarrow P(3) \dots$

归纳法2:

$$\left. \begin{array}{l} P(1) \\ P(1) \Rightarrow P(2) \end{array} \right\} \Rightarrow \left. \begin{array}{l} P(1) \\ P(2) \\ P(3) \end{array} \right\} \Rightarrow P(4) \dots$$

算法正确性归纳证明

证明步骤:

1. 叙述一个有关自然数 n 的命题, 该命题断定该贪心策略的执行最终将导致最优解. 其中自然数 n 可以代表算法步数或者问题规模.
2. 证明命题对所有的自然数为真.
归纳基础(从最小实例规模开始)
归纳步骤(第一或第二数学归纳法)

活动选择算法的命题

命题

算法 Select 执行到第 k 步, 选择 k 项活动

$$i_1 = 1, i_2, \dots, i_k$$

则存在最优解 A 包含活动 $i_1=1, i_2, \dots, i_k$.

根据上述命题: 对于任何 k , 算法前 k 步的选择都将导致最优解, 至多到第 n 步将得到问题实例的最优解

归纳证明： 归纳基础

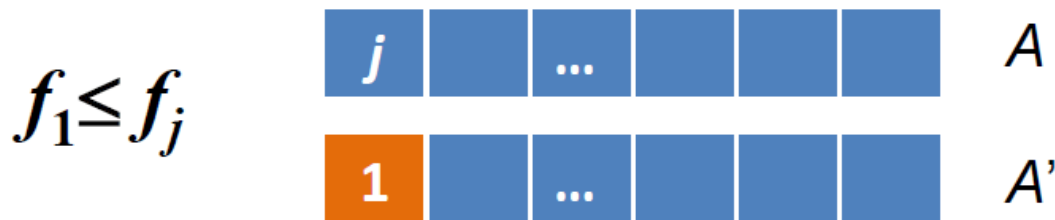
令 $S=\{1,2,\dots,n\}$ 是活动集, 且 $f_1 \leq \dots \leq f_n$

归纳基础: $k=1$, 证明存在最优解包含活动 1

证 任取最优解 A , A 中活动按截止时间递增排列. 如果 A 的第一个活动为 j , $j \neq 1$, 用 1 替换 A 的活动 j 得到解 A' , 即

$$A' = (A - \{j\}) \cup \{1\},$$

由于 $f_1 \leq f_j$, A' 也是最优解, 且含有 1.



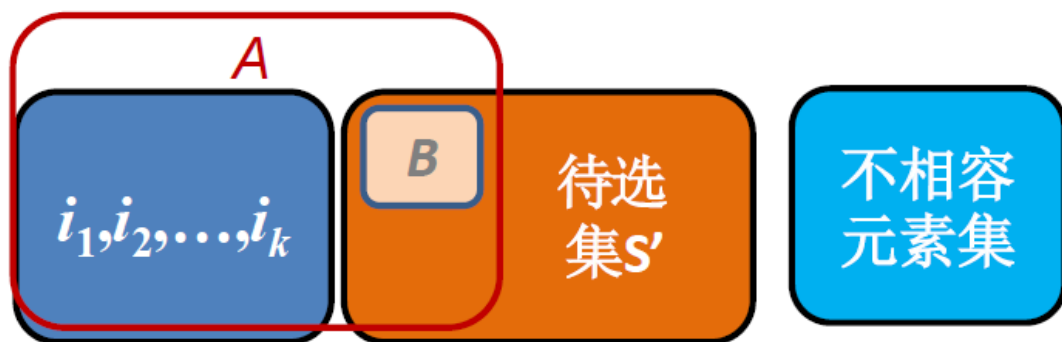
归纳步骤

假设命题对 k 为真, 证明对 $k+1$ 也为真.

证 算法执行到第 k 步, 选择了活动 $i_1=1, i_2, \dots, i_k$, 根据归纳假设存在最优解 A 包含 $i_1=1, i_2, \dots, i_k$, A 中剩下活动选自集合 S'

$$S' = \{ i \mid i \in S, s_i \geq f_k \}$$

$$A = \{ i_1, i_2, \dots, i_k \} \cup B$$

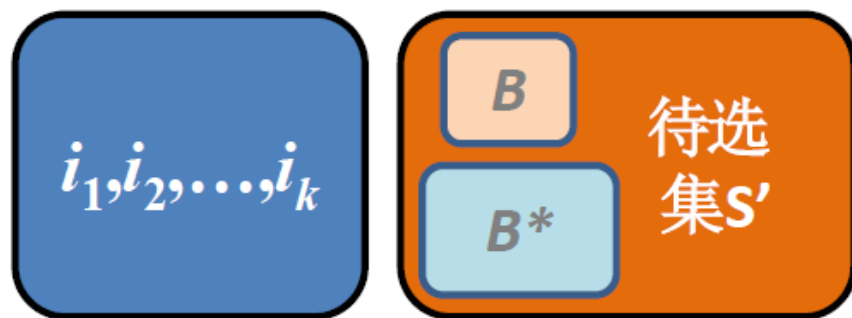


归纳步骤（续）

B 是 S' 的最优解. (若不然, S' 的最优解为 B^* , B^* 的活动比 B 多, 那么

$$B^* \cup \{1, i_2, \dots, i_k\}$$

是 S 的最优解, 且比 A 的活动多, 与 A 的最优性矛盾.)

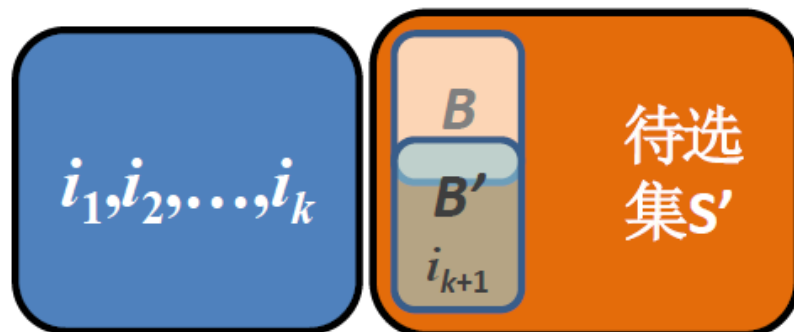


归纳步骤(续)

将 S' 看成子问题，根据归纳基础，
存在 S' 的最优解 B' 有 S' 中的第一个
活动 i_{k+1} ，且 $|B'| = |B|$ ，于是

$$\begin{aligned} & \{ i_1, i_2, \dots, i_k \} \cup B' \\ &= \{ i_1, i_2, \dots, i_k, i_{k+1} \} \cup (B' - \{ i_{k+1} \}) \end{aligned}$$

也是原问题的最优解.



贪心算法的特点

设计要素：

- (1) 贪心法适用于组合优化问题.
- (2) 求解过程是多步判断过程，最终的判断序列对应于问题的最优解.
- (3) 依据某种“短视的”贪心选择性质判断，性质好坏决定算法的成败.
- (4) 贪心法必须进行正确性证明.
- (5) 证明贪心法不正确的技巧：举反例.

贪心法的优势：算法简单，时间和空间复杂性低

最优装载问题

问题:

n 个集装箱 $1, 2, \dots, n$ 装上轮船, 集装箱 i 的重量 w_i , 轮船装载重量限制为 C , 无体积限制. 问如何装使得上船的集装箱最多? 不妨设每个箱子的重量 $w_i \leq C$.

该问题是0-1背包问题的子问题. 集装箱相当于物品, 物品重量是 w_i , 价值 v_i 都等于1, 轮船载重限制 C 相当于背包重量限制 b .

建模

设 $\langle x_1, x_2, \dots, x_n \rangle$ 表示解向量, $x_i = 0, 1$,
 $x_i = 1$ 当且仅当第 i 个集装箱装上船

目标函数 $\max \sum_{i=1}^n x_i$

约束条件 $\sum_{i=1}^n w_i x_i \leq C$

$$x_i = 0, 1 \quad i = 1, 2, \dots, n$$

算法设计

- 贪心策略：轻者优先
- 算法设计：
将集装箱排序，使得

$$w_1 \leq w_2 \leq \dots \leq w_n$$

按照标号从小到大装箱，直到装入下一个箱子将使得集装箱总重超过轮船装载重量限制，则停止.

正确性证明思路

- **命题：**对装载问题任何规模为 n 的输入实例，算法得到最优解.
- 设集装箱从轻到重记为 $1, 2, \dots, n$.

归纳基础 证明对任何只含 1 个箱子的输入实例，贪心法得到最优解.
显然正确.

- **归纳步骤** 证明：假设对于任何 n 个箱子的输入实例贪心法都能得到最优解，那么对于任何 $n+1$ 个箱子的输入实例贪心法也得到最优解.

归纳步骤证明思路

$N=\{1,2,\dots,n+1\}, w_1\leq w_2\leq\dots\leq w_{n+1}$



去掉箱子1, 令 $C' = C - \{w_1\}$,
得到规模 n 的输入 $N' = \{2,3,\dots,n+1\}$



关于输入 N' 和 C' 的最优解 I'



在 I' 加入箱子1, 得到 I



证明 I 是关于输入 N 的最优解

正确性证明

假设对于 n 个集装箱的输入，贪心法都可以得到最优解，考虑 输入

$$N = \{ 1, 2, \dots, n+1 \}$$

其中 $w_1 \leq w_2 \leq \dots \leq w_{n+1}$.

由归纳假设，对于

$$N' = \{ 2, 3, \dots, n+1 \}, \quad C' = C - w_1,$$

贪心法得到最优解 I' . 令

$$I = I' \cup \{ 1 \}$$

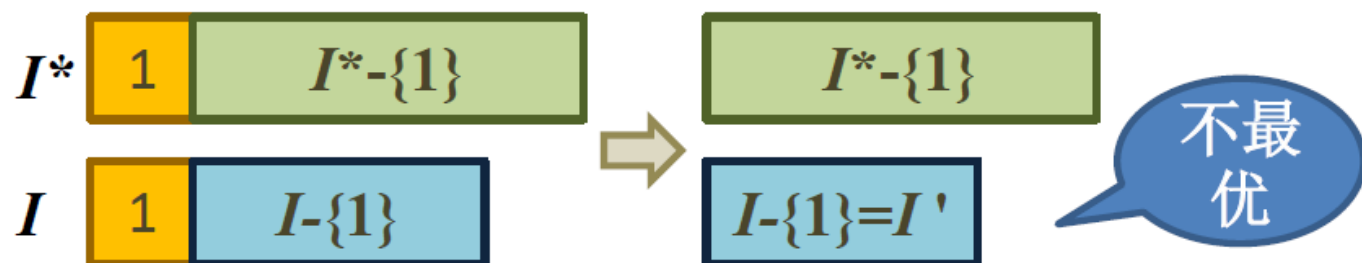
正确性证明 (续)

I (算法解) 是关于 N 的最优解.

若不然, 存在包含 1 的关于 N 的最优解 I^* (如果 I^* 中没有 1, 用 1 替换 I^* 中的第一个元素得到的解也是最优解), 且 $|I^*| > |I|$; 那么 $I^* - \{1\}$ 是 N' 和 C' 的解且

$$|I^* - \{1\}| > |I - \{1\}| = |I'|$$

与 I' 是关于 N' 和 C' 的最优解矛盾.



最小延迟调度

问题:

客户集合 A , $\forall i \in A$, t_i 为服务时间, d_i 为要求完成时间, t_i, d_i 为正整数. 一个调度 $f: A \rightarrow \mathbb{N}$, $f(i)$ 为客户 i 的开始时间. 求最大延迟达到最小的调度, 即求 f 使得

$$\min_f \{ \max_{i \in A} \{ f(i) + t_i - d_i \} \}$$

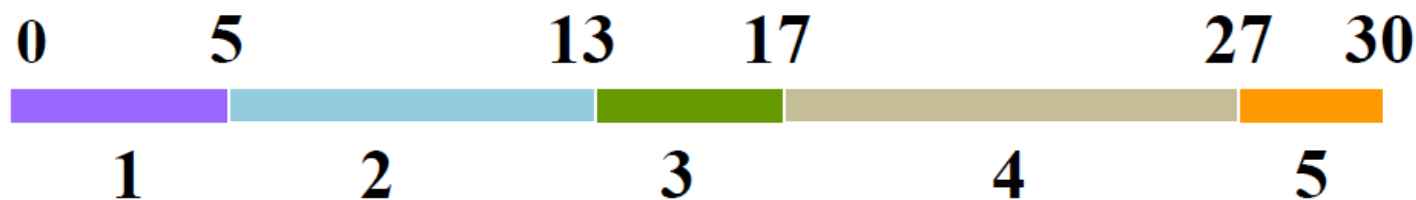
$$\forall i, j \in A, i \neq j, f(i) + t_i \leq f(j) \\ \text{or } f(j) + t_j \leq f(i)$$

实例：调度1

$A = \{1, 2, 3, 4, 5\}$, $T = \langle 5, 8, 4, 10, 3 \rangle$,
 $D = \langle 10, 12, 15, 11, 20 \rangle$

调度1：顺序安排

$f_1(1)=0, f_1(2)=5, f_1(3)=13, f_1(4)=17, f_1(5)=27$



各任务延迟：0, 1, 2, **16**, 10;

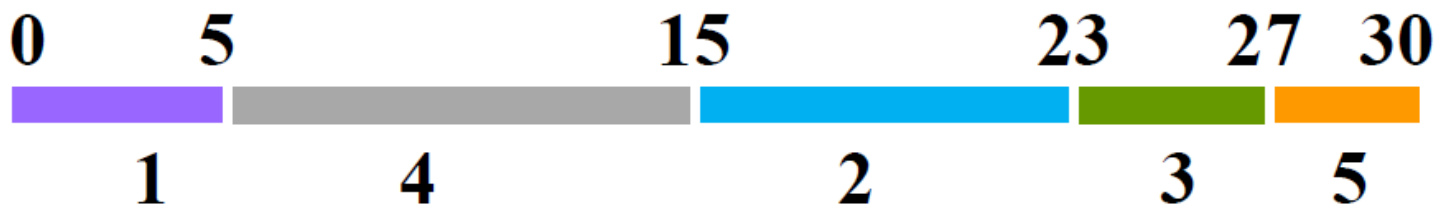
最大延迟：16

更优的调度2

$A = \{1, 2, 3, 4, 5\}$, $T = \langle 5, 8, 4, 10, 3 \rangle$,
 $D = \langle 10, 12, 15, 11, 20 \rangle$

调度2：按截止时间从前到后安排

$f_2(1)=0, f_2(2)=15, f_2(3)=23, f_2(4)=5, f_2(5)=27$



各任务延迟：0, 11, 12, 4, 10;

最大延迟：12

贪心策略

贪心策略1: 按照 t_i 从小到大安排

贪心策略2: 按照 $d_i - t_i$ 从小到大安排

贪心策略3: 按照 d_i 从小到大安排

策略1 对某些实例得不到最优解.

反例: $t_1=1, d_1=100, t_2=10, d_2=10$

策略2 对某些实例得不到最优解.

反例: $t_1=1, d_1=2, t_2=10, d_2=10$

策略3伪码

算法 Schedule

输入: A, T, D

输出: f

1. 排序 A 使得 $d_1 \leq d_2 \leq \dots \leq d_n$

2. $f(1) \leftarrow 0$

从0时
刻起

3. $i \leftarrow 2$

4. while $i \leq n$ do

5. $f(i) \leftarrow f(i-1) + t_{i-1}$

没有
空闲

6. $i \leftarrow i + 1$

设计思想: 按完成时间从早到晚安
排任务, 没有空闲.

交换论证：正确性证明

证明思路：

- 分析一般最优解与算法解的区别（成分,排列顺序不同）
- 设计一种转换操作（替换成分或交换次序），可以在有限步将任意一个普通最优解逐步转换成算法的解
- 上述每步转换都不降低解的最优性质

贪心算法的解的性质：

没有空闲时间，没有逆序。

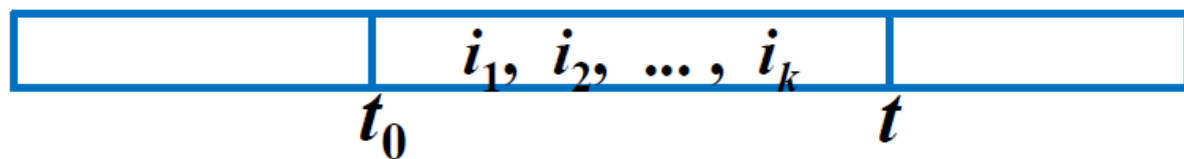
逆序 (i, j) : $f(i) < f(j)$ 且 $d_i > d_j$

引理

引理1 所有没有逆序、没有空闲时间的调度具有相同的最大延迟.

证: 设 f 没有逆序, 在 f 中具有相同完成时间 d 的客户 $i_1, i_2, \dots, i_k$ 连续安排, 其开始时刻为 t_0 , 完成这些任务的时刻是 t , 最大延迟为最后任务延迟 $t-d$, 与 $i_1, i_2, \dots, i_k$ 的排列次序无关.

$$t = t_0 + (t_{i_1} + t_{i_2} + \dots + t_{i_k})$$



证明要点

从一个没有空闲的最优解出发，逐步转变成没有逆序的解。根据引理 1，这个解和算法解具有相同的最大延迟。

- (1) 如果一个最优调度存在逆序，那么存在 $i < n$ 使得 $(i, i+1)$ 构成一个逆序，称为相邻的逆序。
- (2) 交换相邻逆序 i 和 j ，得到的解仍旧最优。
- (3) 每次交换后逆序数减 1，至多经过 $n(n-1)/2$ 次交换得到一个没有逆序的最优调度——等价于算法的解。

交换相邻逆序仍旧最优

设 f_1 是一个任意最优解，存在相邻逆序 (i, j) 。交换 i 和 j 的顺序，得到解 f_2 。那么 f_2 的最大延迟不超过 f_1 的最大延迟。

理由：

- (1) 交换 i, j 与其他客户延迟时间无关
- (2) 交换后不增加 j 的延迟，但可能增加 i 的延迟
- (3) i 在 f_2 的延迟小于 j 在 f_1 的延迟，因此小于 f_1 的最大延迟 r

i 在 f_2 的延迟不超过
 j 在 f_1 的延迟



$$\text{delay}(f_2, i) = \underline{s+t_j+t_i} - d_i$$

$$\text{delay}(f_1, j) = \underline{s+t_i+t_j} - d_j$$

$$d_j < d_i$$

$$\text{delay}(f_2, i) < \text{delay}(f_1, j) \leq r$$

得不到最优解的处理

- 输入参数分析：
考虑输入参数在什么取值范围内使用贪心法可以得到最优解
- 误差分析：
估计贪心法——近似算法所得到的解与最优解的误差（对所有的输入实例在最坏情况下误差的上界）

找零钱问题

问题 设有 n 种零钱，重量分别为 $w_1, w_2, \dots, w_n$ ，价值分别为 $v_1 = 1, v_2, \dots, v_n$ 。需要付的总钱数是 y 。不妨设币值和钱数都为正整数。问：如何付钱使得所付钱的总重最轻？

实例

$v_1=1, v_2=5, v_3=14, v_4=18, w_i=1, i=1,2,3,4.$
 $y=28$

最优解： $x_3=2, x_1=x_2=x_4=0$ ，总重2

建模

令选用第 i 种硬币的数目是 x_i ,
 $i = 1, 2, \dots, n$

$$\min \left\{ \sum_{i=1}^n w_i x_i \right\}$$

$$\sum_{i=1}^n v_i x_i = y, \quad x_i \in \mathbb{N}, \quad i = 1, 2, \dots, n$$

动态规划算法

设 $F_k(y)$ 表示用前 k 种零钱，总钱数为 y 的最小重量

$$F_k(y) = \min_{0 \leq x_k \leq \left\lfloor \frac{y}{v_k} \right\rfloor} \{F_{k-1}(y - v_k x_k) + w_k x_k\}$$

$$F_1(y) = w_1 \left\lfloor \frac{y}{v_1} \right\rfloor = w_1 y$$

贪心法

单位价值重量轻的货币优先，设

$$\frac{w_1}{v_1} \geq \frac{w_2}{v_2} \geq \dots \geq \frac{w_n}{v_n}$$

使用前 k 种零钱，总钱数为 y
贪心法的总重为 $G_k(y)$,

$$G_k(y) = w_k \left\lfloor \frac{y}{v_k} \right\rfloor + G_{k-1}(y \bmod v_k) \quad k > 1$$

$$G_1(y) = w_1 \left\lfloor \frac{y}{v_1} \right\rfloor = w_1 y$$

$n=1,2$ 贪心法是最优解

$n = 1$ 只有一种零钱, $F_1(y) = G_1(y)$

$n = 2$, x_2 越大, 得到的解越好

$$F_2(y) = \min_{0 \leq x_2 \leq \lfloor y/v_2 \rfloor} \{F_1(y - v_2 x_2) + w_2 x_2\}$$

$$\begin{aligned} F_1(y) &= w_1 y & F_1(y - v_2(\underline{x_2 + \delta})) + w_2(\underline{x_2 + \delta}) \\ & & - [F_1(y - v_2 \underline{x_2}) + w_2 \underline{x_2}] \\ &= [w_1(\underline{y - v_2 x_2 - v_2 \delta}) + \underline{w_2 x_2 + w_2 \delta}] \\ & & - [w_1(\underline{y - v_2 x_2}) + \underline{w_2 x_2}] \\ &= -w_1 v_2 \delta + w_2 \delta \\ &= \delta(-w_1 v_2 + w_2) \leq 0 \end{aligned}$$

$$w_1 \geq w_2/v_2$$

判别条件

定理 对每个正整数 k , 假设对所有非负整数 y 有 $G_k(y) = F_k(y)$, 那么 $G_{k+1}(y) \leq G_k(y) \Leftrightarrow F_{k+1}(y) = G_{k+1}(y)$.

定理 对每个正整数 k , 假设对所有非负整数 y 有 $G_k(y) = F_k(y)$, 且存在 p 和 δ 满足

$$v_{k+1} = pv_k - \delta,$$

其中 $0 \leq \delta < v_k$, $v_k \leq v_{k+1}$, p 为正整数,

则下面的命题等价:

- (1) $G_{k+1}(y) = F_{k+1}(y)$ 对一切正整数 y ;
- (2) $G_{k+1}(pv_k) = F_{k+1}(pv_k)$;
- (3) $w_{k+1} + G_k(\delta) \leq pw_k$.

几点说明

- 根据条件(1)与(3)的等价性, 可以对 $k = 3, 4, \dots, n$, 依次利用条件(3)对贪心法是否得到最优解做出判别.
- 条件(3)验证 1 次需 $O(k)$ 时间, $k = O(n)$, 整个验证时间 $O(n^2)$
- 条件(2)是条件(1) 在 $y = pv_k$ 时的特殊情况. 若条件(1)成立, 显然有条件(2)成立. 反之, 若条件(2)不成立, 则条件(1)不成立, 钱数 $y = pv_k$ 恰好提供了一个贪心法不正确的反例.

验证实例

$$\begin{aligned} v_{k+1} &= pv_k - \delta, \quad 0 \leq \delta < v_k, \quad p \in \mathbb{Z}^+ \\ w_{k+1} + G_k(\delta) &\leq pw_k \end{aligned}$$

例 $v_1=1, v_2=5, v_3=14, v_4=18, w_i=1,$
 $i=1,2,3,4$. 对一切 y 有

$$G_1(y)=F_1(y), \quad G_2(y)=F_2(y).$$

验证 $G_3(y) = F_3(y)$

$$v_3 = pv_2 - \delta \Rightarrow p = 3, \delta = 1.$$

$$w_3 + G_2(\delta) = 1 + 1 = 2$$

$$pw_2 = 3 \times 1 = 3$$

$$w_3 + G_2(\delta) \leq pw_2$$

贪心法对于 $n=3$ 的实例得到最优解

验证实例

$$\begin{aligned} v_{k+1} &= pv_k - \delta, \quad 0 \leq \delta < v_k, \quad p \in \mathbb{Z}^+ \\ w_{k+1} + G_k(\delta) &\leq pw_k \end{aligned}$$

例 $v_1=1, v_2=5, v_3=14, v_4=18, w_i=1,$

$i=1,2,3,4.$ 对一切 y 有

$$G_1(y)=F_1(y), G_2(y)=F_2(y), G_3(y)=F_3(y)$$

验证 $G_4(y) = F_4(y)$

$$v_4 = pv_3 - \delta \Rightarrow p=2, \delta=10$$

$$w_4 + G_3(\delta) = 1 + 2 = 3$$

$$pw_3 = 2 \times 1 = 2$$

$$w_4 + G_3(\delta) > pw_3$$

$n=4, y=pv_3=28,$

最优解: $x_3=2$, 贪心法: $x_4=1, x_2=2$

小结

- 贪心策略不一定得到最优解，在这种情况下可以有两种处理方法：
 - (1) 参数化分析：分析参数取什么值可得到最优解
 - (2) 估计贪心法得到的解在最坏情况下与最优解的误差
- 一个参数化分析的例子：找零钱问题

近似算法的有关概念

近似算法

适用于组合优化问题
一般是多项式时间的算法

近似算法A的可行解及值

A是一个多项式时间算法且对组合优化问题 Π 的每一个实例 I 输出一个可行解 σ (满足约束条件的解), 记作

$$A(I) = c(\sigma),$$

称 $c(\sigma)$ 是 σ 的值

算法A的近似比 $\hat{r}$

设 $\text{OPT}(I)$ 表示实例 I 的最优解的值,

(1) Π 是最小化问题, $r_A(I) = A(I)/\text{OPT}(I)$

(2) Π 是最大化问题, $r_A(I) = \text{OPT}(I)/A(I)$

最优化算法A: 对所有的实例 I 恒有

$A(I) = \text{OPT}(I)$, 即 $r_A(I) = 1$.

A的近似比为 r (A是 r -近似算法):

对每一个实例 I , $r_A(I) \leq r$.

A具有常数近似比: r 是一个常数.

可近似性分类

假设 $P \neq NP$, NP 难的组合优化问题按可近似性可分成 3 类:

完全可近似的: 对任意小的 $\varepsilon > 0$, 存在 $(1+\varepsilon)$ -近似算法, 例如背包问题.

可近似的: 存在具有常数比的近似算法, 例如最小顶点覆盖问题、多机调度问题

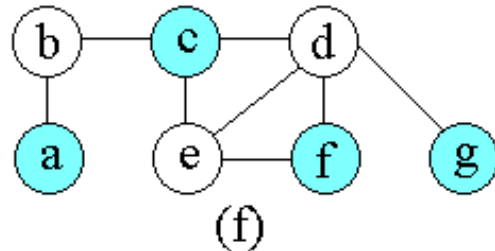
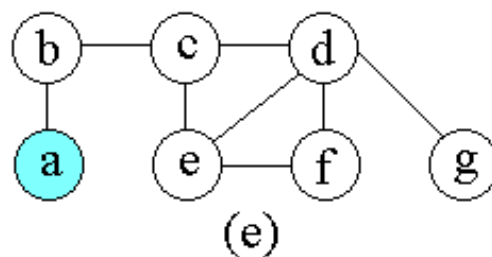
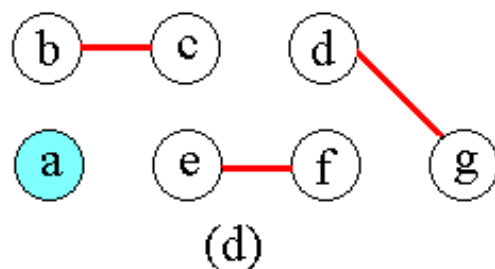
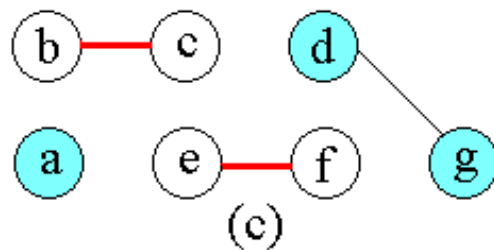
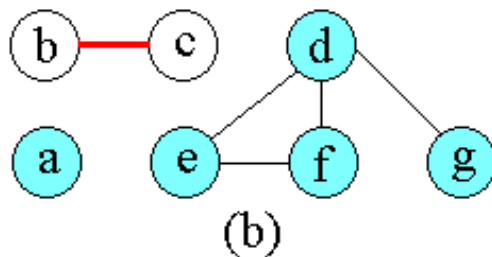
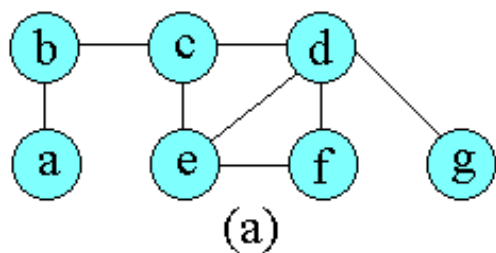
不可近似的: 不存在具有常数比的近似算法, 例如货郎问题

顶点覆盖问题的近似算法

问题描述：无向图 $G=(V, E)$ 的顶点覆盖是它的顶点集 V 的一个子集 $V' \subseteq V$ ，使得若 (u, v) 是 G 的一条边，则 $v \in V'$ 或 $u \in V'$ 。顶点覆盖 V' 的大小是它所包含的顶点个数 $|V'|$ 。求最小顶点覆盖。

```
VertexSet approxVertexCover ( Graph g )
{
    cset=∅;
    e1=g.e;
    while (e1 != ∅) {
        从e1中任取一条边(u, v);
        cset=cset ∪ {u, v};
        从e1中删去与u和v相关联的所有边;
    }
    return c
}
```

Cset用来存储顶点覆盖中的各顶点。初始为空，不断从边集e1中选取一边(u, v)，将边的端点加入cset中，并将e1中已被u和v覆盖的边删去，直至cset已覆盖所有边。即e1为空。



图(a)~(e)说明了算法的运行过程及结果。(e)表示算法产生的近似最优顶点覆盖 $cset$ ，它由顶点 b, c, d, e, f, g 所组成。(f)是图G的一个最小顶点覆盖，它只含有3个顶点： b, d 和 e 。

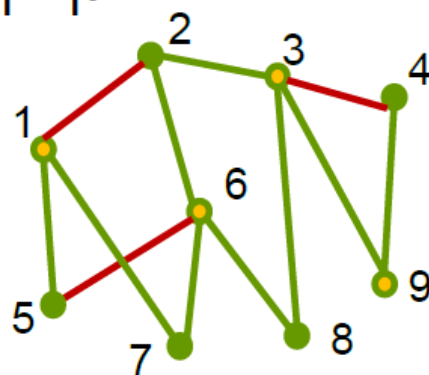
算法approxVertexCover的近似比为2。

MVC算法分析

时间复杂度 $O(m)$, $m=|E|$.

近似比

上界：若算法选取
 k 条边，那么 $|V'| = 2k$ ，
 V' 由 k 条互不关联的边的端点组成。
为了覆盖这 k 条互不关联的边需要 k
个顶点，从而 $\text{OPT}(I) \geq k$. 于是



$$\frac{\text{MVC}(I)}{\text{OPT}(I)} \leq \frac{2k}{k} = 2$$

MVC的近似比

下界

设图 G 由 k 条互不关联的边构成.

显然

$$\text{MVC}(I) = 2k, \quad \text{OPT}(I) = k,$$

表明 MVC 的近似比不会小于2, 即上述MVC近似比已不可能再改进.

结论: MVC是2-近似算法.



$k=4$

近似算法的分析

研究近似算法的两个基本方面——**设计算法**和**分析算法**的运行时间与近似比.

分析近似比

- (1) 关键是估计最优解的值.
- (2) 构造使算法产生最坏的解的实例. 如果这个解的值与最优值的比达到或者可以任意的接近得到的近似比 (这样的实例称作**紧实例**), 那么说明这个近似比已经是最好的、不可改进的了; 否则说明还有进一步的研究余地.
- (3) 研究问题本身的可近似性, 即在 $P \neq NP$ (或其他更强) 的假设下, 该问题近似算法的近似比的下界.

小结

近似算法

适用于组合优化问题

对每个实例都能找到一个可行解

近似算法的评价指标及要求

时间复杂度——算法效率

要求：多项式时间

近似比——性能

要求：常数近似比

小结

近似比的分析方法

(1) 上界

分析近似解与最优解的关系：找到对所有的实例都成立的量化不等式

(2) 下界

找到一个规模可以任意大的紧实例，对这个实例算法的解与最优解的误差与上界尽可能接近

10.2 多机调度问题

多机调度问题：任给有穷的作业集 A 和 m 台相同的机器，作业 a 的处理时间为正整数 $t(a)$ ，每一项作业可以在任一台机器上处理。如何把作业分配给机器才能使完成所有作业的时间最短？即，如何把 A 划分成 m 个不相交的子集 A_i 使得

$$\max \left\{ \sum_{a \in A_i} t(a) \mid i = 1, 2, \dots, m \right\}$$

最小？

10.2.1 贪心的近似算法

负载: 分配给一台机器的作业的处理时间之和.

贪心法G-MPS: 按输入的顺序分配作业, 把每一项作业分配给当前负载最小的机器. 如果当前负载最小的机器有 2 台或 2 台以上, 则分配给其中的任意一台.

例如 3台机器, 8项作业, 处理时间为 3, 4, 3, 6, 5, 3, 8, 4.

算法的分配方案是 {1, 4}, {2, 6, 7}, {3, 5, 8},

负载分别为 $3+6 = 9$, $4+3+8 = 15$, $3+5+4 = 12$,

完成时间为 15.

最优的分配方案是 {1, 3, 4}, {2, 5, 6}, {7, 8},

负载分别为 $3+3+6 = 12$, $4+5+3 = 12$, $8+4 = 12$,

完成时间为 12.

贪心法的性能

定理10.1 对多机调度问题的每一个有 m 台机器的实例 I ,

$$\text{G-MPS}(I) \leq \left(2 - \frac{1}{m}\right) \text{OPT}(I).$$

证 (1) $\text{OPT}(I) \geq \frac{1}{m} \sum_{a \in A} t(a)$, (2) $\text{OPT}(I) \geq \max_{a \in A} t(a)$.

设机器 M_j 的负载最大, 记作 $t(M_j)$. 又设 b 是最后被分配给机器 M_j 的作业. 根据算法, 在考虑分配 b 时 M_j 的负载最

小, 故
$$t(M_j) - t(b) \leq \frac{1}{m} \left(\sum_{a \in A} t(a) - t(b) \right).$$

证明

于是

$$\begin{aligned}\mathbf{G-MPS}(I) = t(M_j) &\leq \frac{1}{m} \left(\sum_{a \in A} t(a) - t(b) \right) + t(b) \\ &= \frac{1}{m} \sum_{a \in A} t(a) + \left(1 - \frac{1}{m} \right) t(b) \\ &\leq \mathbf{OPT}(I) + \left(1 - \frac{1}{m} \right) \mathbf{OPT}(I) \\ &= \left(2 - \frac{1}{m} \right) \mathbf{OPT}(I).\end{aligned}$$

紧实例

m 台机器, $m(m-1)+1$ 项作业, 前 $m(m-1)$ 项作业的处理时间都为 1, 最后一项作业的处理时间为 m .

算法方案: 把前 $m(m-1)$ 项作业平均地分配给 m 台机器, 每台 $m-1$ 项, 最后一项任意地分配给一台机器.

$$\text{G-MPS}(I) = 2m-1.$$

最优方案: 把前 $m(m-1)$ 项作业平均地分配给 $m-1$ 台机器, 每台 m 项, 最后一项分配给留下的机器, $\text{OPT}(I)=m$.

由于 m 可以任意大, G-MPS 是 2-近似算法.

10.2.2 改进的贪心近似算法

递降贪心法 DG-MPS: 首先按处理时间从大到小重新排列作业, 然后运用 G-MPS.

例如 对上一小节的紧实例得到最优解.

对前面的实例, 计算结果如下: 先重新排序 8, 6, 5, 4, 4, 3, 3, 3; 3台机器的负载分别为 $8+3=11$, $6+4+3=13$, $5+4+3=12$. 比 G-MPS的结果好.

与 G-MPS相比, DG-MPS 仅增加对作业的排序, 需要增加时间 $O(n\log n)$, 仍然是多项式时间的.

定理10.2 对多机调度问题的每一个有 m 台机器的实例 I ,

$$\text{DG-MPS}(I) \leq \frac{1}{m} \left(\frac{3}{2} - \frac{1}{2m} \right) \text{OPT}(I)$$

证明

证 设作业按处理时间从大到小排列为 $a_1, a_2, \dots, a_n$, 仍考虑负载最大的机器 M_j 和最后分配给 M_j 的作业 a_i .

(1) M_j 只有一个作业, 则 $i=1$, 必为最优解.

(2) M_j 有 2 个或 2 个以上作业, 则 $i \geq m+1$, $\text{OPT}(I) \geq 2t(a_i)$.

$$\begin{aligned}\text{DG} - \text{MPS}(I) &= t(M_j) \leq \frac{1}{m} \left(\sum_{k=1}^n t(a_k) - t(a_i) \right) + t(a_i) \\ &= \frac{1}{m} \sum_{k=1}^n t(a_k) + \left(1 - \frac{1}{m} \right) t(a_i) \leq \text{OPT}(I) + \left(1 - \frac{1}{m} \right) \cdot \frac{1}{2} \text{OPT}(I) \\ &= \left(\frac{3}{2} - \frac{1}{2m} \right) \text{OPT}(I)\end{aligned}$$

二元前缀码

- 二元前缀码

用0-1字符串作为代码表示字符，要求任何字符的代码都不能作为其它字符代码的前缀

- 非前缀码的例子

$a: 001, \quad b: 00, \quad c: 010, \quad d: 01$

- 解码的歧义，例如字符串 0100001

解码1: 01, 00, 001 d, b, a

解码2: 010, 00, 01 c, b, d

前缀码的二叉树表示

前缀码：

$$\{00000, 00001, 0001, 001, 01, 100, 101, 11\}$$

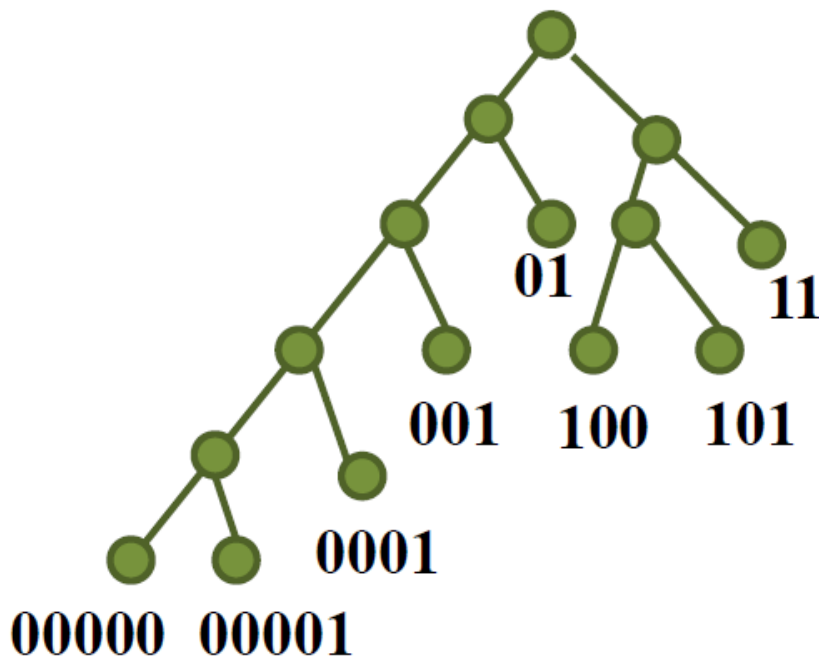
构造树:

0-左子树

1-右子树

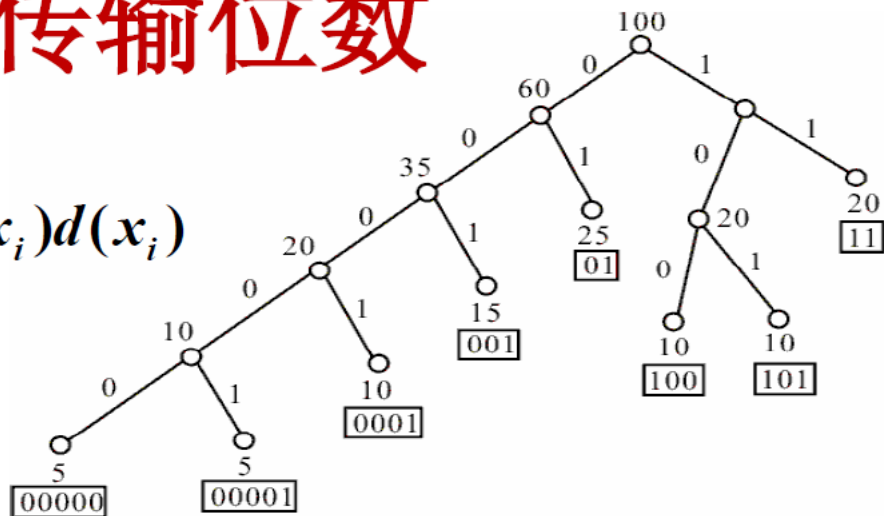
码对应一片树叶

最大位数为树深



平均传输位数

$$B = \sum_{i=1}^n f(x_i) d(x_i)$$



$$B = [(5+5) \times 5 + 10 \times 4 + (15+10+10) \times 3 + (25+20) \times 2] / 100 = 2.85$$

问题：给定字符集 $C = \{x_1, x_2, \dots, x_n\}$ 和每个字符的频率 $f(x_i)$, $i=1, 2, \dots, n$. 求关于 C 的一个**最优前缀码**(平均传输位数最小).

哈夫曼树算法伪码

算法 Huffman(C)

输入: $C = \{x_1, x_2, \dots, x_n\}, f(x_i), i=1, 2, \dots, n.$

输出: Q //队列

1. $n \leftarrow |C|$
2. $Q \leftarrow C$ //频率递增队列 Q
3. for $i \leftarrow 1$ to $n-1$ do
4. $z \leftarrow \text{Allocate-Node}()$ //生成结点 z
5. $z.\text{left} \leftarrow Q$ 中最小元 //最小作 z 左儿子
6. $z.\text{right} \leftarrow Q$ 中最小元 //最小作 z 右儿子
7. $f(z) \leftarrow f(x) + f(y)$
8. Insert(Q, z) // 将 z 插入 Q
9. return Q

实例

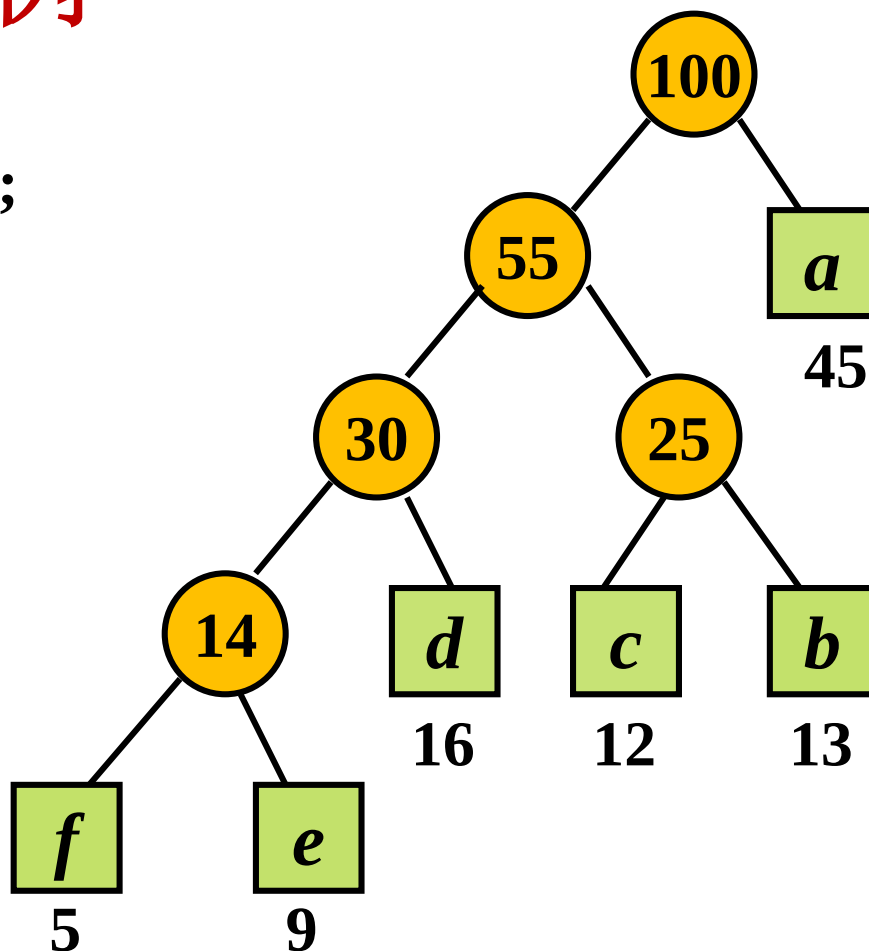
例如 $a: 45, b: 13; c: 12;$
 $d: 16; e: 9; f: 5$

编码:

$f--0000, e--0001,$
 $d--001, c--010,$
 $b--011, a--1$

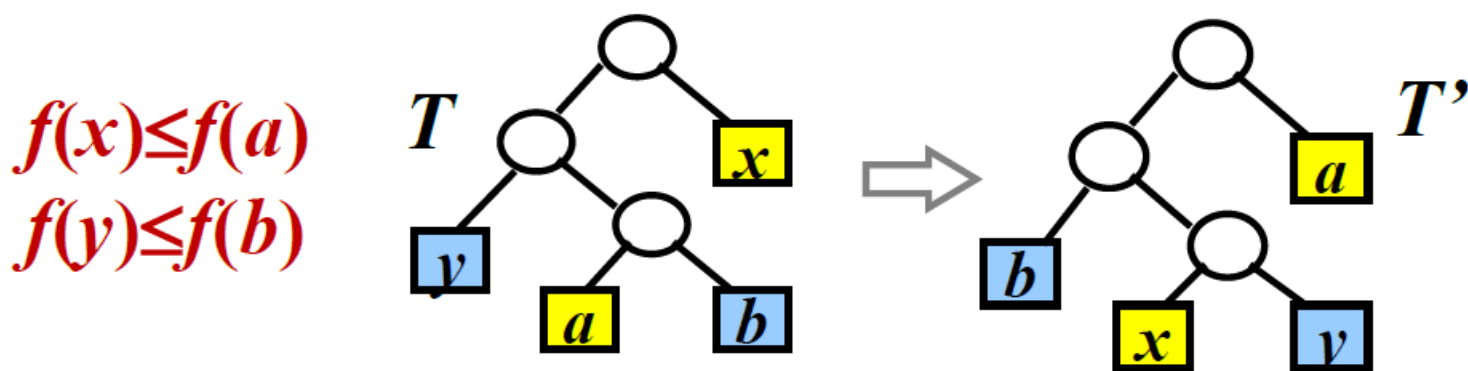
平均位数:

$$4 \times (0.05 + 0.09) \\ + 3 \times (0.16 + 0.12 + 0.13) + 1 \times 0.45 = 2.24$$



最优前缀码性质：引理1

引理1: C 是字符集, $\forall c \in C, f(c)$ 为频率, $x, y \in C$, $f(x), f(y)$ 频率最小, 那么存在最优二元前缀码使得 x, y 码字等长且仅在最后一位不同.



$$B(T) - B(T') = \sum_{i \in C} f[i]d_T(i) - \sum_{i \in C} f[i]d_{T'}(i) \geq 0$$

其中 $d_T(i)$ 为 i 在 T 中的层数 (i 到根的距离)

引理2

引理 设 T 是二元前缀码的二叉树, $\forall x, y \in T$, x, y 是树叶兄弟, z 是 x, y 的父亲, 令

$$T' = T - \{x, y\}$$

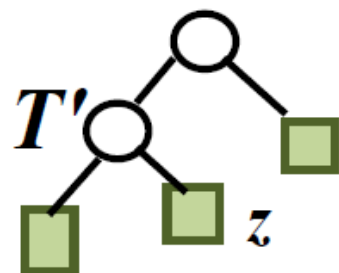
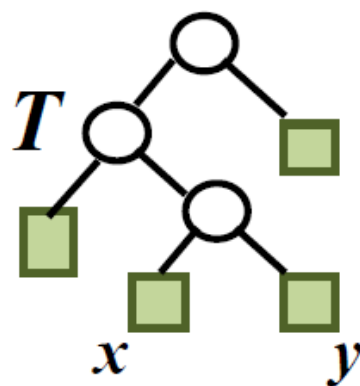
且令 z 的频率

$$f(z) = f(x) + f(y)$$

T' 是对应二元前缀码

$C' = (C - \{x, y\}) \cup \{z\}$
的二叉树, 那么

$$B(T) = B(T') + f(x) + f(y)$$



引理2证明

证 $\forall c \in C - \{x, y\}$, 有

$$d_T(c) = d_{T'}(c) \Rightarrow f(c)d_T(c) = f(c)d_{T'}(c)$$

$$d_T(x) = d_T(y) = d_{T'}(z) + 1$$

$$B(T) = \sum_{i \in T} f(i)d_T(i)$$

$$= \sum_{i \in T, i \neq x, y} f(i)d_T(i) + f(x)d_T(x) + f(y)d_T(y)$$

$$= \sum_{i \in T', i \neq z} f(i)d_{T'}(i) + f(z)d_{T'}(z) + (f(x) + f(y))$$

$$= B(T') + f(x) + f(y)$$

算法正确性证明思路

定理 Huffman 算法对任意规模为 n ($n \geq 2$) 的字符集 C 都得到关于 C 的最优前缀码的二叉树.

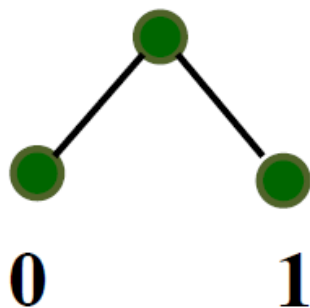
归纳基础 证明: 对于 $n=2$ 的字符集, Huffman算法得到最优前缀码.

归纳步骤 证明: 假设Huffman算法对于规模为 k 的字符集都得到最优前缀码, 那么对于规模为 $k+1$ 的字符集也得到最优前缀码.

归纳基础

$n=2$, 字符集 $C=\{x_1, x_2\}$,

对任何代码的字符至少都需要1位二进制数字. **Huffman**算法得到的代码是 0 和 1, 是最优前缀码.



归纳步骤

假设Huffman算法对于规模为 k 的字符集都得到最优前缀码. 考虑规模为 $k+1$ 的字符集

$$C = \{x_1, x_2, \dots, x_{k+1}\},$$

其中 $x_1, x_2 \in C$ 是频率最小的两个字符.

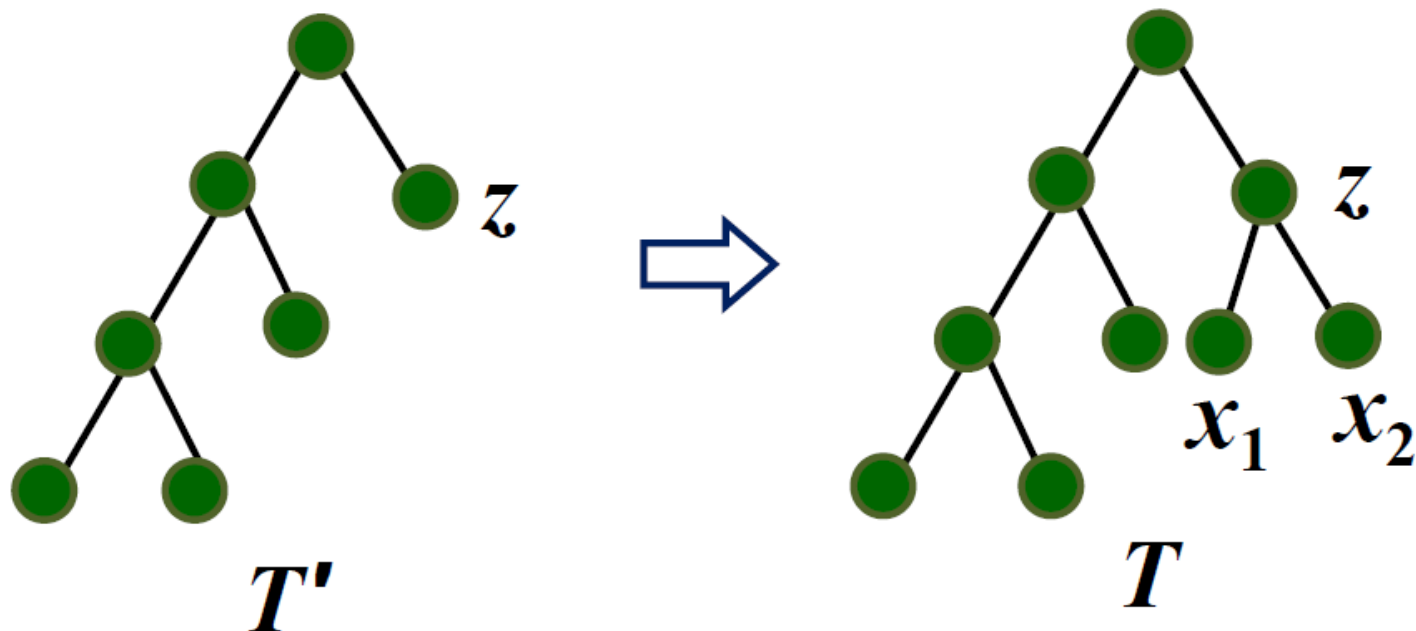
令

$$C' = (C - \{x_1, x_2\}) \cup \{z\},$$
$$f(z) = f(x_1) + f(x_2)$$

根据归纳假设, 算法得到一棵关于字符集 C' , 频率 $f(z)$ 和 $f(x_i)$ ($i=3, 4, \dots, k+1$) 的最优前缀码的二叉树 T' .

归纳步骤(续)

把 x_1, x_2 作为 z 的儿子附到 T' 上, 得到树 T , 那么 T 是关于 $C=(C'-\{z\})\cup\{x_1, x_2\}$ 的最优前缀码的二叉树.



归纳步骤 (续)

如若不然, 存在更优树 T^* , $B(T^*) < B(T)$,
且由引理1, 其树叶兄弟是 x_1 和 x_2 .

去掉 T^* 中 x_1 和 x_2 , 得到 $T^{*'}.$ 根据引理2

$$\begin{aligned} B(T^{*'}) &= B(T^*) - \underline{(f(x_1) + f(x_2))} \\ &< B(T) - \underline{(f(x_1) + f(x_2))} \\ &= B(T') \end{aligned}$$

与 T' 是一棵关于 C' 的最优前缀码的二叉树矛盾.

应用：文件归并

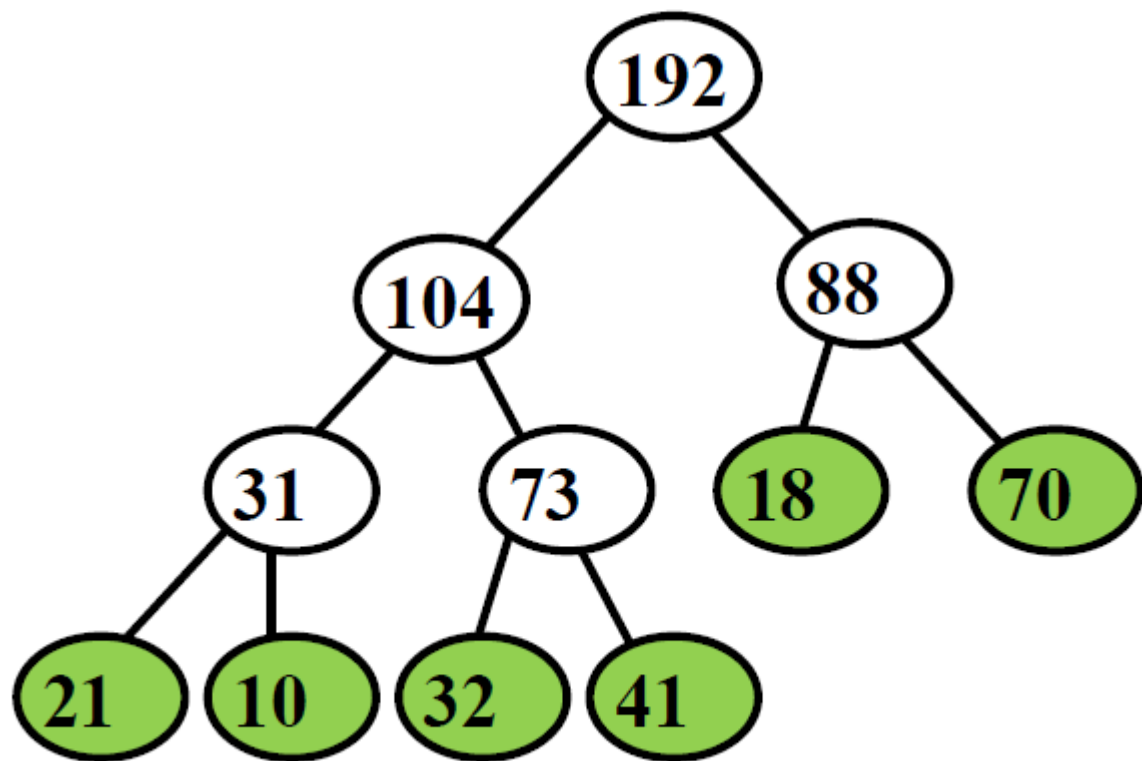
问题：给定一组不同长度的排好序文件构成的集合 $S = \{f_1, \dots, f_n\}$ ，其中 f_i 表示第 i 个文件含有的项数. 使用二分归并将这些文件归并成一个有序文件.

归并过程对应于二叉树：

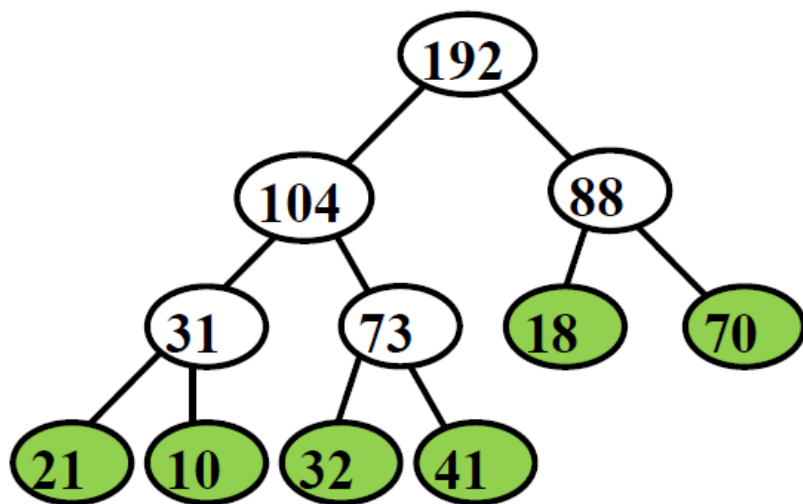
文件为树叶. f_i 与 f_j 归并的文件是它们的父结点.

两两顺序归并

实例： $S = \{ 21, 10, 32, 41, 18, 70 \}$



归并代价



$$(1) (21+10-1)+(32+41-1)+(18+70-1)+ \\ (31+73-1)+(104+88-1) = 483$$

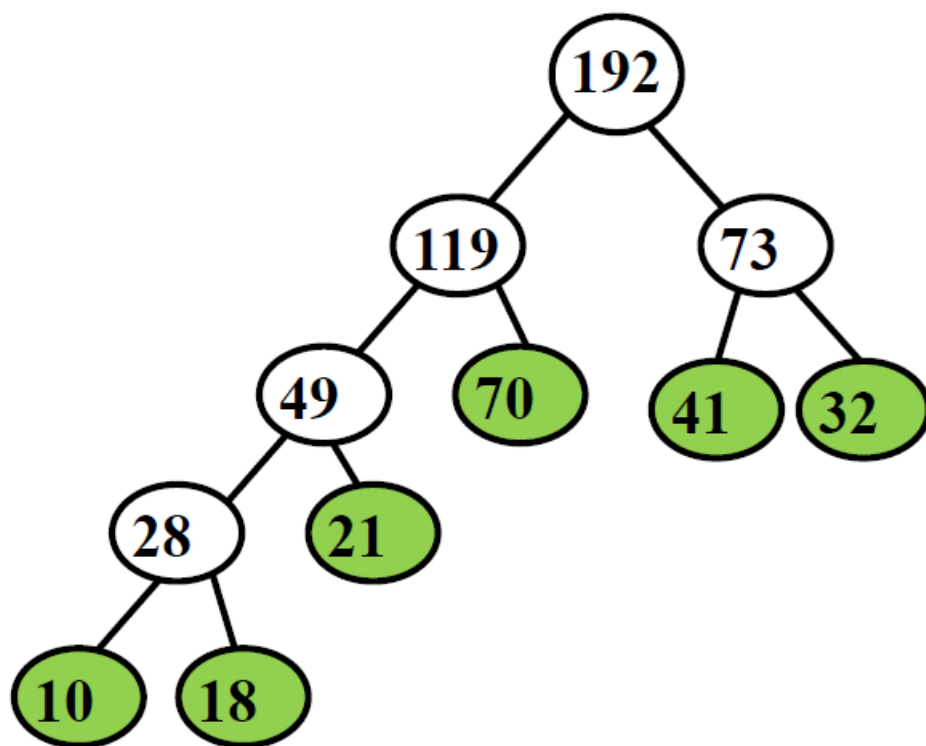
$$(2) (21+10+32+41) \times 3 + (18+70) \times 2 - 5 = 483$$

代价计算公式

$$\sum_{i \in S} d(i) f_i - (n-1)$$

实例：Huffman树归并

输入： $S=\{21,10,32,41,18,70\}$



代价： $(10+18) \times 4 + 21 \times 3 + (70+41+32) \times 2 - 5 = 456$

最小生成树

无向连通带权图

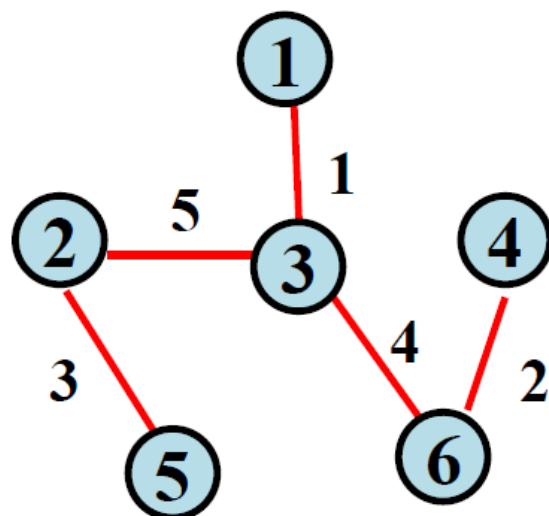
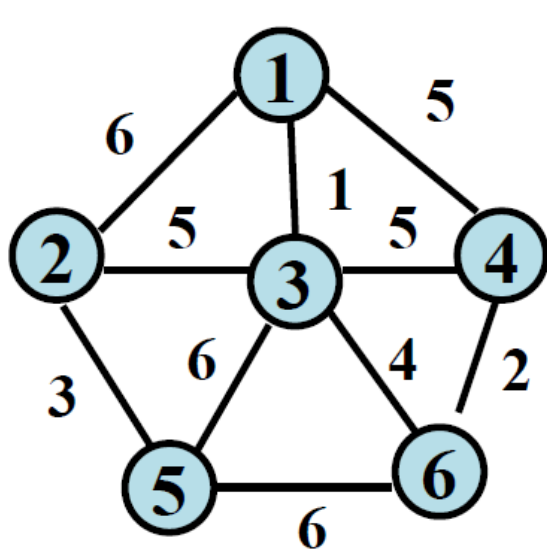
$$G = (V, E, W),$$

其中 $w(e) \in W$ 是边 e 的权.

G 的一棵生成树 T 是包含了 G 的所有顶点的树, 树中各边的权之和 $W(T)$ 称为树的权, 具有最小权的生成树称为 G 的最小生成树.

最小生成树的实例

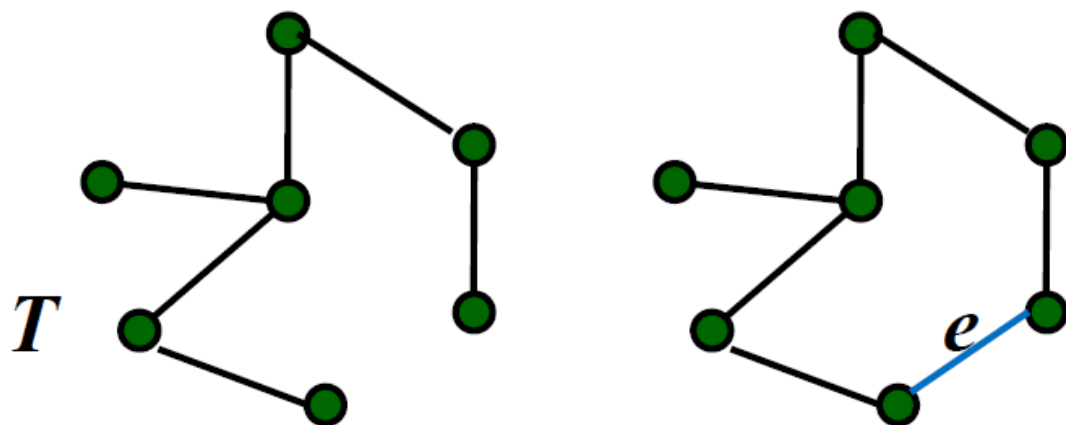
$G=(V,E,W), V=\{1,2,3,4,5,6\}, W$ 如图所示
 $E = \{\{1,2\},\{1,3\},\{1,4\},\{2,3\},\{2,5\},$
 $\{3,4\},\{3,5\},\{3,6\},\{4,6\},\{5,6\}\}$



生成树的性质

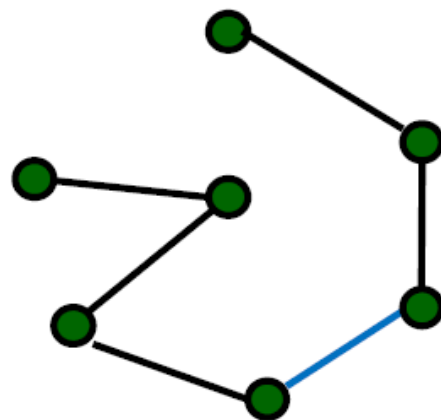
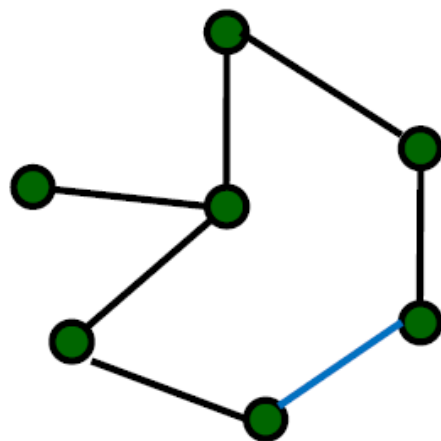
命题1 设 G 是 n 阶连通图, 那么

- (1) T 是 G 的生成树当且仅当 T 无圈且有 $n-1$ 条边.
- (2) 如果 T 是 G 的生成树, $e \notin T$, 那么 $T \cup \{e\}$ 含有一个圈 C (回路).



生成树的性质（续）

(3) 去掉圈 C 的任意一条边，就得到 G 的另外一棵生成树 T' 。



T'

生成树性质的应用

- 算法步骤：选择边。
约束条件：不形成回路
截止条件：边数达到 $n-1$.
- 改进生成树 T 的方法
在 T 中加一条非树边 e , 形成回路 C , 在 C 中去掉一条树边 e_i , 形成一棵新的生成树 T'
$$W(T') - W(T) = W(e) - W(e_i)$$

若 $W(e) \leq W(e_i)$, 则 $W(T') \leq W(T)$

求最小生成树

问题:

给定连通带权图 $G = (V, E, W)$,
 $w(e) \in W$ 是边 e 的权. 求 G 的一棵最小生成树.

贪心法:

Prim 算法,
Kruskal 算法

生成树在网络中有着重要应用

Prim(普里姆)算法设计思想

输入：图 $G=(V,E,W)$, $V=\{1,2,\dots,n\}$

输出：最小生成树 T

设计思想：

初始 $S = \{1\}$,

选择连接 S 与 $V-S$ 集合的最短边 $e = \{i,j\}$, 其中 $i \in S, j \in V-S$. 将 e 加入树 T , j 加入 S .

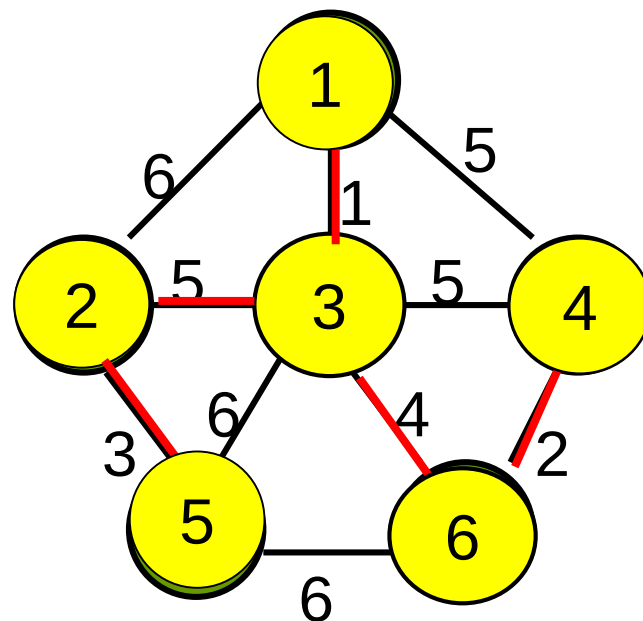
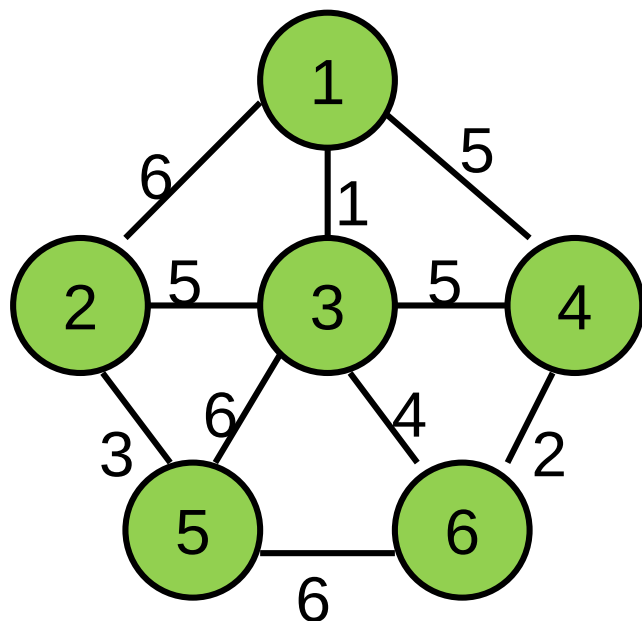
继续执行上述过程, 直到 $S=V$ 为止.

算法4.5 Prim

输入：连通图 $G = \langle V, E, W \rangle$

输出： G 的最小生成树 T

1. $S \leftarrow \{1\}; T \leftarrow \emptyset$
2. **while** $V - S \neq \emptyset$ **do**
3. 从 $V - S$ 中选择 j 使得 j 到 S 中顶点的边 e 的权最小
4. $S \leftarrow S \cup \{j\}, T \leftarrow T \cup \{e\}$



Prim算法

正确性证明: 归纳法

命题: 对于任意 $k < n$, 存在一棵最小生成树包含算法前 k 步选择的边.

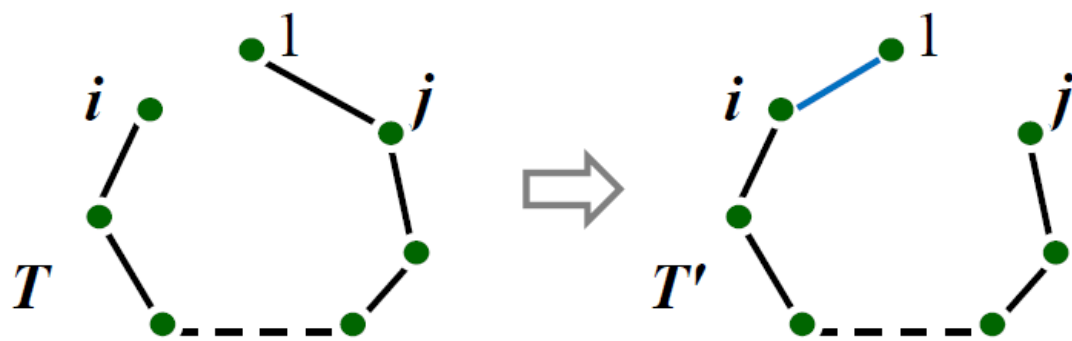
归纳基础: $k = 1$, 存在一棵最小生成树 T 包含边 $e = \{1, i\}$, 其中 $\{1, i\}$ 是所有关联 1 的边中权最小的.

归纳步骤: 假设算法前 k 步选择的边构成一棵最小生成树的边, 则算法前 $k+1$ 步选择的边也构成一棵最小生成树的边.

归纳基础

证明： 存在一棵最小生成树 T 包含关联结点1的最小权的边 $e=\{1,i\}$.

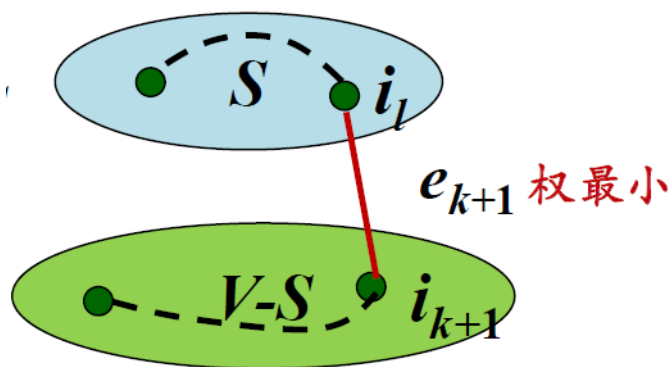
证 设 T 为一棵最小生成树，假设 T 不包含 $\{1,i\}$ ，则 $T \cup \{1,i\}$ 含有一条回路，回路中关联1的另一条边 $\{1,j\}$. 用 $\{1,i\}$ 替换 $\{1,j\}$ 得到树 T' ，则 T' 也是生成树，且 $W(T') \leq W(T)$.



归纳步骤

假设算法进行了 k 步，生成树的边为 $e_1, e_2, \dots, e_k$ ，这些边的端点构成集合 S 。由归纳假设存在 G 的一棵最小生成树 T 包含这些边。

算法第 $k+1$ 步选择顶点 i_{k+1} ，则 i_{k+1} 到 S 中顶点边权最小，设此边 $e_{k+1} = \{i_{k+1}, i_l\}$ 。若 $e_{k+1} \in T$ ，算法 $k+1$ 步显然正确。



归纳步骤（续）

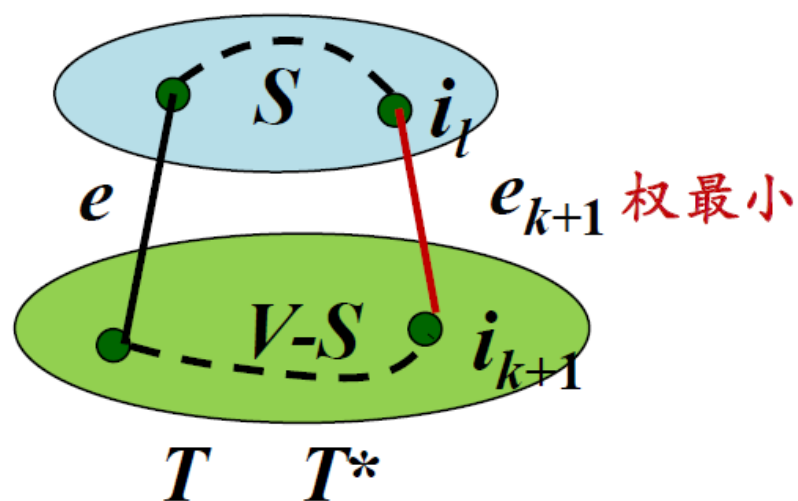
假设 T 不含有 e_{k+1} ，则将 e_{k+1} 加到 T 中形成一条回路。这条回路有另外一条连接 S 与 $V-S$ 中顶点的边 e ，

令 $T^* = (T - \{e\}) \cup \{e_{k+1}\}$

则 T^* 是 G 的一棵生成树，包含 $e_1, e_2, \dots, e_{k+1}$ ，且

$$W(T^*) \leq W(T)$$

算法到 $k+1$ 步仍得到最小生成树。



时间复杂度

算法步骤执行 $O(n)$ 次

每次执行 $O(n)$ 时间：

找连接 S 与 $V-S$ 的最短边

算法时间： $T(n) = O(n^2)$

Kruskal(克鲁斯卡尔)算法设计思想

输入：图 $G=(V,E,W)$, $V=\{1,2,\dots,n\}$

输出： G 的最小生成树 T

设计思想：

- (1) 按照长度从小到大对边排序.
- (2) 依次考察当前最短边 e ，如果 e 与 T 的边不构成回路，则把 e 加入树 T ，否则跳过 e . 直到选择了 $n-1$ 条边为止.

Kruskal算法将这 n 个顶点看成是 n 个孤立的连通分支

每一个顶点初始化为孤立的连通分支,对应并查集中的一个子集合

用并查集来实现连通分支找和合的相关操作

伪码

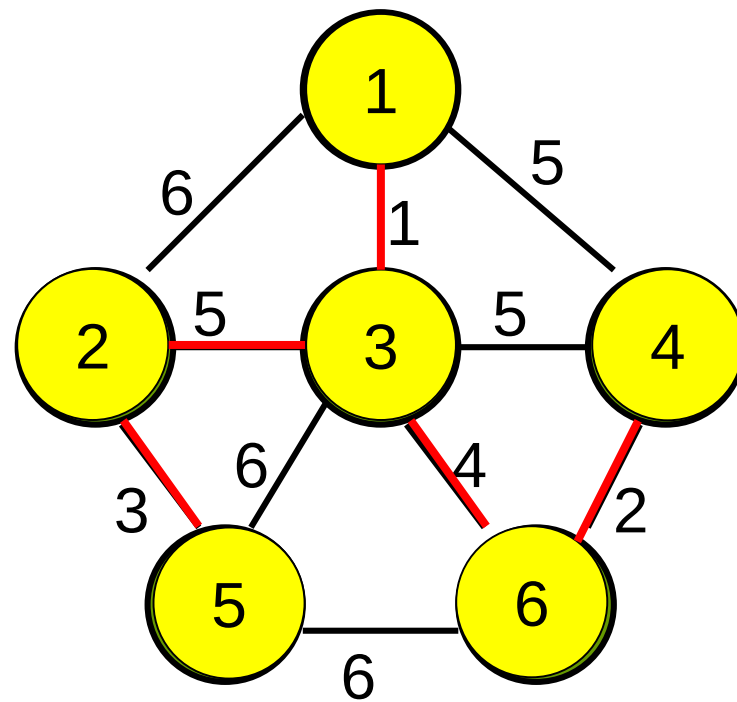
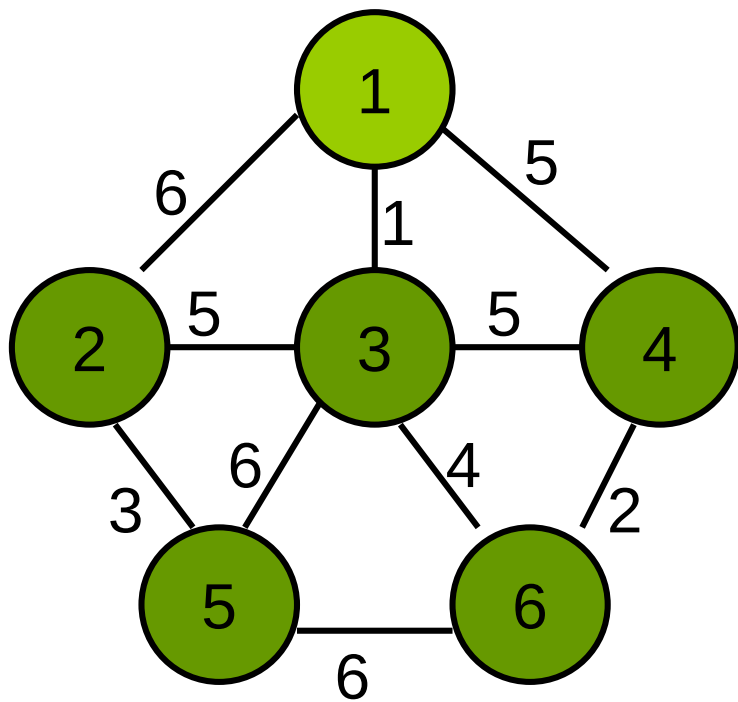
算法 Kruskal

输入：连通图 G // 顶点数 n ，边数 m

输出： G 的最小生成树

1. 权从小到大排序 E 的边, $E=\{e_1, e_2, \dots, e_m\}$
2. $T \leftarrow \emptyset$
3. repeat
4. $e \leftarrow E$ 中的最短边
5. if e 的两端点不在同一连通分支
6. then $T \leftarrow T \cup \{e\}$
7. $E \leftarrow E - \{e\}$
8. until T 包含了 $n-1$ 条边

实例



正确性证明思路

命题：对于任意 n , 算法对 n 阶图找到一棵最小生成树.

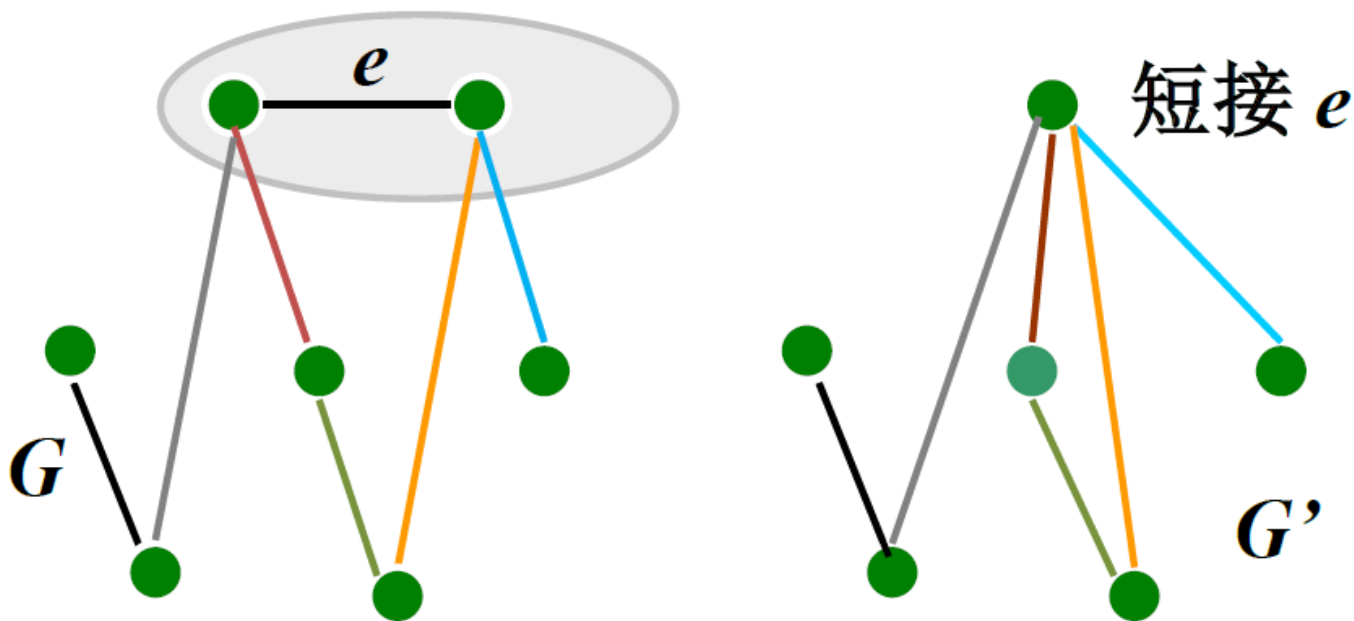
证明思路：

归纳基础 证明： $n = 2$, 算法正确.
 G 只有一条边，最小生成树就是 G .

归纳步骤 证明：假设算法对于 n 阶图是正确的，其中 $n > 1$ ，则对于任何 $n+1$ 阶图算法也得到一棵最小生成树.

短接操作

任给 $n+1$ 个顶点的图 G , G 中最小权边 $e = \{i, j\}$, 从 G 中短接 i 和 j , 得到图 G' .



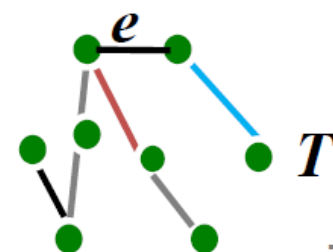
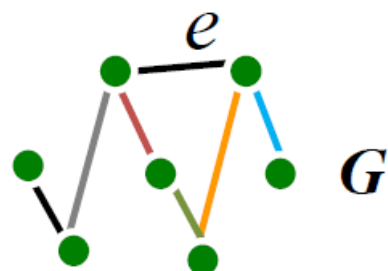
归纳步骤证明

对于任意 $n+1$ 阶图 G 短接最短边 e , 得到 n 阶图 G'

根据归纳假设算法得到 G' 的最小生成树 T'

将被短接的边 e “拉伸”回到原来长度, 得到树 T

证明 T 是 G 的最小生成树



T 是 G 的最小生成树

$T = T' \cup \{e\}$ 是关于 G 的最小生成树.

否则存在 G 的含边 e 的最小生成树 T^* , $W(T^*) < W(T)$. (如果 $e \notin T^*$, 在 T^* 中加边 e , 形成回路. 去掉回路中任意别的边所得生成树的权仍旧最小).

在 T^* 短接 e 得到 G' 的生成树 $T^* - \{e\}$,

$$\begin{aligned} W(T^* - \{e\}) &= W(T^*) - w(e) \\ &< W(T) - w(e) = W(T') \end{aligned}$$

与 T' 的最优性矛盾.

算法实现与时间复杂度

建立**FIND**数组，**FIND**[i] 是结点 i 的连通分支标记。

(1) 初始 **FIND**[i] = i .

(2) 连通分支合并，较小分支标记更新为较大分支标记

每个结点至多更新 $\log n$ 次，

建立和更新 **FIND**数组: $O(n \log n)$

时间: $O(m \log m) + O(n \log n) + O(m)$
 $= O(m \log n)$

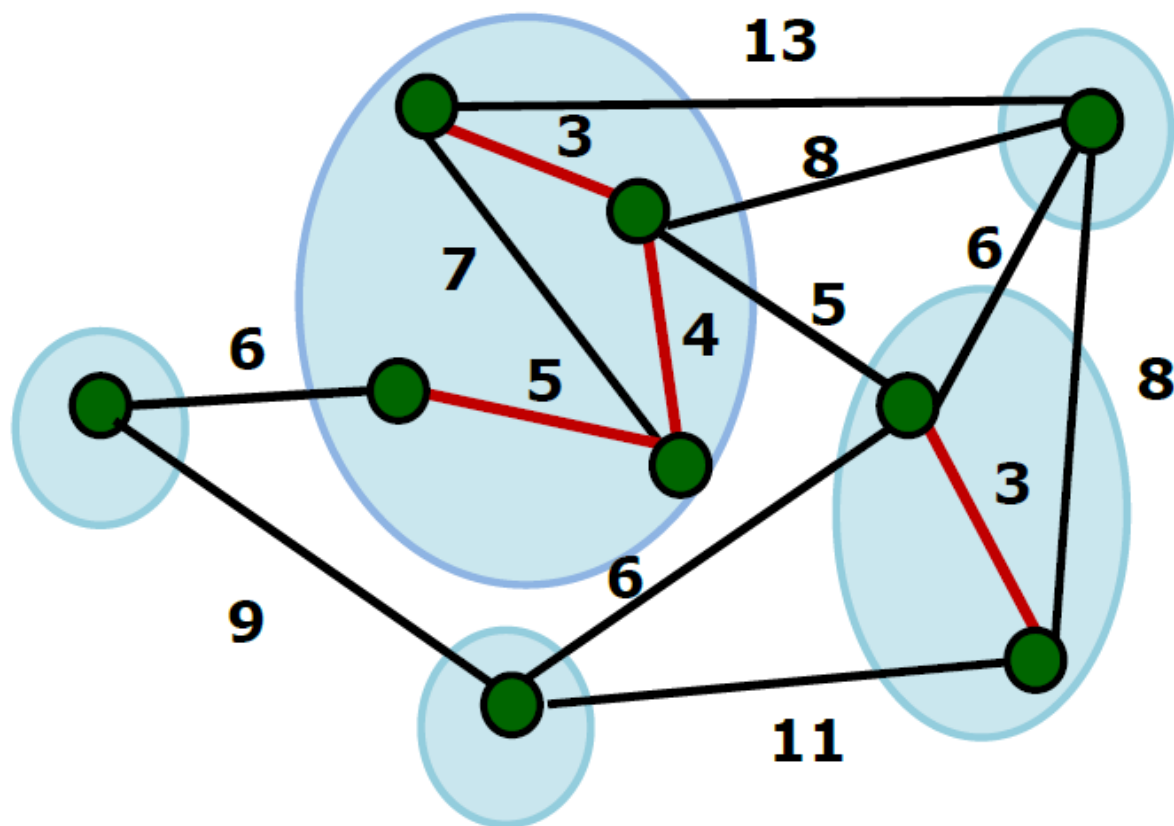
应用：数据分组问题

一组数据(照片, 文件, 生物标本)要把它们按照相关性进行分类.

用相似度函数或“距离”来描述个体之间的差距.

如果分成5类, 使得每类内部的个体尽可能相近, 不同类之间的个体尽可能地“远离”. 如何划分?

应用：数据分组问题



单链聚类

类似于Kruskal算法

- (1) 按照边长从小到大对边排序
- (2) 依次考察当前最短边 e , 如果 e 与 已经选中的边不构成回路, 则把 e 加入集合, 否则跳过 e . 计数图的连通分支个数.
- (3) 直到保留了 k 个连通分支为止.

贪心法小结

- 贪心法适用于组合优化问题.
- 求解过程是多步判断过程，最终的判断序列对应于问题的最优解.
- 判断依据某种“短视的”贪心选择性质，性质的好坏决定了算法的正确性. 贪心性质的选择往往依赖于直觉或者经验.

贪心法小结（续）

- 贪心法正确性证明方法：
 - (1) 直接计算优化函数，贪心法的解恰好取得最优值
 - (2) 数学归纳法（对算法步数或者问题规模归纳）
 - (3) 交换论证
- 证明贪心策略不对：举反例

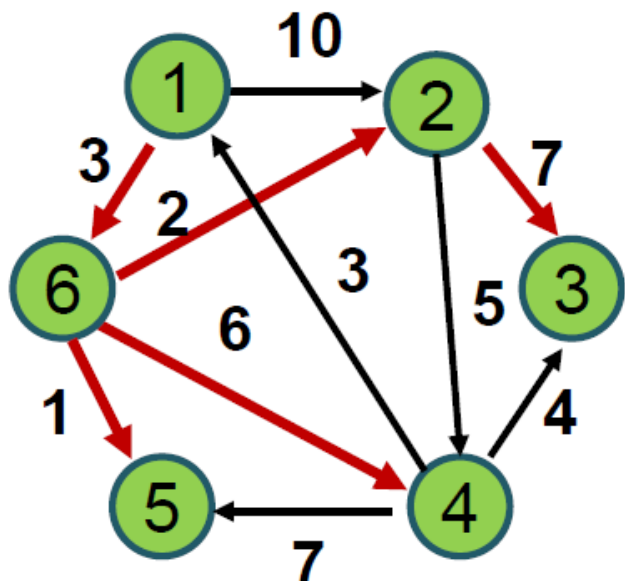
贪心法小结（续）

- 对于某些不能保证对所有的实例都得到最优解的贪心算法（近似算法），可做参数化分析或者误差分析.
- 贪心法的优势：算法简单，时间和空间复杂性低

单源最短路问题

给定带权有向网络 $G=(V,E,W)$ ，每条边 $e=\langle i,j \rangle$ 的权 $w(e)$ 为非负实数，表示从 i 到 j 的距离. **源点** $s \in V$.

求: 从 s 出发到达其它结点的最短路径.



源点: 1

$1 \rightarrow 6 \rightarrow 2$: $short[2] = 5$

$1 \rightarrow 6 \rightarrow 2 \rightarrow 3$: $short[3] = 12$

$1 \rightarrow 6 \rightarrow 4$: $short[4] = 9$

$1 \rightarrow 6 \rightarrow 5$: $short[5] = 4$

$1 \rightarrow 6$: $short[6] = 3$

Dijkstra算法有关概念

$x \in S \Leftrightarrow x \in V$ 且从 s 到 x 的最短路径已经找到

初始: $S = \{s\}$, $S = V$ 时算法结束

从 s 到 u 相对于 S 的最短路径: 从 s 到 u 且仅经过 S 中顶点的最短路径

$dist[u]$: 从 s 到 u 相对 S 最短路径的长度

$short[u]$: 从 s 到 u 的最短路径的长度

$dist[u] \geq short[u]$

算法设计思想

输入：有向图 $G = (V, E, W)$,

$V = \{ 1, 2, \dots, n \}$, $s = 1$

输出：从 s 到每个顶点的最短路径

1. 初始 $S = \{1\}$
2. 对于 $i \in V - S$, 计算1到 i 的相对 S 的最短路, 长度 $dist[i]$
3. 选择 $V - S$ 中 $dist$ 值最小的 j , 将 j 加入 S , 修改 $V - S$ 中顶点的 $dist$ 值.
4. 继续上述过程, 直到 $S = V$ 为止.

伪码

算法 Dijkstra

1. $S \leftarrow \{ s \}$
2. $dist[s] \leftarrow 0$
3. for $i \in V - \{ s \}$ do
4. $dist[i] \leftarrow w(s,i)$ // s 到 i 没边, $w(s,i)=\infty$
5. while $V - S \neq \emptyset$ do
6. 从 $V - S$ 取相对 S 的最短路径顶点 j
7. $S \leftarrow S \cup \{ j \}$
8. for $i \in V - S$ do
9. if $dist[j] + w(j,i) < dist[i]$
10. then $dist[i] \leftarrow dist[j] + w(j,i)$



更新
dist值

运行实例

输入: $G=<V,E,W>$, 源点 1
 $V=\{1,2,3,4,5,6\}$

$S=\{1\}$,

$dist[1]=0$

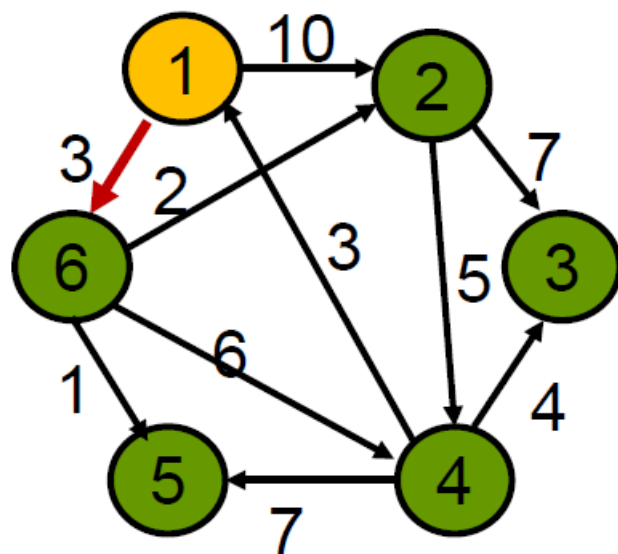
$dist[2]=10$

$dist[6]=3$

$dist[3]=\infty$

$dist[4]=\infty$

$dist[5]=\infty$



实例（续）

$$S=\{1,6\}$$

$$\textit{dist}[1] = 0$$

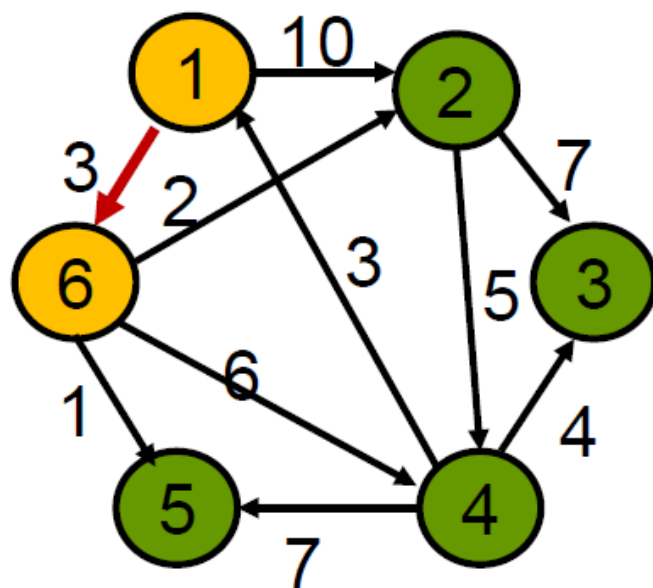
$$\textit{dist}[6] = 3$$

$$\textit{dist}[2] = 5$$

$$\textit{dist}[4] = 9$$

$$\textit{dist}[\underline{5}] = 4$$

$$\textit{dist}[3] = \infty$$



运行实例（续）

$S=\{1,6,5\}$

$dist[1]=0$

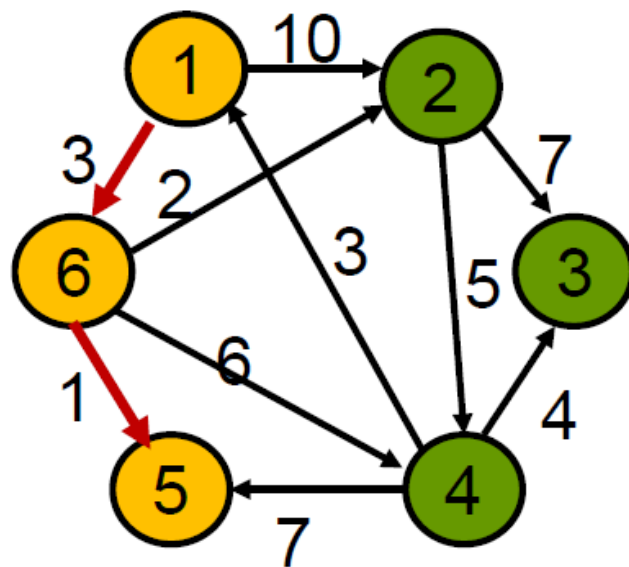
$dist[6]=3$

$dist[5]=4$

$dist[2]=5$

$dist[4]=9$

$dist[3]=\infty$



运行实例（续）

$S = \{1, 6, 5, 2\}$

$dist[1] = 0$

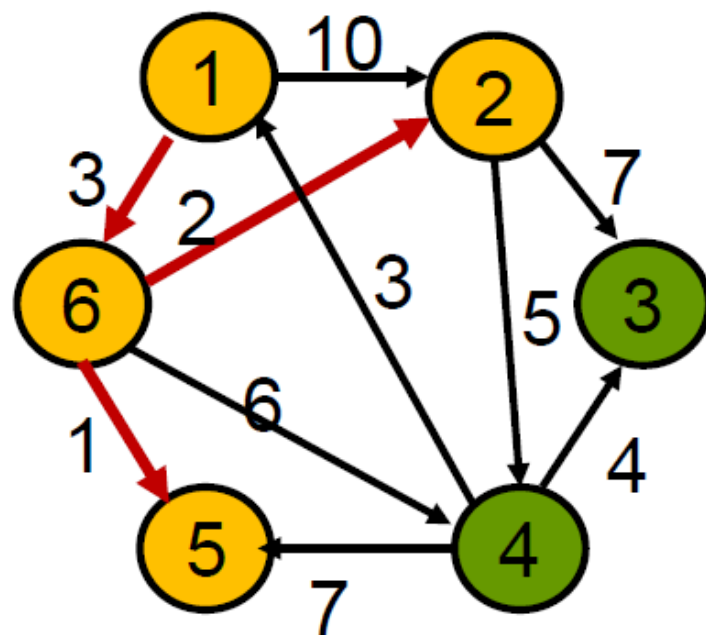
$dist[6] = 3$

$dist[5] = 4$

$dist[2] = 5$

$dist[4] = 9$

$dist[3] = 12$



运行实例（续）

$S=\{1,6,5,2,4\}$

$dist[1]=0$

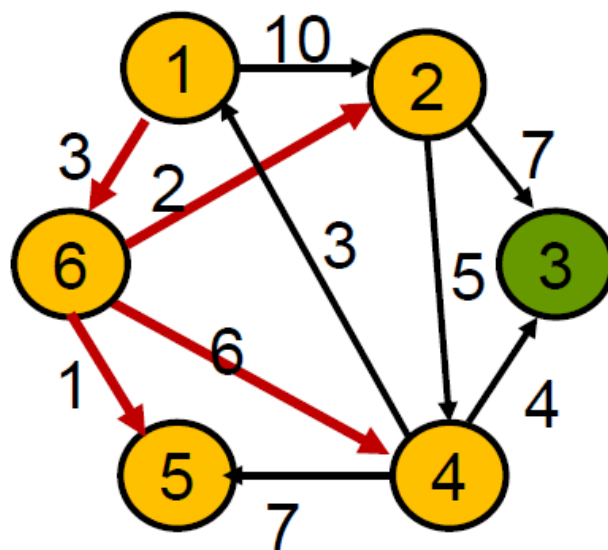
$dist[6]=3$

$dist[5]=4$

$dist[2]=5$

$dist[4]=9$

$dist[3]=12$



运行实例（续）

$S=\{1,6,5,2,4,3\}$

$dist[1]=0$

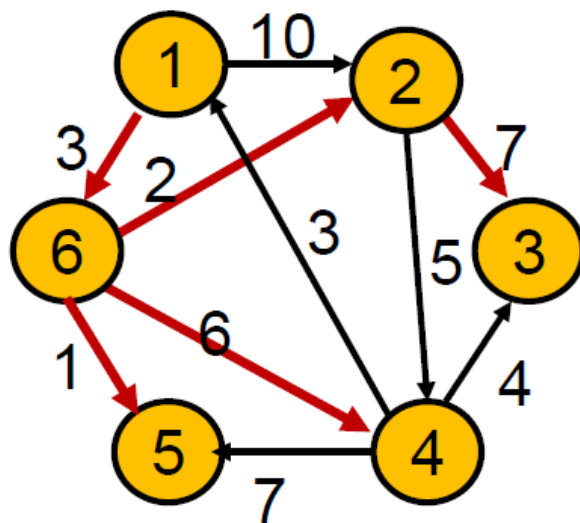
$dist[6]=3$

$dist[5]=4$

$dist[2]=5$

$dist[4]=9$

$dist[3]=12$



$short[1]=0, short[2]=5, short[3]=12,$
 $short[4]=9, short[5]=4, short[6]=3.$

归纳证明思路

命题：当算法进行到第 k 步时，对于 S 中每个结点 i ,

$$\textit{dist} [i] = \textit{short} [i]$$

归纳基础

$$k = 1, S = \{s\}, \textit{dist} [s] = \textit{short} [s] = 0.$$

归纳步骤

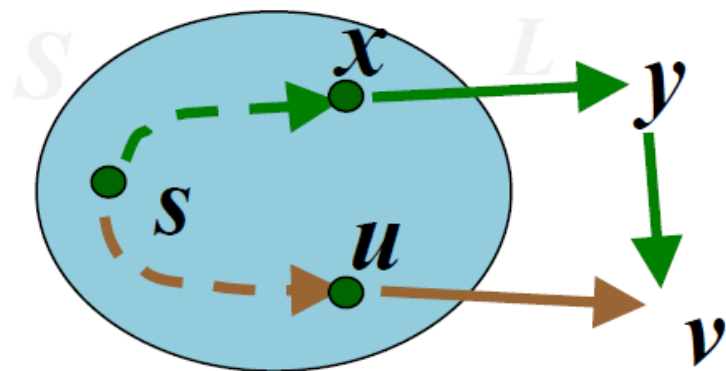
证明：假设命题对 k 为真，则对 $k+1$ 命题也为真.

归纳步骤证明

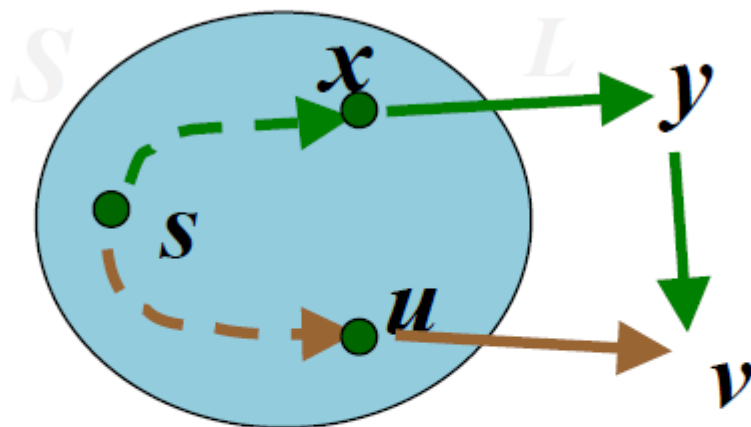
假设命题对 k 为真, 考虑 $k+1$ 步算法
选择顶点 v (边 $\langle u, v \rangle$). 需要证明

$$\text{dist}[v] = \text{short}[v]$$

若存在另一条 s - v 路径 L (绿色), 最后一次出 S 的顶点为 x , 经过 $V-S$ 的第一个顶点 y , 再由 y 经过一段在 $V-S$ 中的路径到达 v .



归纳步骤证明（续）



在 $k+1$ 步算法选择顶点 v ，而不是 y ，

$$\text{dist}[v] \leq \text{dist}[y]$$

令 y 到 v 的路径长度为 $d(y,v)$

$$\text{dist}[y] + d(y,v) = L$$

于是 $\text{dist}[v] \leq L$ ，即 $\text{dist}[v] = \text{short}[v]$

时间复杂度

- 时间复杂度: $O(n^2)$
算法进行 $n-1$ 步
每步更新 $dist$ 函数值并且挑选1个具有最小 $dist$ 函数值的结点进入 S , 需要 $O(n)$ 时间
- 选用基于堆实现的优先队列的数据结构, 可以将算法时间复杂度降到 $O(m \log n)$

Universal Optimality of Dijkstra via Beyond-Worst-Case Heaps*

Bernhard Haeupler
INSAIT, Sofia University
“St. Kliment Ohridski”
& ETH Zurich

Richard Hladík
INSAIT, Sofia University
“St. Kliment Ohridski”
& ETH Zurich

Václav Rozhoň
INSAIT, Sofia University
“St. Kliment Ohridski”

Robert E. Tarjan
Princeton University

Jakub Tětek
INSAIT, Sofia University
“St. Kliment Ohridski”

Best paper awards of IEEE Symposium on Foundations of Computer Science (FOCS) 2024

Abstract

This paper proves that Dijkstra’s shortest-path algorithm is universally optimal in both its running time and number of comparisons when combined with a sufficiently efficient heap data structure.

Universal optimality is a powerful beyond-worst-case performance guarantee for graph algorithms that informally states that a single algorithm performs as well as possible for every single graph topology. We give the first application of this notion to any sequential algorithm.

We design a new heap data structure with a working-set property guaranteeing that the heap takes advantage of locality in heap operations. Our heap matches the optimal (worst-case) bounds of Fibonacci heaps but also provides the beyond-worst-case guarantee that the cost of extracting the minimum element is merely logarithmic in the number of elements inserted after it instead of logarithmic in the number of all elements in the heap. This makes the extraction of recently added elements cheaper.

We prove that our working-set property guarantees universal optimality for the problem of ordering vertices by their distance from the source vertex: The sequence of heap operations generated by any run of Dijkstra’s algorithm on a fixed graph possesses enough locality that one can couple the number of comparisons performed by any heap with our working-set bound to the minimum number of comparisons required to solve the distance ordering problem on this graph for a worst-case choice of arc lengths.

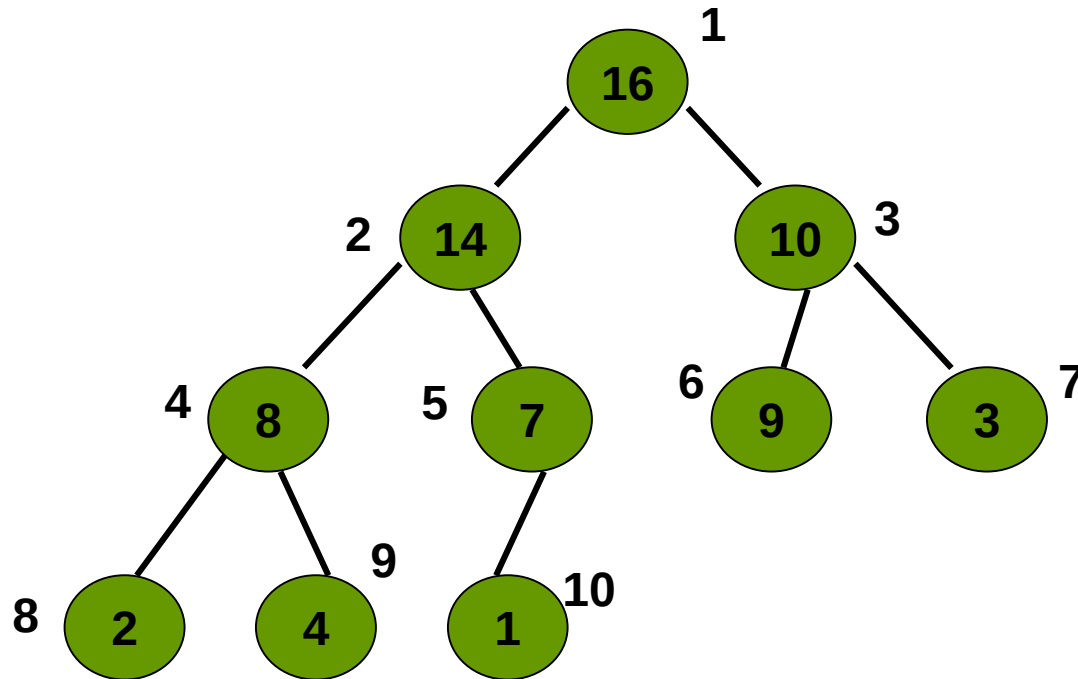
堆的定义

设 T 是一棵深度为 d 的二叉树，结点为 L 中的元素。
若满足以下条件，称作堆。

- (1) 所有内结点（可能一点除外）的度数为 2
- (2) 所有树叶至多在相邻的两层
- (3) $d-1$ 层的所有树叶在内结点的右边
- (4) $d-1$ 层最右边的内结点可能度数为 1（没有右儿子）
- (5) 每个结点的元素不小于儿子的元素

若只满足前(4)条，不满足第(5)条，称作堆结构（伪堆）

实例



堆存储在数组 A

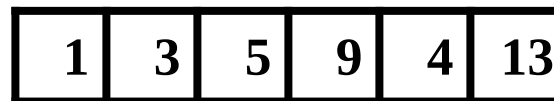
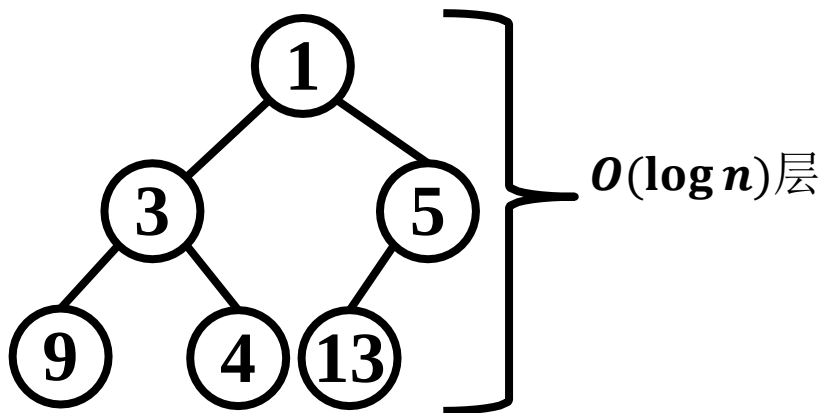
$A[i]$: 结点 i 的元素, 例如 $A[2]=14$.

$left(i)$, $right(i)$ 分别表示 i 的左儿子和右儿子

数据结构：优先队列

优先队列

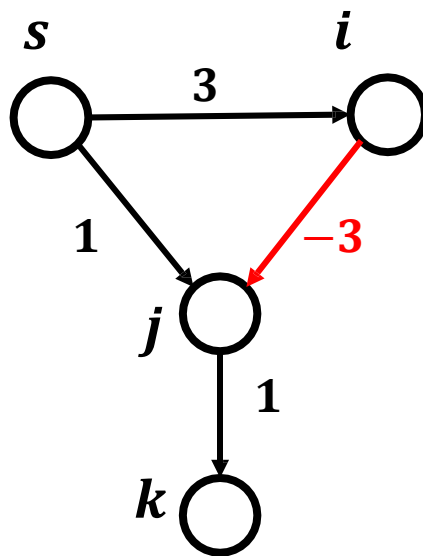
- 队列中每个元素有一个关键字，依据关键字大小离开队列
- 通过二叉堆来实现优先队列
 - ***Q.Insert()*** 时间复杂度 $O(\log n)$
 - ***Q.ExtractMin()*** 时间复杂度 $O(\log n)$
 - ***Q.DecreaseKey()*** 时间复杂度 $O(\log n)$



优先队列 Q

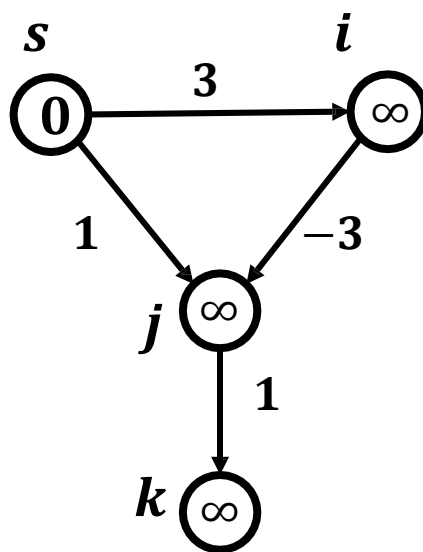
问题背景

- 图中存在负权边，**Dijkstra**算法不再适用



问题背景

- 图中存在负权边，**Dijkstra**算法不再适用



V_A 中顶点



$V - V_A$ 中顶点

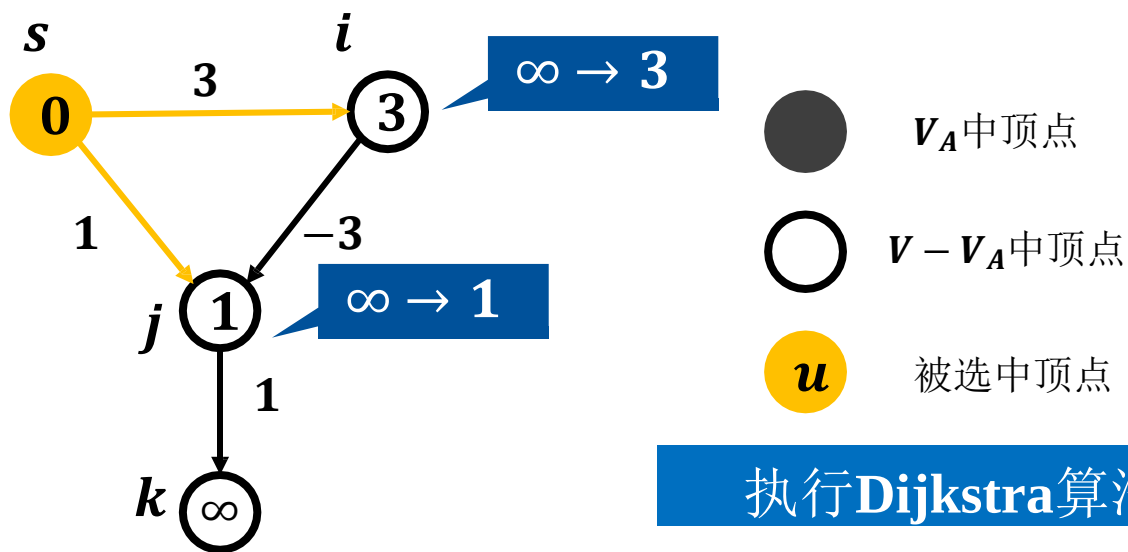


被选中顶点

执行**Dijkstra**算法

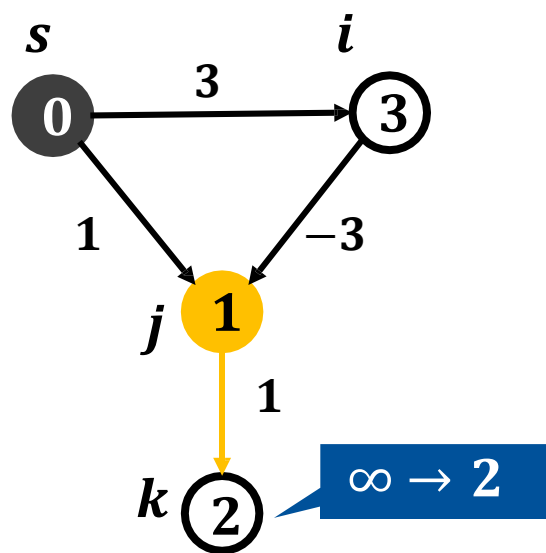
问题背景

- 图中存在负权边，**Dijkstra**算法不再适用



问题背景

- 图中存在负权边，**Dijkstra**算法不再适用



V_A 中顶点



$V - V_A$ 中顶点

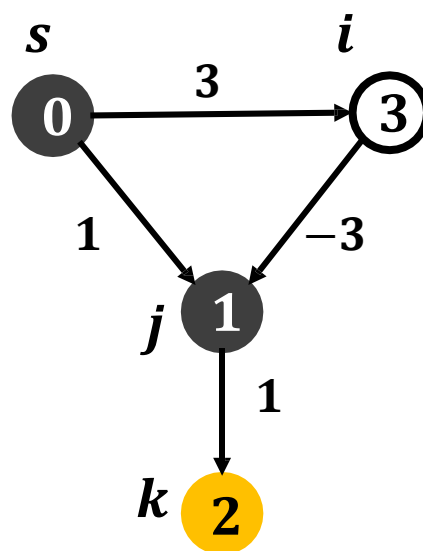


被选中顶点

执行Dijkstra算法

问题背景

- 图中存在负权边，**Dijkstra**算法不再适用



V_A 中顶点



$V - V_A$ 中顶点

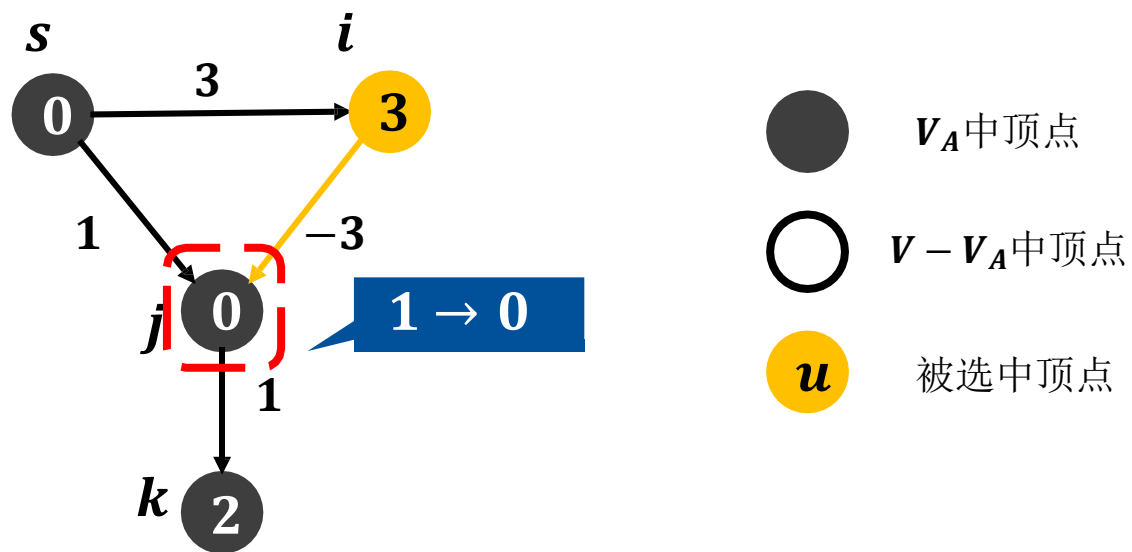


被选中顶点

执行**Dijkstra**算法

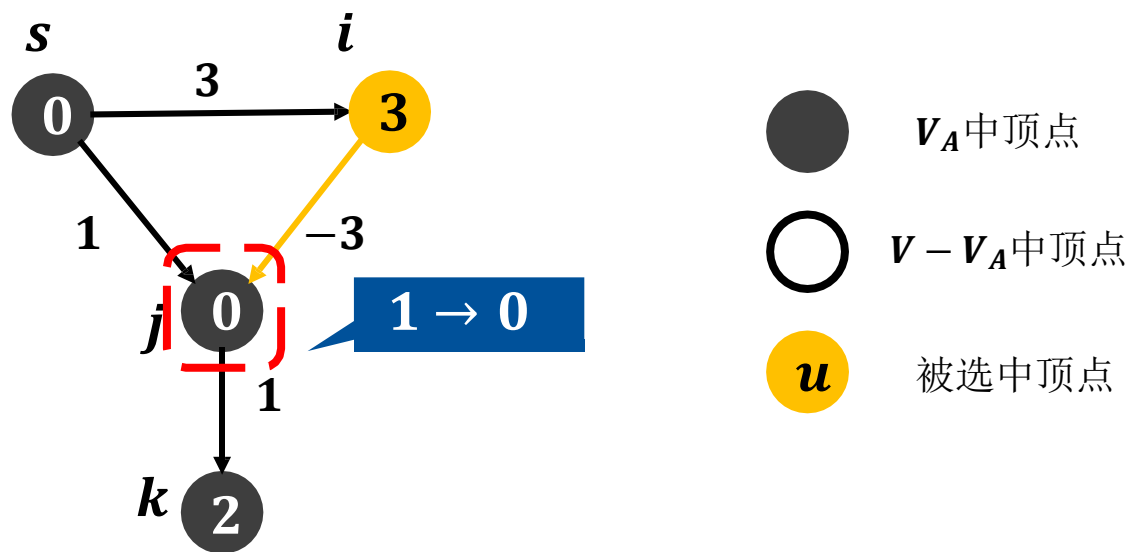
问题背景

- 图中存在负权边，**Dijkstra**算法不再适用



问题背景

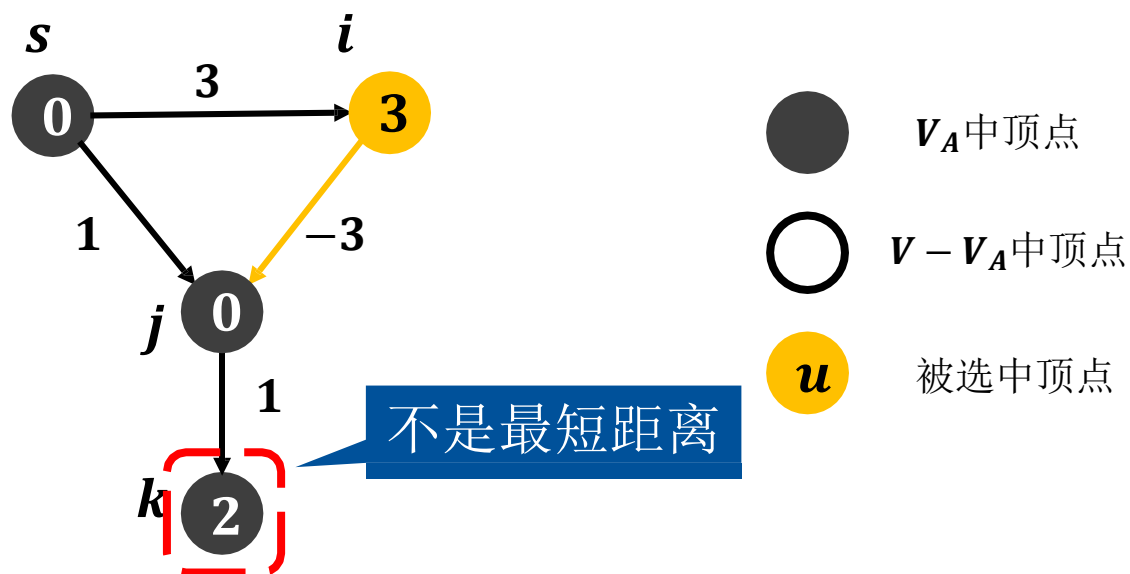
- 图中存在负权边，**Dijkstra**算法不再适用



Dijkstra算法：到黑色顶点的最短路应该已经计算出

问题背景

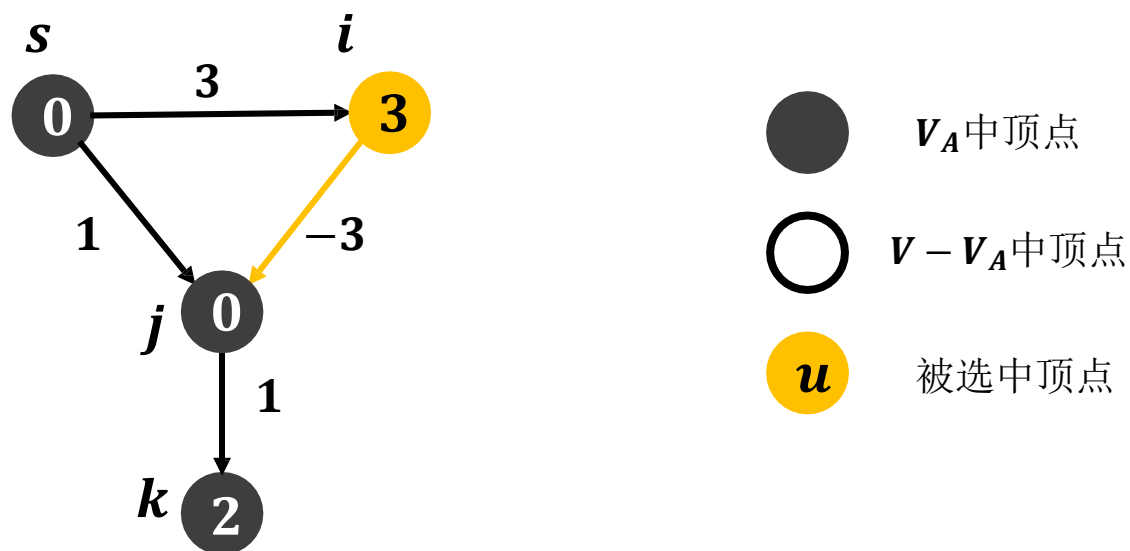
- 图中存在负权边，**Dijkstra**算法不再适用



Dijkstra算法：到黑色顶点的最短路应该已经计算出

问题背景

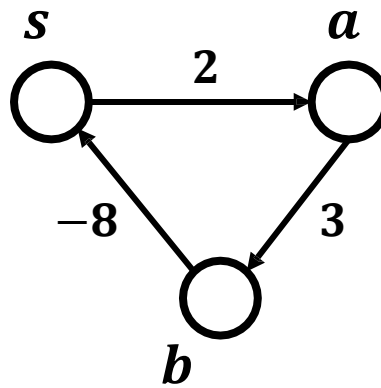
- 图中存在负权边，**Dijkstra**算法不再适用



问题：图中存在负权边时，是否存在单源最短路径？

问题背景

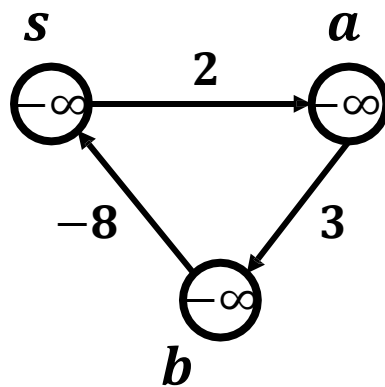
- 图中存在负权边时，是否存在单源最短路径？
 - 如果源点 s 可达负环，则难以定义最短路径



总有更小距离

问题背景

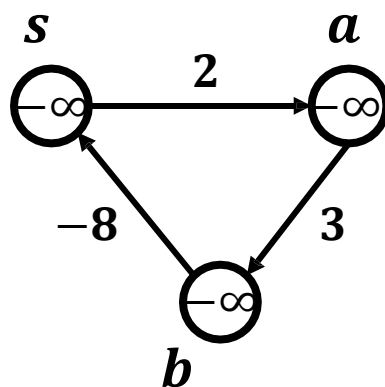
- 图中存在负权边时，是否存在单源最短路径？
 - 如果源点 s 可达负环，则难以定义最短路径



最终松弛到 $-\infty$

问题背景

- 图中存在负权边时，是否存在单源最短路径？
 - 如果源点 s 可达负环，则难以定义最短路径



最终松弛到 $-\infty$

若源点 s 无可达负环，则存在源点 s 的单源最短路径

问题定义

单源最短路径问题

Single Source Shortest Paths Problem

输入

- 带权图 $G = \langle V, E, W \rangle$
- 源点编号 s

输出

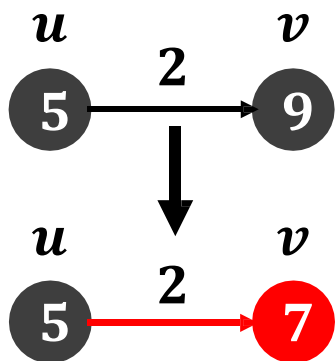
- 源点 s 到所有其他顶点 t 的最短距离 $\delta(s, t)$ 和最短路径 $\langle s, \dots, t \rangle$
- 或存在源点 s 可达的负环

挑战1：图中存在负权边时，如何求解单源最短路径？

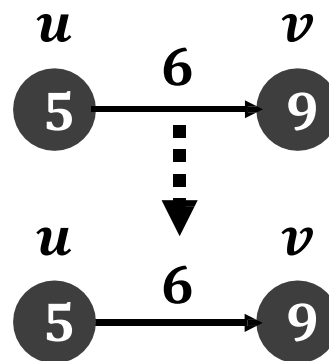
挑战2：图中存在负权边时，如何发现源点可达负环？

算法思想

- **Dijkstra**算法通过松弛操作迭代更新最短距离



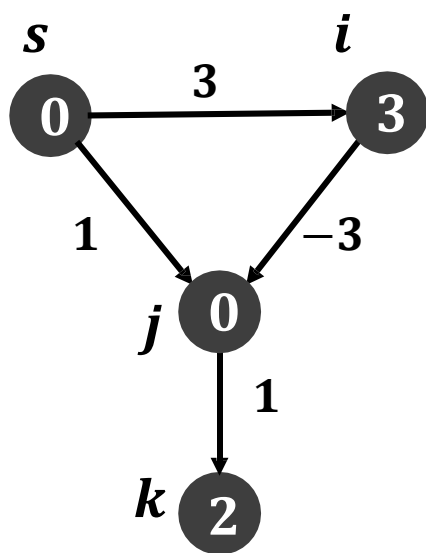
松弛成功



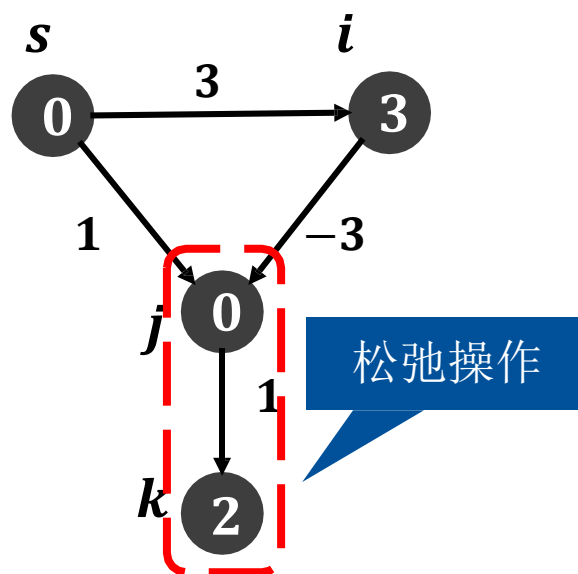
松弛失败

算法思想

- 存在负权边时，需要比**Dijkstra**算法更多次数的松弛操作



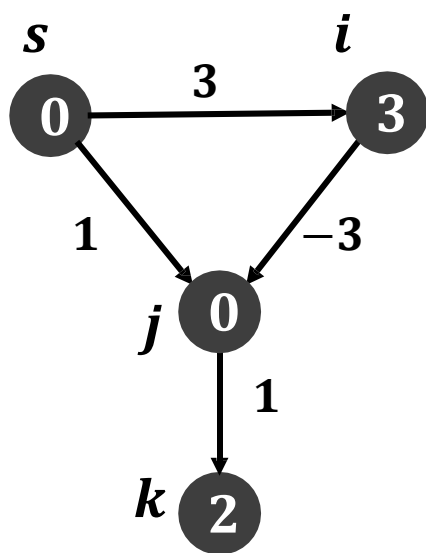
*Dijkstra*算法



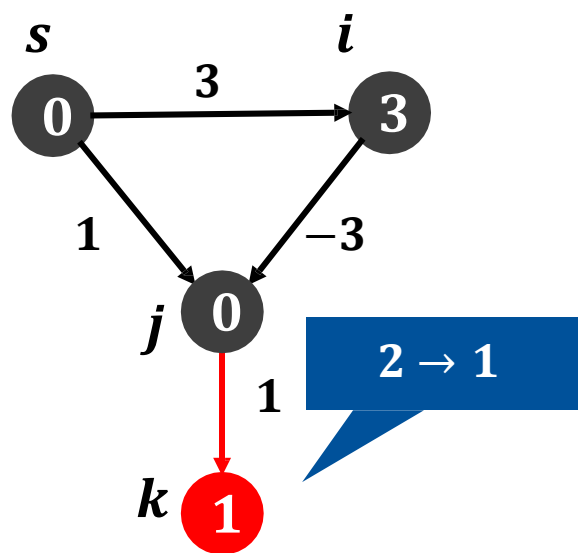
目标算法

算法思想

- 存在负权边时，需要比**Dijkstra**算法更多次数的松弛操作



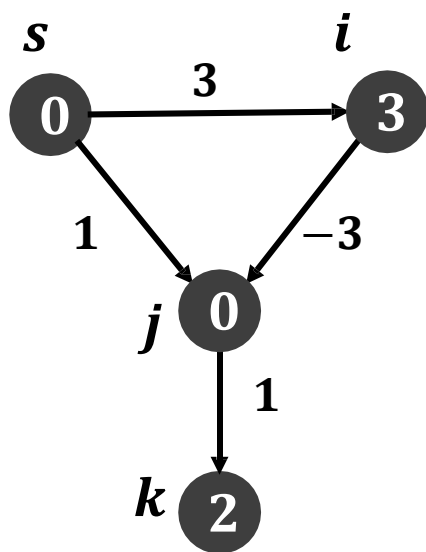
*Dijkstra*算法



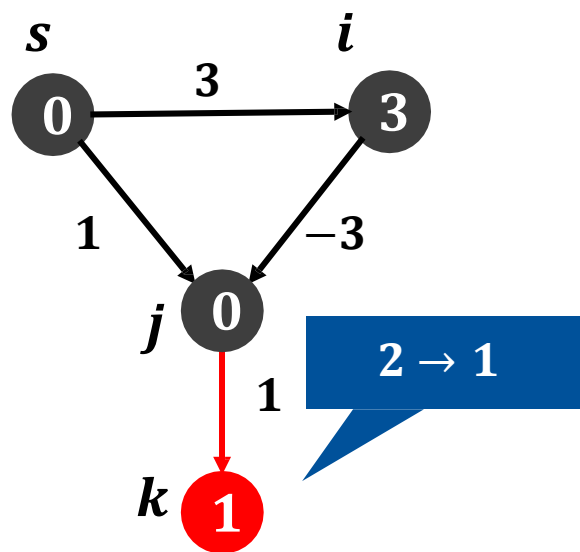
目标算法

算法思想

- 存在负权边时，需要比**Dijkstra**算法更多次数的松弛操作



*Dijkstra*算法



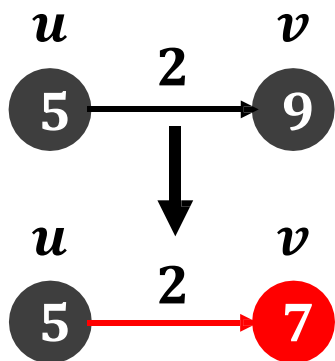
目标算法

问题：图中存在负权边时，如何利用松弛操作求解单源最短路？

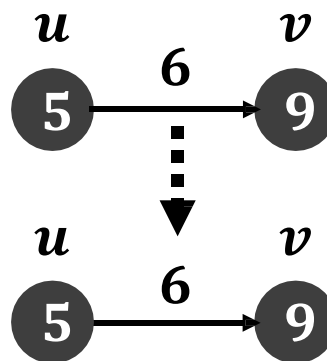
算法思想

- **Bellman-Ford**算法

- 解决挑战1：图中存在负权边时，如何求解单源最短路径？
 - 每轮对所有边进行松弛，持续迭代 $|V| - 1$ 轮



松弛成功

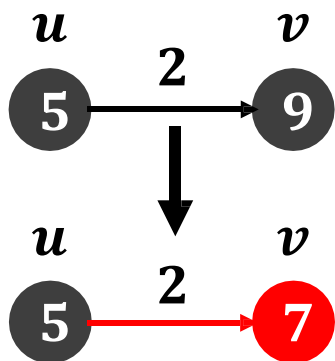


松弛失败

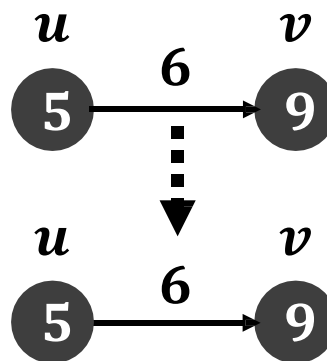
算法思想

- **Bellman-Ford**算法

- 解决挑战1：图中存在负权边时，如何求解单源最短路径？
 - 每轮对所有边进行松弛，持续迭代 $|V| - 1$ 轮
- 解决挑战2：图中存在负权边时，如何发现源点可达负环？
 - 若第 $|V|$ 轮仍松弛成功，存在源点 s 可达的负环



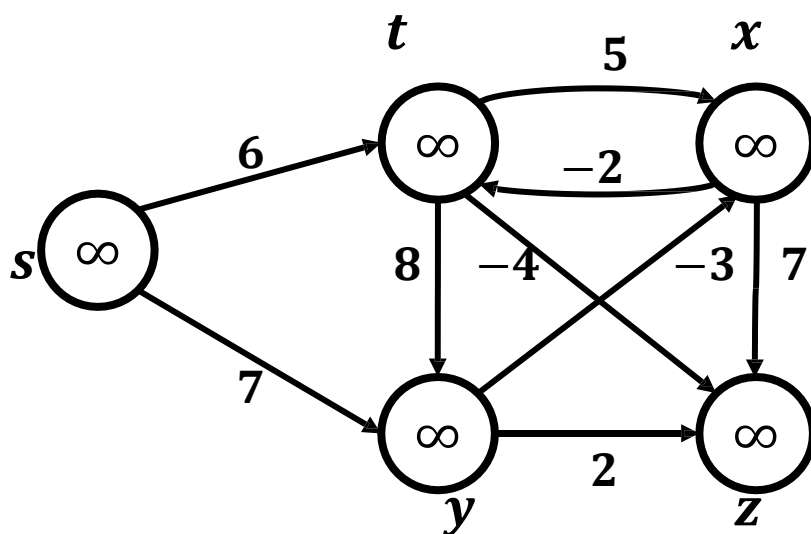
松弛成功



松弛失败

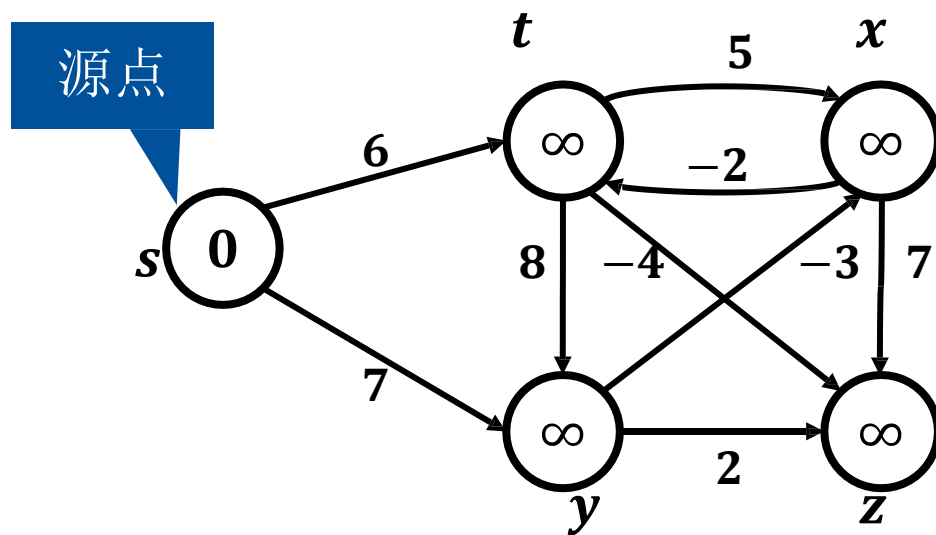
算法实例

V	s	t	x	y	z
$pred$	N	N	N	N	N
$dist$	∞	∞	∞	∞	∞



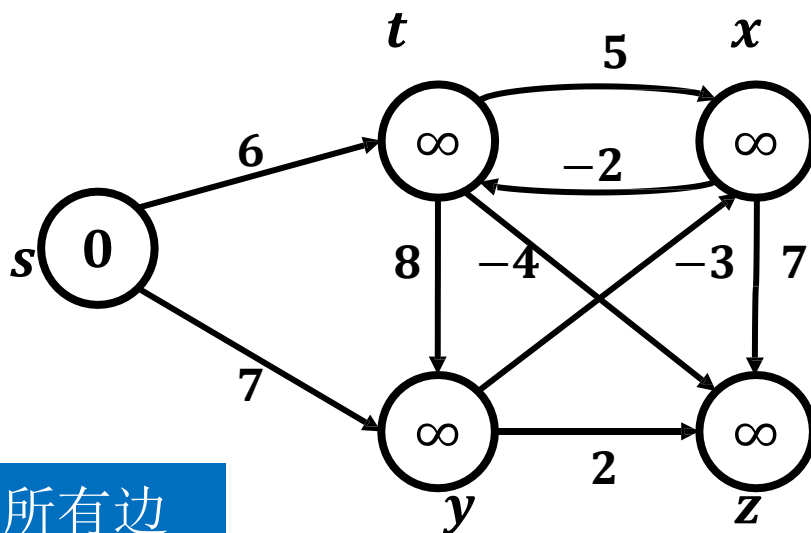
算法实例

V	s	t	x	y	z
$pred$	N	N	N	N	N
$dist$	0	∞	∞	∞	∞



算法实例

V	s	t	x	y	z
$pred$	N	N	N	N	N
$dist$	0	∞	∞	∞	∞



松弛顺序: $(x, z), (t, x), (x, t), (t, z), (y, x), (y, z), (t, y), (s, t), (s, y)$

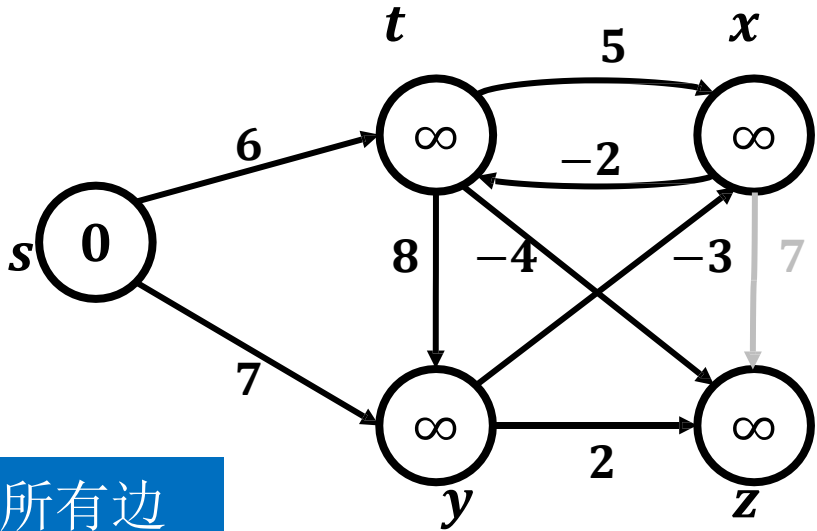
→ 松弛失败

→ 松弛成功

第1轮: 松弛所有边

算法实例

<i>V</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>pred</i>	N	N	N	N	N
<i>dist</i>	0	∞	∞	∞	∞



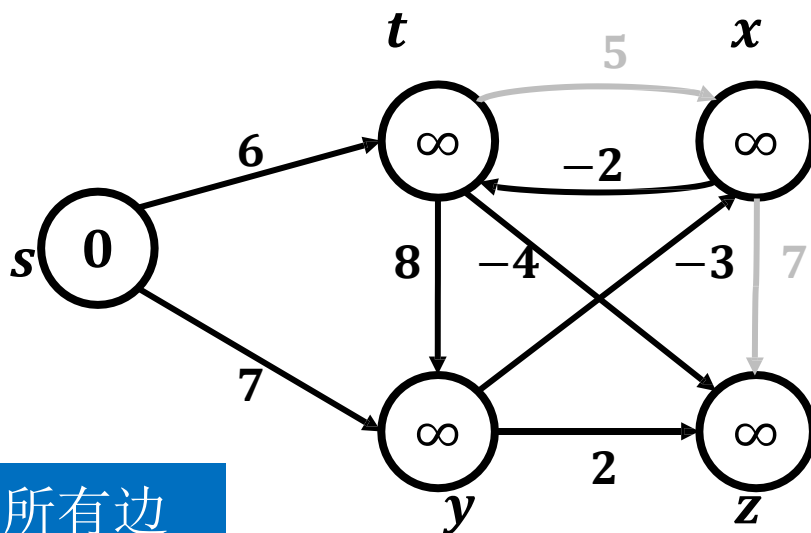
松弛顺序: $(x, z), (t, x), (x, t), (t, z), (y, x), (y, z), (t, y), (s, t), (s, y)$

→ 松弛失败
→ 松弛成功

第1轮: 松弛所有边

算法实例

V	s	t	x	y	z
$pred$	N	N	N	N	N
$dist$	0	∞	∞	∞	∞



松弛顺序: $(x, z), (t, x),$
 $(x, t), (t, z), (y, x), (y, z),$
 $(t, y), (s, t), (s, y)$

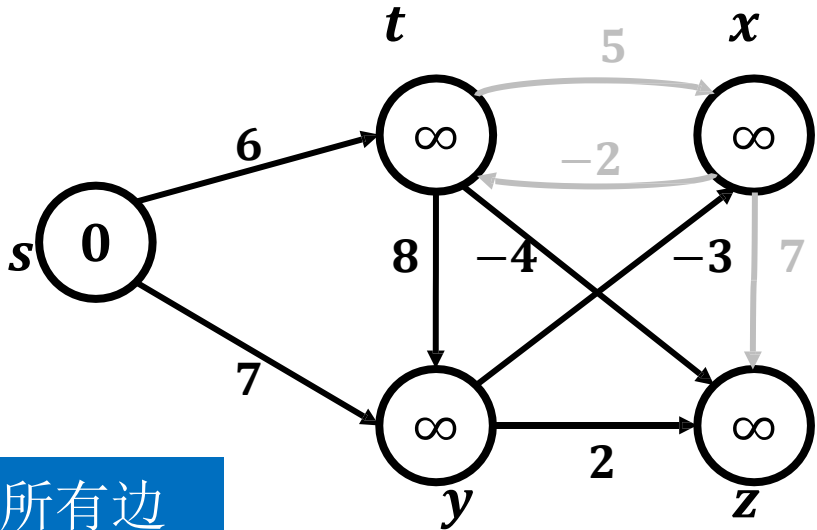
→ 松弛失败

→ 松弛成功

第1轮: 松弛所有边

算法实例

<i>V</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>pred</i>	N	N	N	N	N
<i>dist</i>	0	∞	∞	∞	∞



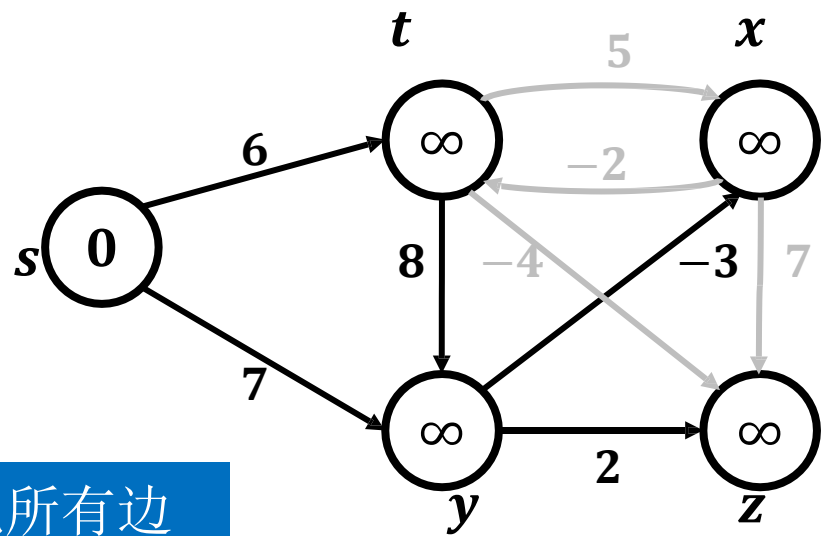
松弛顺序: $(x, z), (t, x), (x, t), (t, z), (y, x), (y, z), (t, y), (s, t), (s, y)$

→ 松弛失败
→ 松弛成功

第1轮: 松弛所有边

算法实例

<i>V</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>pred</i>	N	N	N	N	N
<i>dist</i>	0	∞	∞	∞	∞

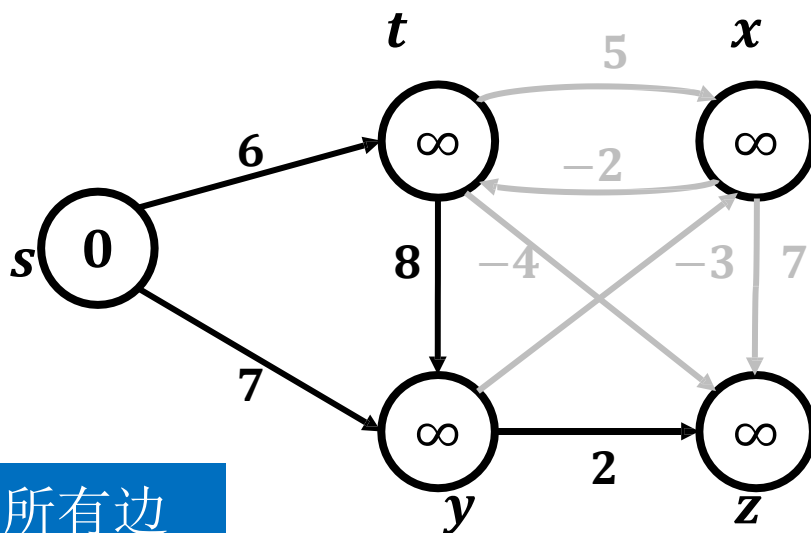


松弛顺序: (*x*, *z*), (*t*, *x*), (*x*, *t*), (*t*, *z*), (*y*, *x*), (*y*, *z*), (*t*, *y*), (*s*, *t*), (*s*, *y*)

第1轮: 松弛所有边

算法实例

V	s	t	x	y	z
$pred$	N	N	N	N	N
$dist$	0	∞	∞	∞	∞



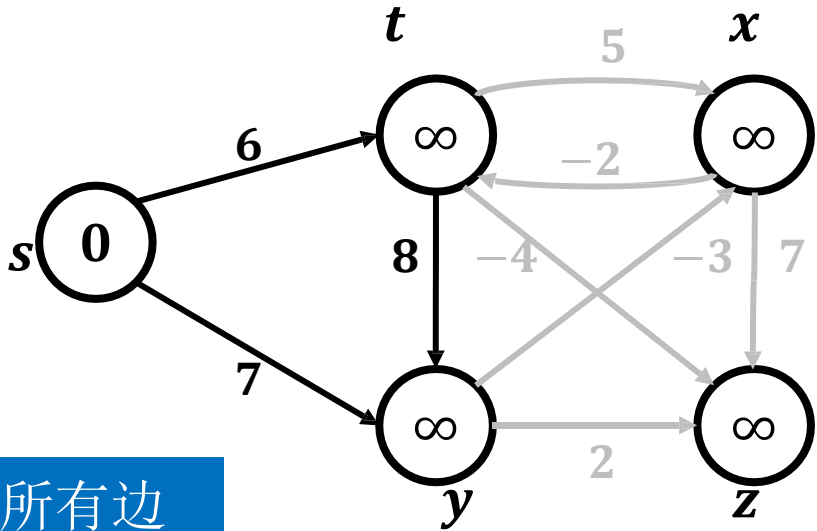
松弛顺序: $(x, z), (t, x),$
 $(x, t), (t, z), (y, x), (y, z),$
 $(t, y), (s, t), (s, y)$

→ 松弛失败
→ 松弛成功

第1轮: 松弛所有边

算法实例

<i>V</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>pred</i>	N	N	N	N	N
<i>dist</i>	0	∞	∞	∞	∞



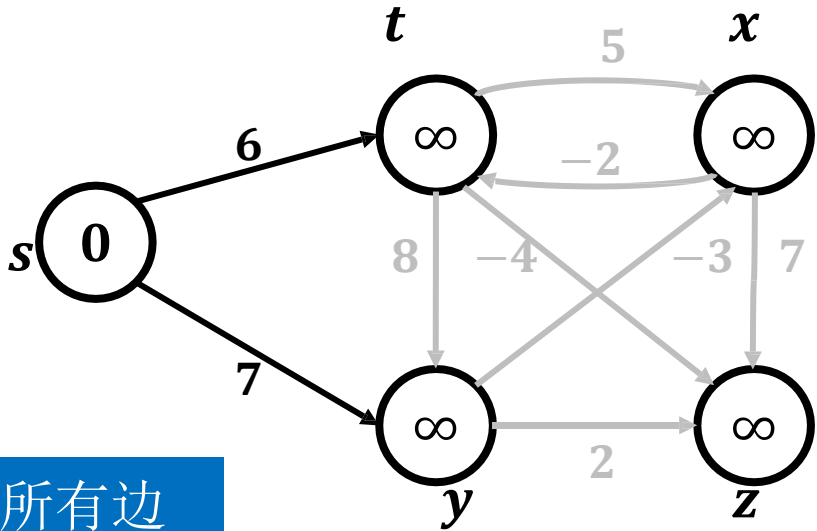
松弛顺序: $(x, z), (t, x), (x, t), (t, z), (y, x), (y, z), (t, y), (s, t), (s, y)$

→ 松弛失败
→ 松弛成功

第1轮：松弛所有边

算法实例

<i>V</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>pred</i>	N	N	N	N	N
<i>dist</i>	0	∞	∞	∞	∞



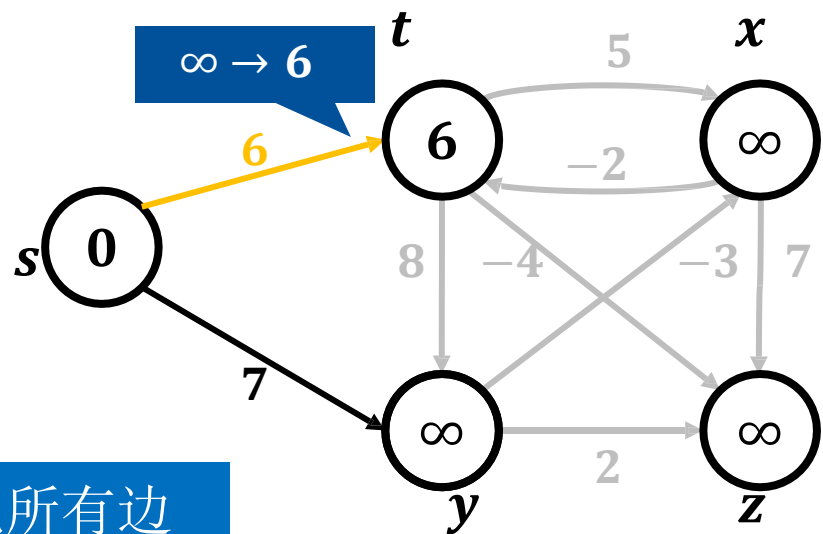
松弛顺序: $(x, z), (t, x), (x, t), (t, z), (y, x), (y, z), (t, y), (s, t), (s, y)$

→ 松弛失败
→ 松弛成功

第1轮: 松弛所有边

算法实例

<i>V</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>pred</i>	N	s	N	N	N
<i>dist</i>	0	6	∞	∞	∞



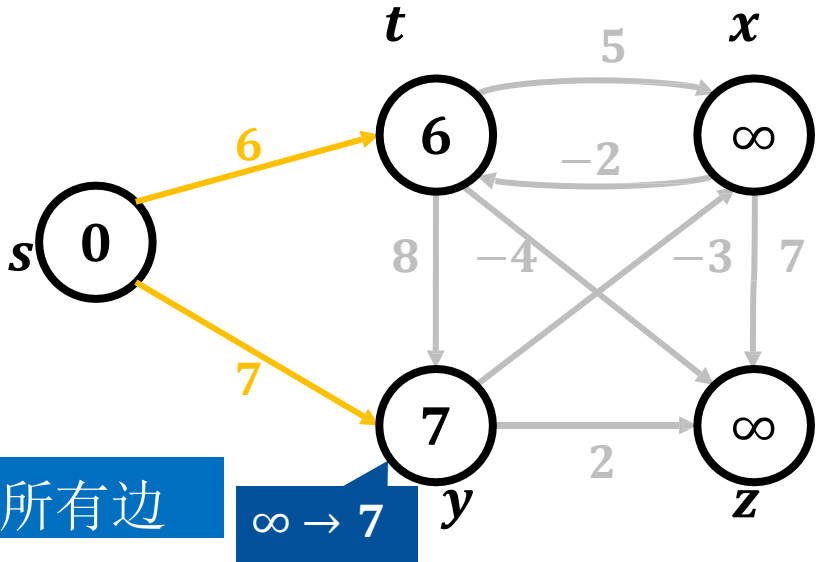
松弛顺序: $(x, z), (t, x), (x, t), (t, z), (y, x), (y, z), (t, y), (s, t), (s, y)$

→ 松弛失败
→ 松弛成功

第1轮: 松弛所有边

算法实例

<i>V</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>pred</i>	N	<i>s</i>	N	<i>s</i>	N
<i>dist</i>	0	6	∞	7	∞

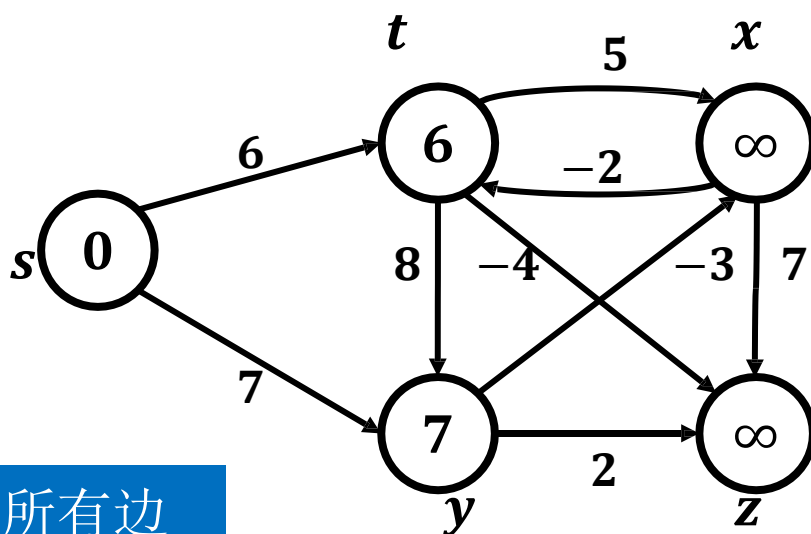


松弛顺序： $(x, z), (t, x), (x, t), (t, z), (y, x), (y, z), (t, y), (s, t), (s, y)$

→ 松弛失败
→ 松弛成功

算法实例

V	s	t	x	y	z
$pred$	N	s	N	s	N
$dist$	0	6	∞	7	∞



松弛顺序: $(x, z), (t, x), (x, t), (t, z), (y, x), (y, z), (t, y), (s, t), (s, y)$

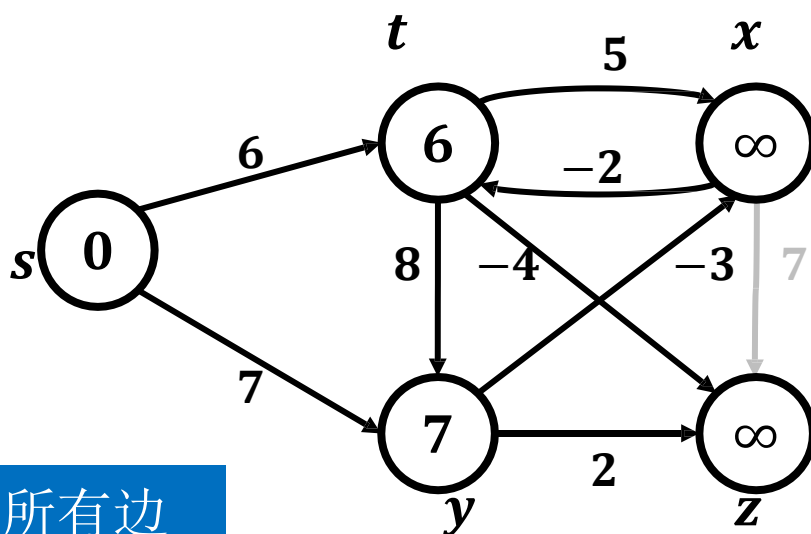
→ 松弛失败

→ 松弛成功

第2轮: 松弛所有边

算法实例

V	s	t	x	y	z
$pred$	N	s	N	s	N
$dist$	0	6	∞	7	∞



松弛顺序: $(x, z), (t, x),$
 $(x, t), (t, z), (y, x), (y, z),$
 $(t, y), (s, t), (s, y)$

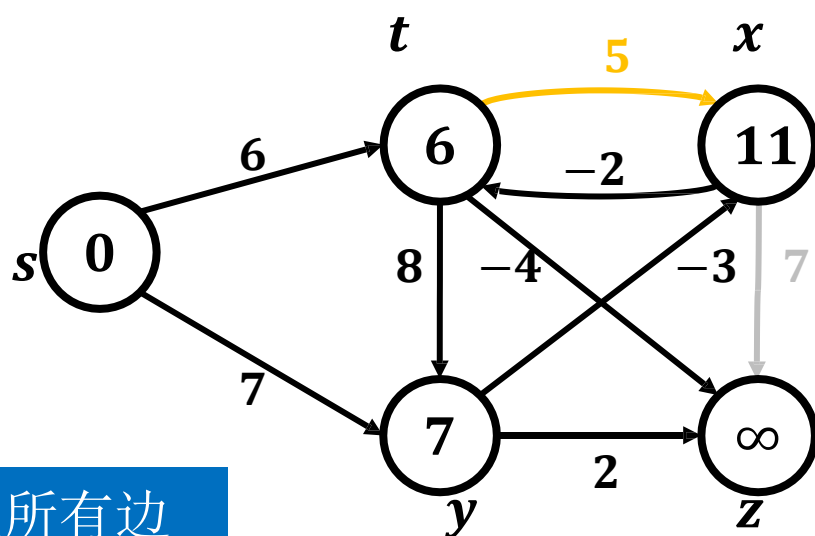
→ 松弛失败

→ 松弛成功

第2轮: 松弛所有边

算法实例

V	s	t	x	y	z
$pred$	N	s	t	s	N
$dist$	0	6	11	7	∞



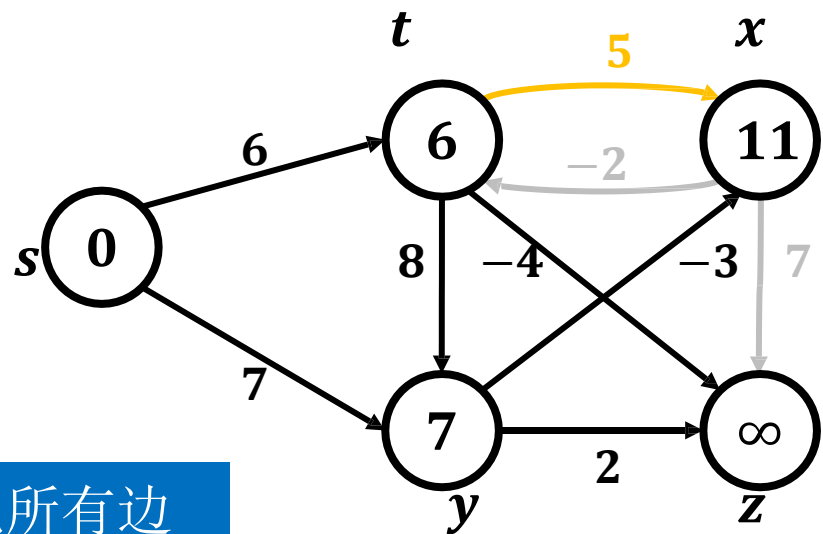
松弛顺序: (x, z) , (t, x) ,
 (x, t) , (t, z) , (y, x) , (y, z) ,
 (t, y) , (s, t) , (s, y)

→ 松弛失败
→ 松弛成功

第2轮: 松弛所有边

算法实例

<i>V</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>pred</i>	N	<i>s</i>	<i>t</i>	<i>s</i>	N
<i>dist</i>	0	6	11	7	∞



松弛顺序: (x, z) , (t, x) ,
 (x, t) , (t, z) , (y, x) , (y, z) ,
 (t, y) , (s, t) , (s, y)

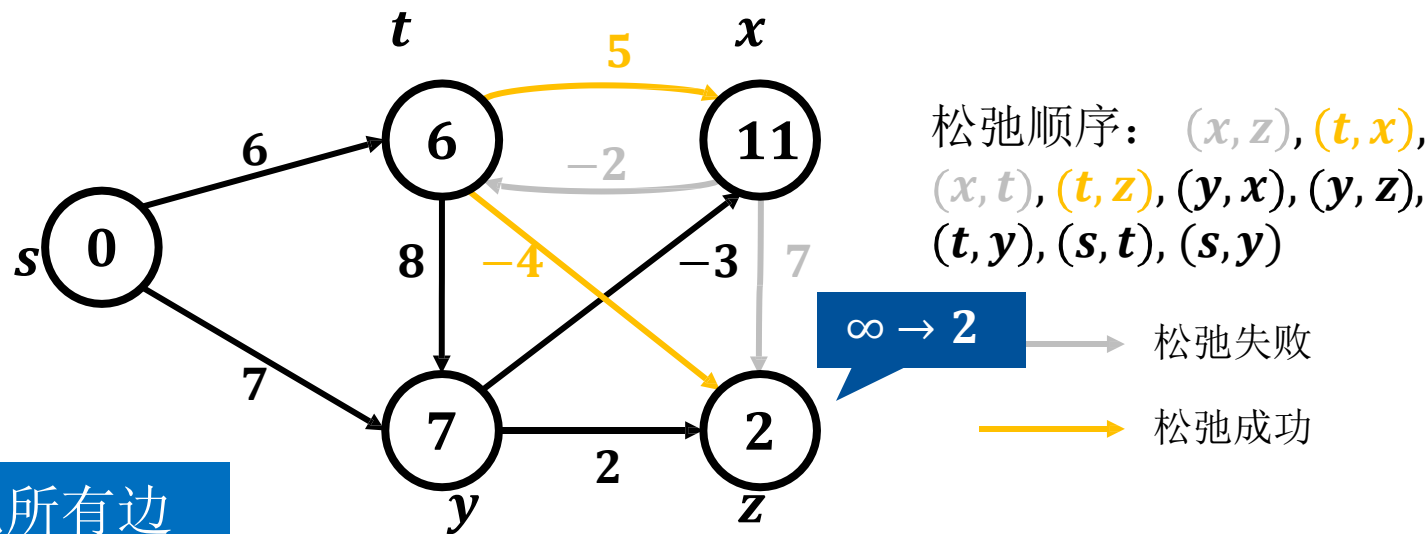
→ 松弛失败

→ 松弛成功

第2轮：松弛所有边

算法实例

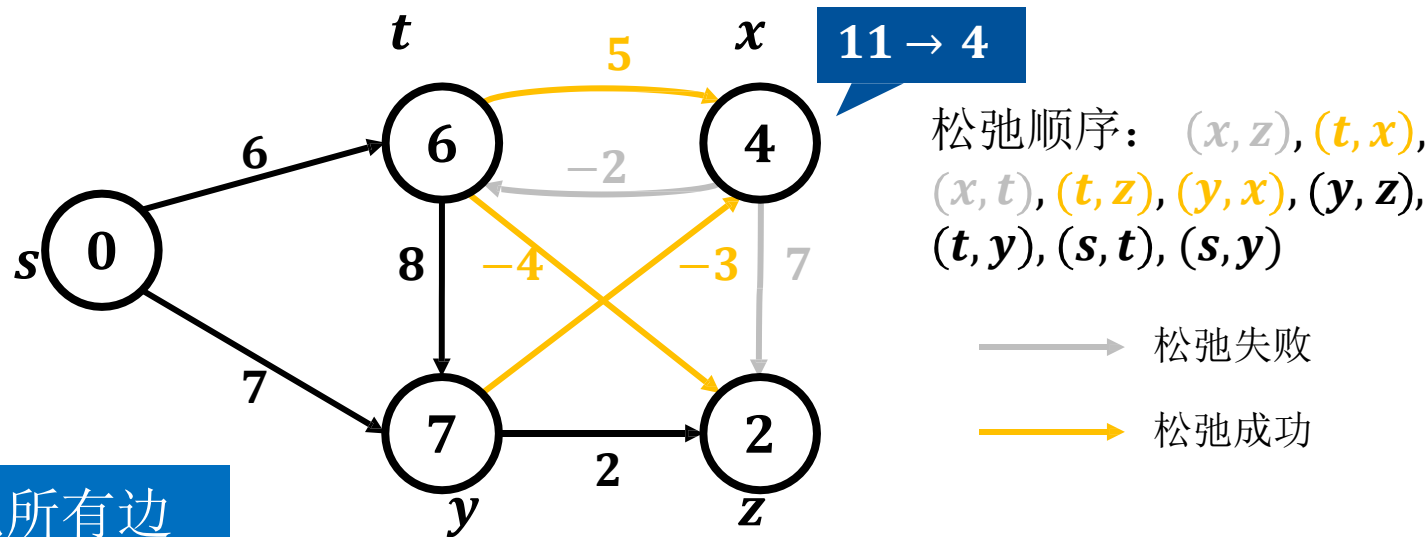
<i>V</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>pred</i>	N	<i>s</i>	<i>t</i>	<i>s</i>	<i>t</i>
<i>dist</i>	0	6	11	7	2



第2轮：松弛所有边

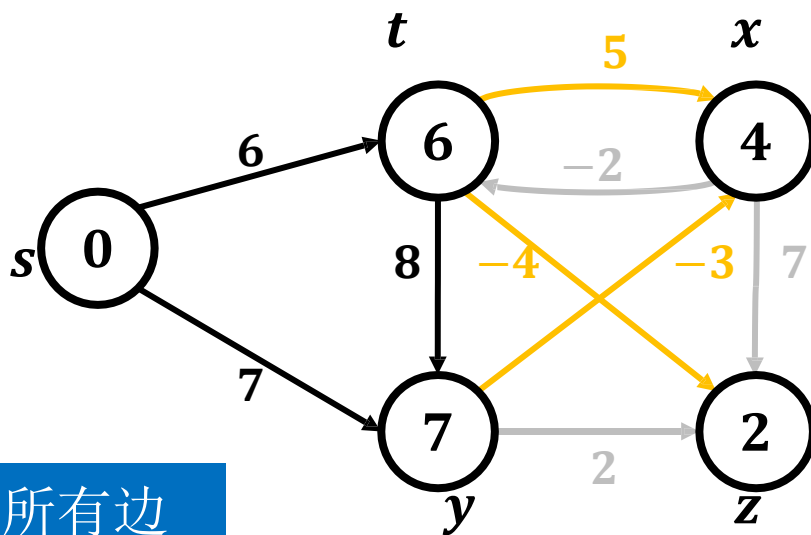
算法实例

<i>V</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>pred</i>	N	<i>s</i>	<i>y</i>	<i>s</i>	<i>t</i>
<i>dist</i>	0	6	4	7	2



算法实例

V	s	t	x	y	z
$pred$	N	s	y	s	t
$dist$	0	6	4	7	2



松弛顺序: (x, z) , (t, x) ,
 (x, t) , (t, z) , (y, x) , (y, z) ,
 (t, y) , (s, t) , (s, y)

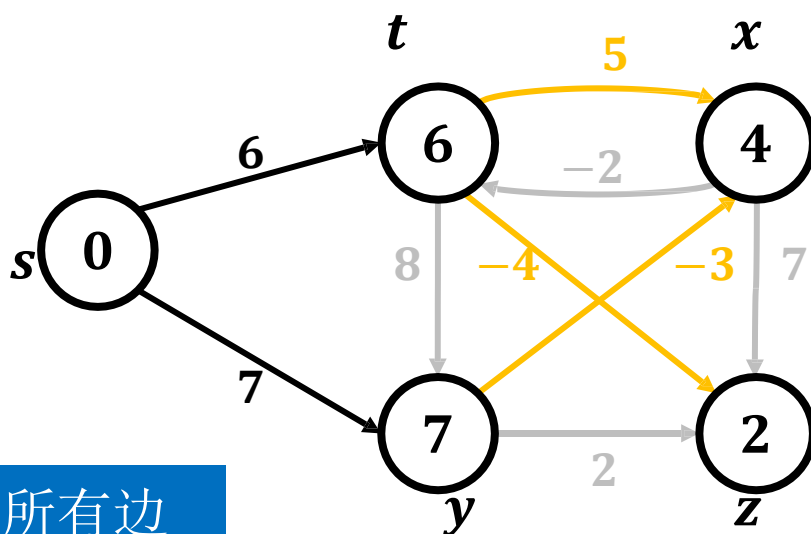
→ 松弛失败

→ 松弛成功

第2轮: 松弛所有边

算法实例

V	s	t	x	y	z
$pred$	N	s	y	s	t
$dist$	0	6	4	7	2



松弛顺序: (x, z) , (t, x) ,
 (x, t) , (t, z) , (y, x) , (y, z) ,
 (t, y) , (s, t) , (s, y)

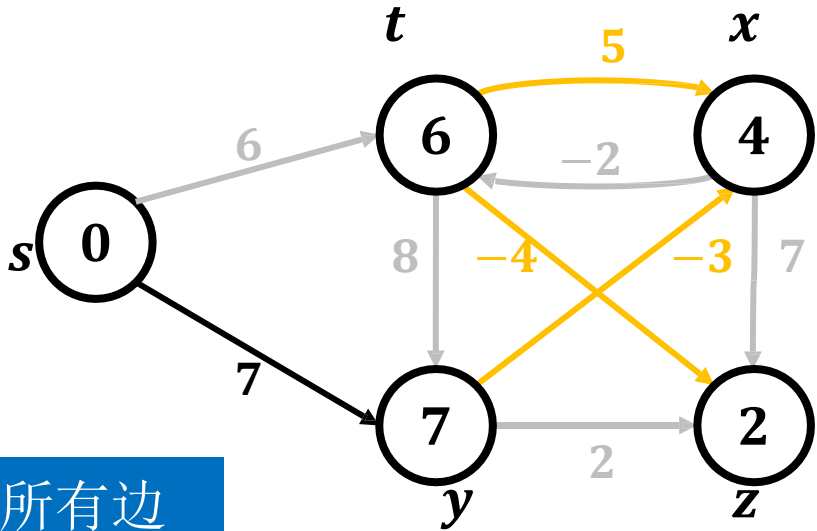
→ 松弛失败

→ 松弛成功

第2轮: 松弛所有边

算法实例

<i>V</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>pred</i>	N	<i>s</i>	<i>y</i>	<i>s</i>	<i>t</i>
<i>dist</i>	0	6	4	7	2



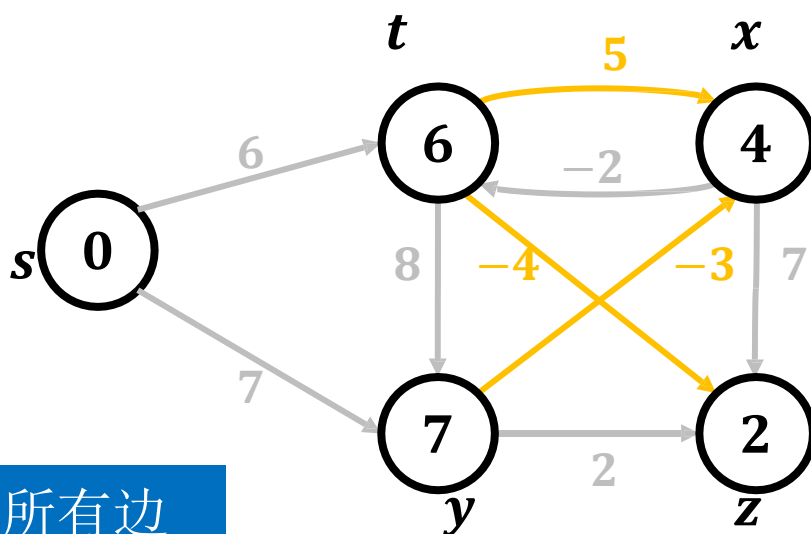
松弛顺序: (*x*, *z*), (*t*, *x*),
(*x*, *t*), (*t*, *z*), (*y*, *x*), (*y*, *z*),
(*t*, *y*), (*s*, *t*), (*s*, *y*)

→ 松弛失败
→ 松弛成功

第2轮: 松弛所有边

算法实例

V	s	t	x	y	z
$pred$	N	s	y	s	t
$dist$	0	6	4	7	2



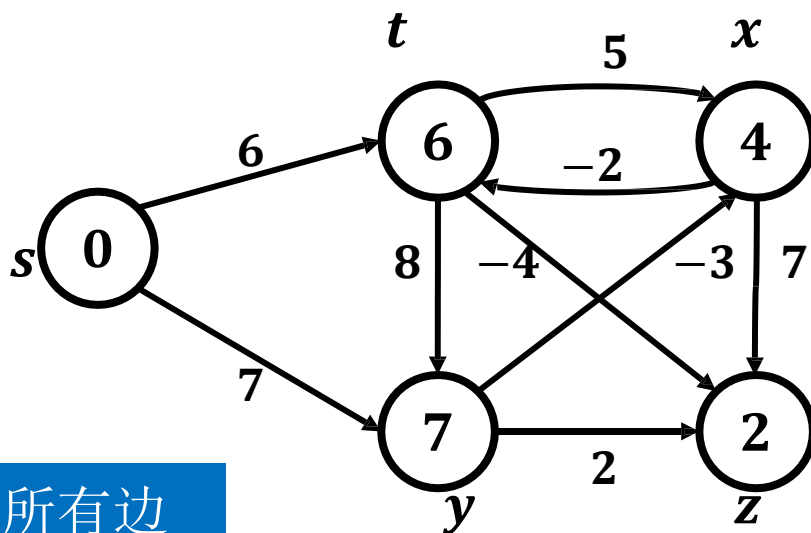
松弛顺序: (x, z) , (t, x) ,
 (x, t) , (t, z) , (y, x) , (y, z) ,
 (t, y) , (s, t) , (s, y)

→ 松弛失败
→ 松弛成功

第2轮: 松弛所有边

算法实例

<i>V</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>pred</i>	N	<i>s</i>	<i>y</i>	<i>s</i>	<i>t</i>
<i>dist</i>	0	6	4	7	2



松弛顺序: $(x, z), (t, x),$
 $(x, t), (t, z), (y, x), (y, z),$
 $(t, y), (s, t), (s, y)$

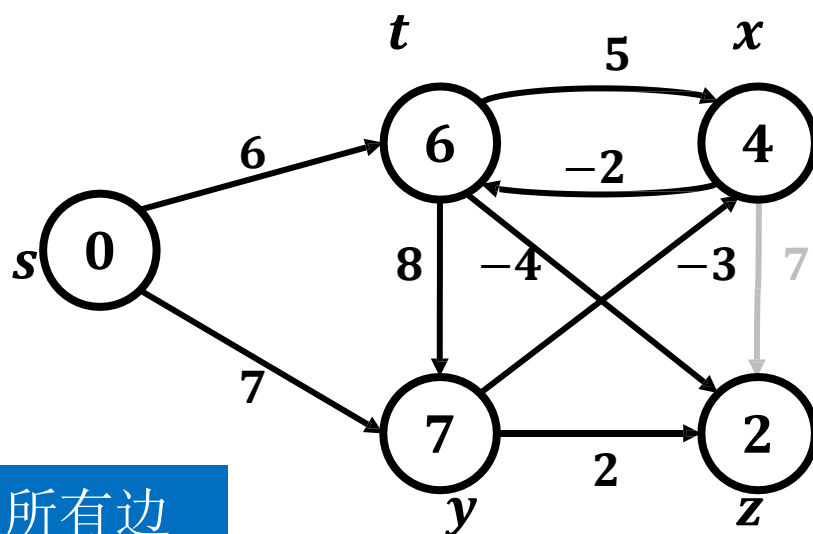
→ 松弛失败

→ 松弛成功

第3轮: 松弛所有边

算法实例

<i>V</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>pred</i>	N	<i>s</i>	<i>y</i>	<i>s</i>	<i>t</i>
<i>dist</i>	0	6	4	7	2



松弛顺序: $(x, z), (t, x), (x, t), (t, z), (y, x), (y, z), (t, y), (s, t), (s, y)$

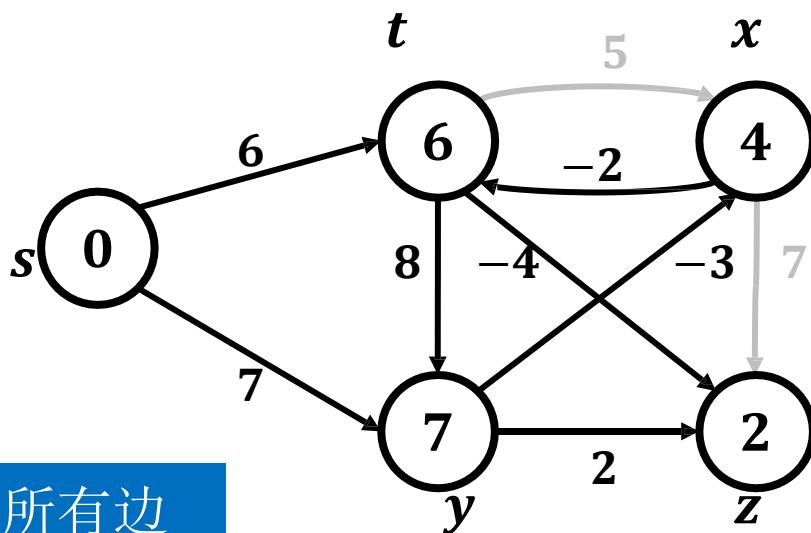
→ 松弛失败

→ 松弛成功

第3轮: 松弛所有边

算法实例

V	s	t	x	y	z
$pred$	N	s	y	s	t
$dist$	0	6	4	7	2



松弛顺序: (x, z) , (t, x) ,
 (x, t) , (t, z) , (y, x) , (y, z) ,
 (t, y) , (s, t) , (s, y)

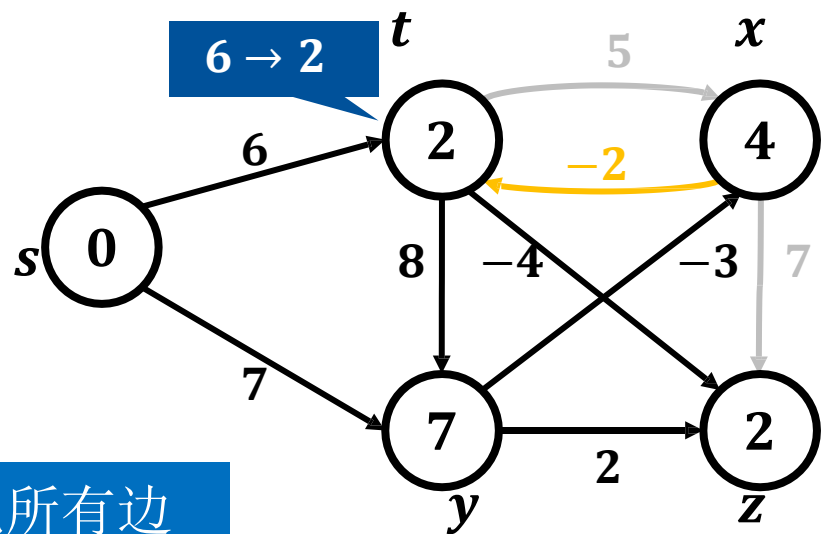
→ 松弛失败

→ 松弛成功

第3轮: 松弛所有边

算法实例

<i>V</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>pred</i>	N	x	<i>y</i>	<i>s</i>	<i>t</i>
<i>dist</i>	0	2	4	7	2



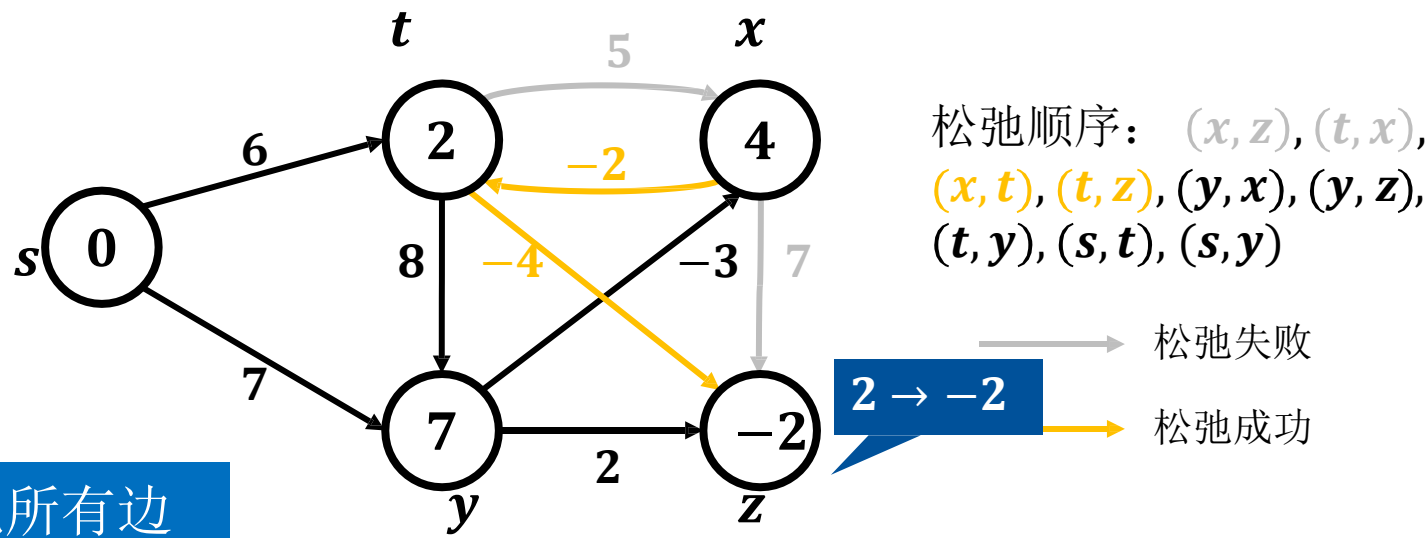
松弛顺序: $(x, z), (t, x), (x, t), (t, z), (y, x), (y, z), (t, y), (s, t), (s, y)$

→ 松弛失败
→ 松弛成功

第3轮: 松弛所有边

算法实例

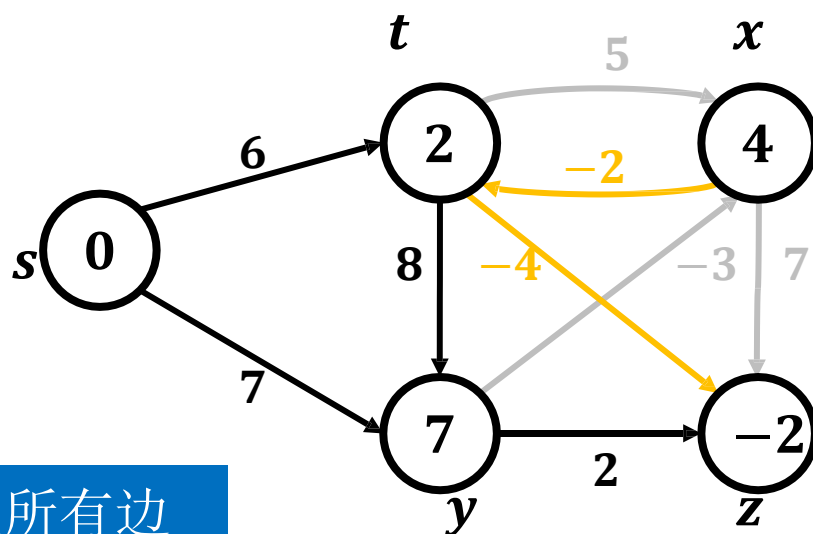
<i>V</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>pred</i>	N	<i>x</i>	<i>y</i>	<i>s</i>	<i>t</i>
<i>dist</i>	0	2	4	7	-2



第3轮：松弛所有边

算法实例

V	s	t	x	y	z
$pred$	N	x	y	s	t
$dist$	0	2	4	7	-2



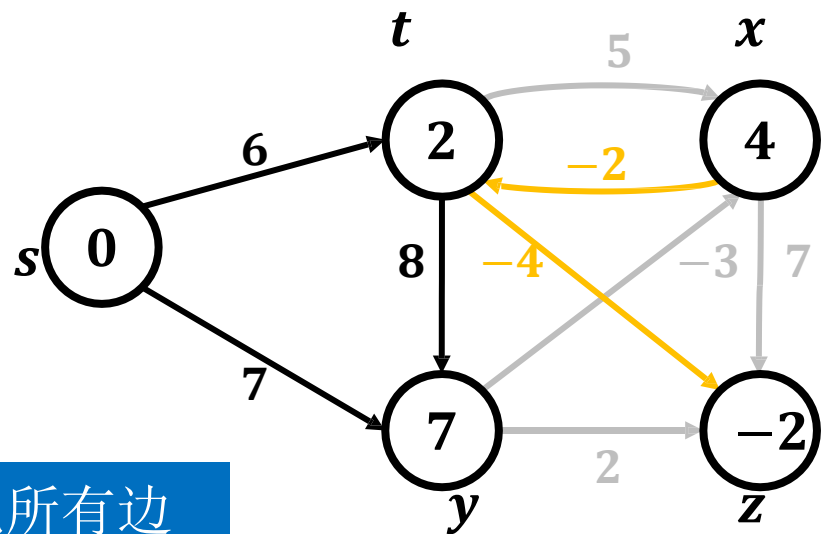
松弛顺序: $(x, z), (t, x), (x, t), (t, z), (y, x), (y, z), (t, y), (s, t), (s, y)$

——→ 松弛失败
——→ 松弛成功

第3轮: 松弛所有边

算法实例

<i>V</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>pred</i>	N	<i>x</i>	<i>y</i>	<i>s</i>	<i>t</i>
<i>dist</i>	0	2	4	7	-2



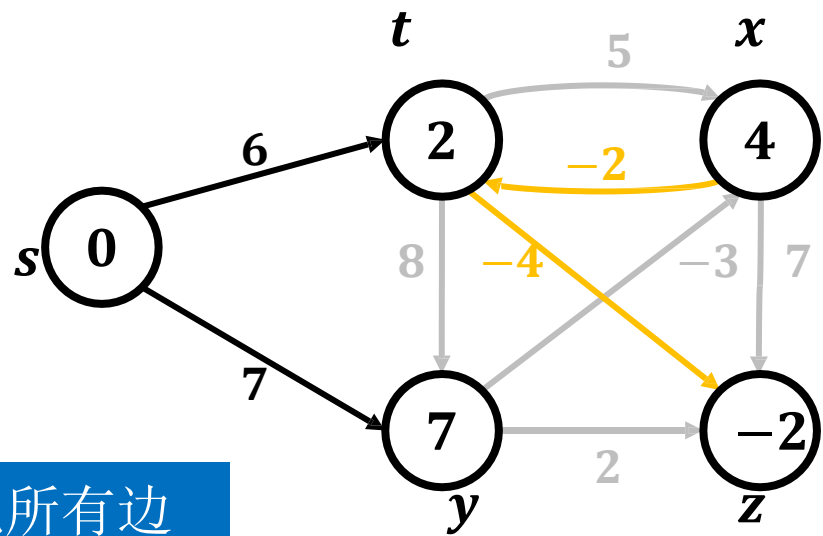
松弛顺序: (*x*, *z*), (*t*, *x*), (*x*, *t*), (*t*, *z*), (*y*, *x*), (*y*, *z*), (*t*, *y*), (*s*, *t*), (*s*, *y*)

→ 松弛失败
→ 松弛成功

第3轮: 松弛所有边

算法实例

<i>V</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>pred</i>	N	<i>x</i>	<i>y</i>	<i>s</i>	<i>t</i>
<i>dist</i>	0	2	4	7	-2



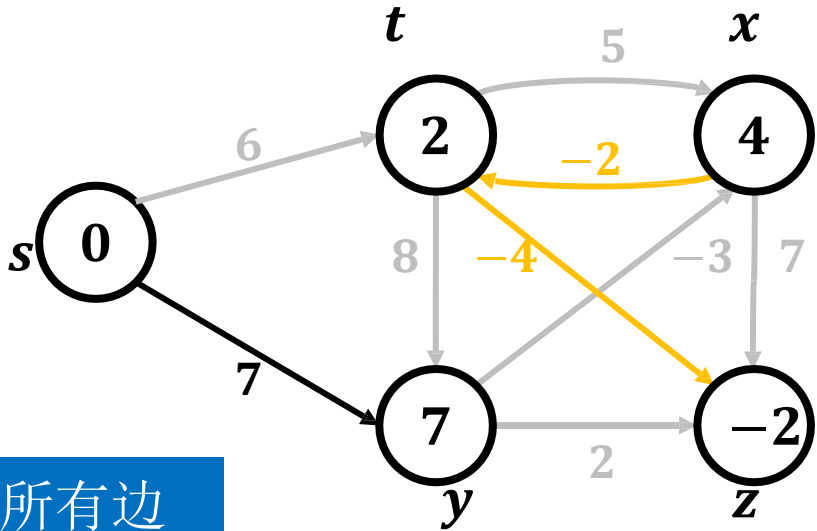
松弛顺序: (*x*, *z*), (*t*, *x*),
(*x*, *t*), (*t*, *z*), (*y*, *x*), (*y*, *z*),
(*t*, *y*), (*s*, *t*), (*s*, *y*)

→ 松弛失败
→ 松弛成功

第3轮: 松弛所有边

算法实例

<i>V</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>pred</i>	N	<i>x</i>	<i>y</i>	<i>s</i>	<i>t</i>
<i>dist</i>	0	2	4	7	-2



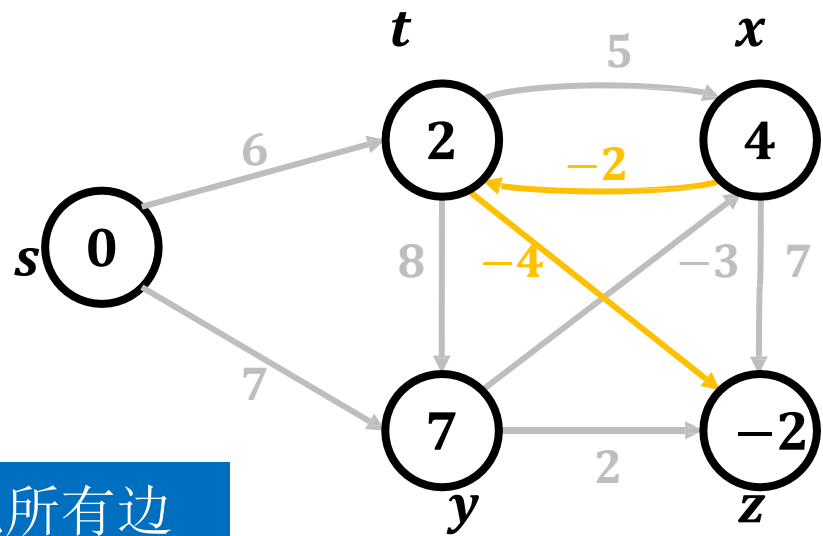
松弛顺序: (*x*, *z*), (*t*, *x*), (*x*, *t*), (*t*, *z*), (*y*, *x*), (*y*, *z*), (*t*, *y*), (*s*, *t*), (*s*, *y*)

→ 松弛失败
→ 松弛成功

第3轮: 松弛所有边

算法实例

<i>V</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>pred</i>	N	<i>x</i>	<i>y</i>	<i>s</i>	<i>t</i>
<i>dist</i>	0	2	4	7	-2



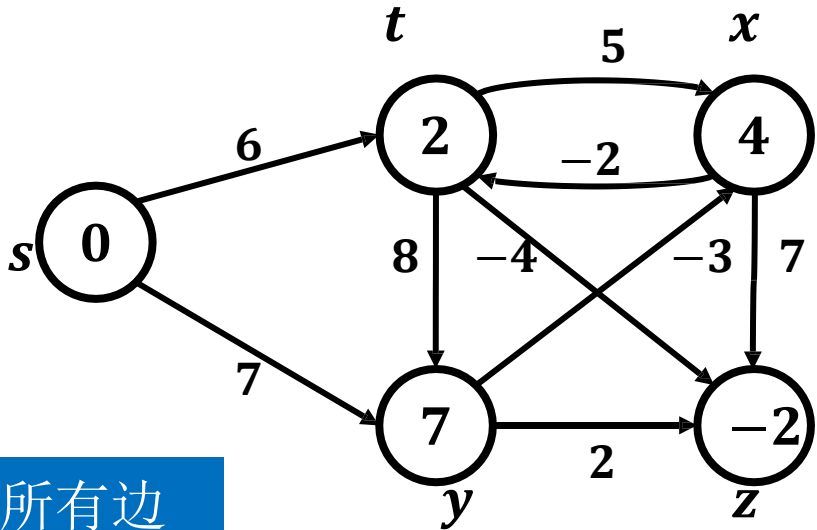
松弛顺序: (*x*, *z*), (*t*, *x*),
(*x*, *t*), (*t*, *z*), (*y*, *x*), (*y*, *z*),
(*t*, *y*), (*s*, *t*), (*s*, *y*)

→ 松弛失败
→ 松弛成功

第3轮: 松弛所有边

算法实例

<i>V</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>pred</i>	N	<i>x</i>	<i>y</i>	<i>s</i>	<i>t</i>
<i>dist</i>	0	2	4	7	-2



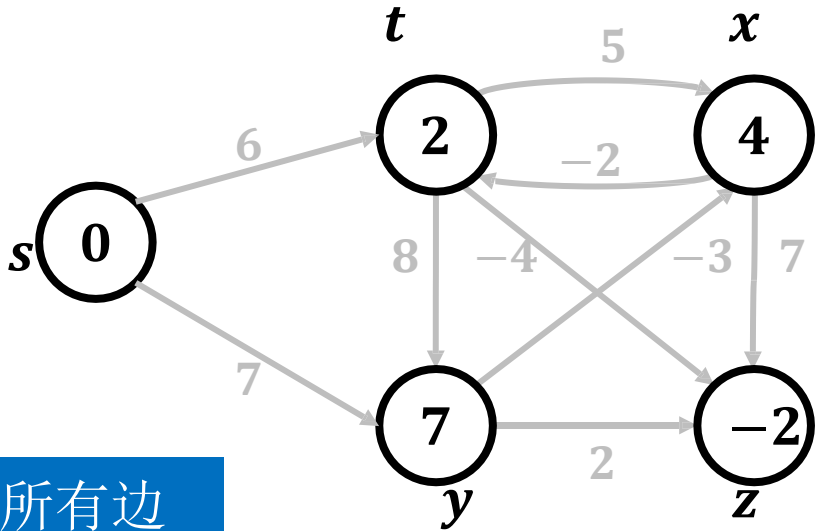
松弛顺序: $(x, z), (t, x), (x, t), (t, z), (y, x), (y, z), (t, y), (s, t), (s, y)$

→ 松弛失败
→ 松弛成功

第4轮: 松弛所有边

算法实例

<i>V</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>pred</i>	N	<i>x</i>	<i>y</i>	<i>s</i>	<i>t</i>
<i>dist</i>	0	2	4	7	-2



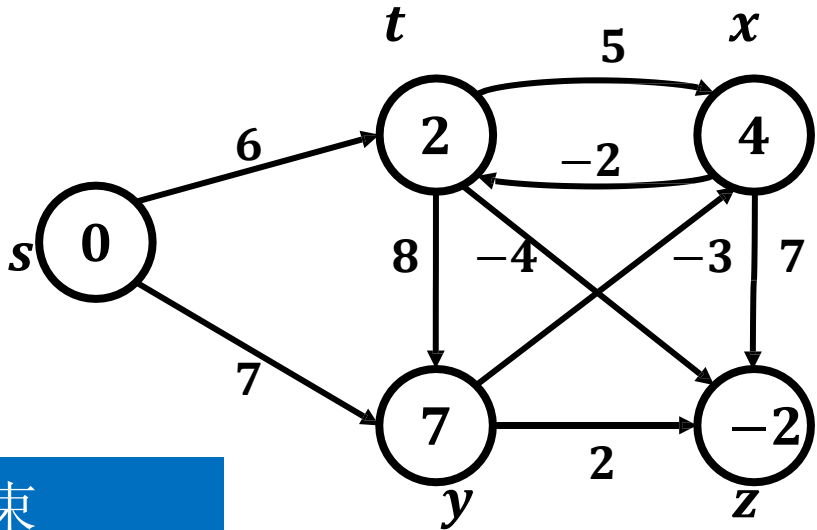
松弛顺序: (*x*, *z*), (*t*, *x*), (*x*, *t*), (*t*, *z*), (*y*, *x*), (*y*, *z*), (*t*, *y*), (*s*, *t*), (*s*, *y*)

→ 松弛失败
→ 松弛成功

第4轮: 松弛所有边

算法实例

<i>V</i>	<i>s</i>	<i>t</i>	<i>x</i>	<i>y</i>	<i>z</i>
<i>pred</i>	N	<i>x</i>	<i>y</i>	<i>s</i>	<i>t</i>
<i>dist</i>	0	2	4	7	-2



松弛顺序: $(x, z), (t, x), (x, t), (t, z), (y, x), (y, z), (t, y), (s, t), (s, y)$

→ 松弛失败
→ 松弛成功

松弛结束

伪代码

- **Bellman-Ford(G, s)**

输入: 图 $G = \langle V, E, W \rangle$, 源点 s

输出: 单源最短路径 P

新建一维数组 $dist[1..|V|]$, $pred[1..|V|]$

//初始化

```
for  $u \in V$  do  
     $dist[u] \leftarrow \infty$   
     $pred[u] \leftarrow NULL$   
end  
 $dist[s] \leftarrow 0$ 
```

初始化辅助数组

伪代码

- **Bellman-Ford(G, s)**

输入: 图 $G = \langle V, E, W \rangle$, 源点 s

输出: 单源最短路径 P

新建一维数组 $dist[1..|V|]$, $pred[1..|V|]$

//初始化

for $u \in V$ do

$dist[u] \leftarrow \infty$

$pred[u] \leftarrow NULL$

end

$dist[s] \leftarrow 0$

初始化源点距离

伪代码

- **Bellman-Ford(G, s)**

//执行单源最短路径算法

```
for  $i \leftarrow 1$  to  $|V| - 1$  do
  for  $(u, v) \in E$  do
    if  $dist[u] + w(u, v) < dist[v]$  then
       $dist[v] \leftarrow dist[u] + w(u, v)$ 
       $pred[v] \leftarrow u$ 
    end
  end
end
for  $(u, v) \in E$  do
  if  $dist[u] + w(u, v) < dist[v]$  then
    print 存在负环
    break
  end
end
```

进行 $|V| - 1$ 轮松弛

伪代码

- **Bellman-Ford(G, s)**

//执行单源最短路径算法

```
for  $i \leftarrow 1$  to  $|V| - 1$  do
  for  $(u, v) \in E$  do
    if  $dist[u] + w(u, v) < dist[v]$  then
       $dist[v] \leftarrow dist[u] + w(u, v)$ 
       $pred[v] \leftarrow u$ 
    end
  end
end
for  $(u, v) \in E$  do
  if  $dist[u] + w(u, v) < dist[v]$  then
    print 存在负环
    break
  end
end
```

对所有边进行松弛操作

伪代码

- **Bellman-Ford(G, s)**

//执行单源最短路径算法

```
for  $i \leftarrow 1$  to  $|V| - 1$  do
  for  $(u, v) \in E$  do
    if  $dist[u] + w(u, v) < dist[v]$  then
       $dist[v] \leftarrow dist[u] + w(u, v)$ 
       $pred[v] \leftarrow u$ 
    end
  end
end
for  $(u, v) \in E$  do
  if  $dist[u] + w(u, v) < dist[v]$  then
    print 存在负环
    break
  end
end
```

更新辅助数组

伪代码

- **Bellman-Ford(G, s)**

//执行单源最短路径算法

```
for  $i \leftarrow 1$  to  $|V| - 1$  do
  for  $(u, v) \in E$  do
    if  $dist[u] + w(u, v) < dist[v]$  then
       $dist[v] \leftarrow dist[u] + w(u, v)$ 
       $pred[v] \leftarrow u$ 
    end
  end
end
for  $(u, v) \in E$  do
  if  $dist[u] + w(u, v) < dist[v]$  then
    print 存在负环
    break
  end
end
```

判断是否存在负环

时间复杂度分析

- **Bellman-Ford(G, s)**

输入: 图 $G = \langle V, E, W \rangle$, 源点 s

输出: 单源最短路径 P

新建一维数组 $dist[1..|V|]$, $pred[1..|V|]$

//初始化

for $u \in V$ do

$dist[u] \leftarrow \infty$

$pred[u] \leftarrow NULL$

end

$dist[s] \leftarrow 0$

} $O(|V|)$

时间复杂度分析

- **Bellman-Ford(G, s)**

//执行单源最短路径算法

```
for  $i \leftarrow 1$  to  $|V| - 1$  do
  for  $(u, v) \in E$  do
    if  $dist[u] + w(u, v) < dist[v]$  then
       $dist[v] \leftarrow dist[u] + w(u, v)$ 
       $pred[v] \leftarrow u$ 
    end
  end
end
for  $(u, v) \in E$  do
  if  $dist[u] + w(u, v) < dist[v]$  then
    print 存在负环
    break
  end
end
```

$O(|E|)$

时间复杂度分析

- **Bellman-Ford(G, s)**

//执行单源最短路径算法

```
for  $i \leftarrow 1$  to  $|V| - 1$  do
  for  $(u, v) \in E$  do
    if  $dist[u] + w(u, v) < dist[v]$  then
       $dist[v] \leftarrow dist[u] + w(u, v)$ 
       $pred[v] \leftarrow u$ 
    end
  end
end
for  $(u, v) \in E$  do
  if  $dist[u] + w(u, v) < dist[v]$  then
    print 存在负环
    break
  end
end
```

$O(|E|)$ $O(|E| \cdot |V|)$

$O(|E|)$

时间复杂度分析

- **Bellman-Ford(G, s)**

//执行单源最短路径算法

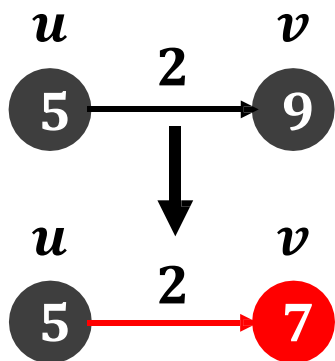
```
for  $i \leftarrow 1$  to  $|V| - 1$  do
  for  $(u, v) \in E$  do
    if  $dist[u] + w(u, v) < dist[v]$  then
       $dist[v] \leftarrow dist[u] + w(u, v)$ 
       $pred[v] \leftarrow u$ 
    end
  end
end
for  $(u, v) \in E$  do
  if  $dist[u] + w(u, v) < dist[v]$  then
    print 存在负环
    break
  end
end
```

时间复杂度 $O(|E| \cdot |V|)$

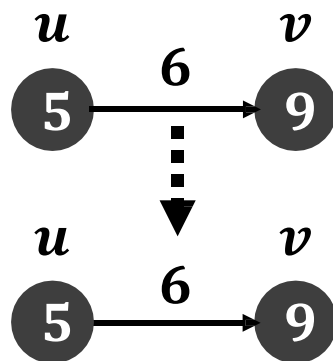
算法思想

- **Bellman-Ford**算法

- 挑战1：图中存在负权边时，如何求解单源最短路径？
 - 解决方案：每轮对所有边进行松弛，持续迭代 $|V| - 1$ 轮
- 挑战2：图中存在负权边时，如何发现源点可达负环？
 - 解决方案：若第 $|V|$ 轮仍松弛成功，存在源点 s 可达的负环



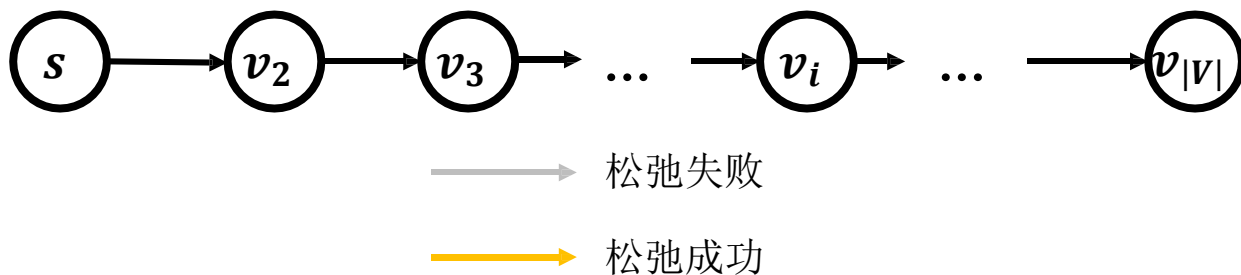
松弛成功



松弛失败

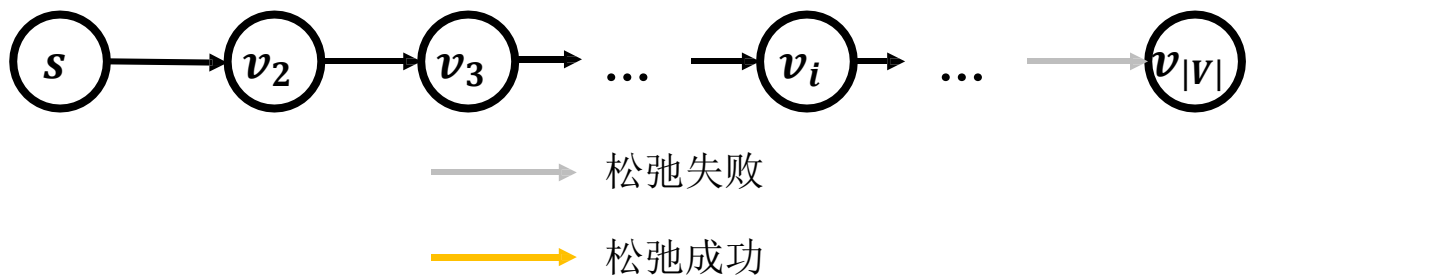
算法正确性

- 挑战1：图中存在负权边时，如何求解单源最短路径？
 - 解决方案：每轮对所有边进行松弛，持续迭代 $|V| - 1$ 轮
- 最坏情况
 - 非环路的路径 $\langle s, v_2, v_3, \dots, v_{|V|} \rangle$ 至多经过 $|V| - 1$ 条边



算法正确性

- 挑战1：图中存在负权边时，如何求解单源最短路径？
 - 解决方案：每轮对所有边进行松弛，持续迭代 $|V| - 1$ 轮
- 最坏情况
 - 非环路的路径 $\langle s, v_2, v_3, \dots, v_{|V|} \rangle$ 至多经过 $|V| - 1$ 条边



算法正确性

- 挑战1：图中存在负权边时，如何求解单源最短路径？
 - 解决方案：每轮对所有边进行松弛，持续迭代 $|V| - 1$ 轮
- 最坏情况
 - 非环路的路径 $\langle s, v_2, v_3, \dots, v_{|V|} \rangle$ 至多经过 $|V| - 1$ 条边



算法正确性

- 挑战1：图中存在负权边时，如何求解单源最短路径？
 - 解决方案：每轮对所有边进行松弛，持续迭代 $|V| - 1$ 轮
- 最坏情况
 - 非环路的路径 $\langle s, v_2, v_3, \dots, v_{|V|} \rangle$ 至多经过 $|V| - 1$ 条边



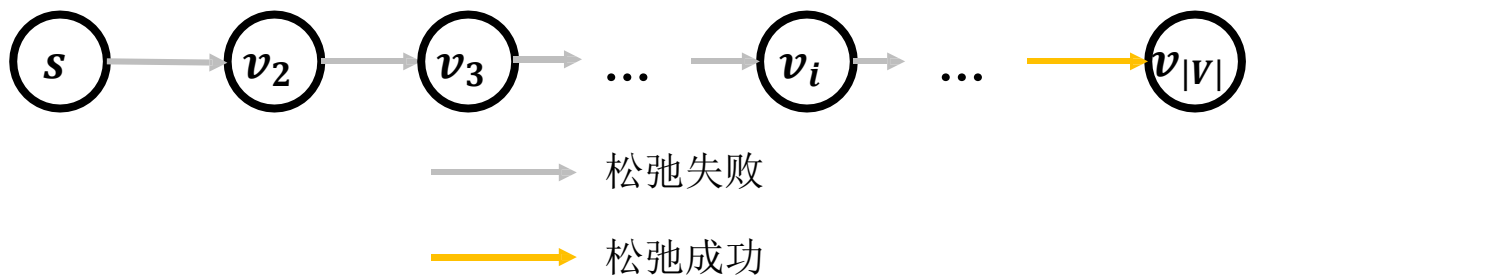
算法正确性

- 挑战1：图中存在负权边时，如何求解单源最短路径？
 - 解决方案：每轮对所有边进行松弛，持续迭代 $|V| - 1$ 轮
- 最坏情况
 - 非环路的路径 $\langle s, v_2, v_3, \dots, v_{|V|} \rangle$ 至多经过 $|V| - 1$ 条边



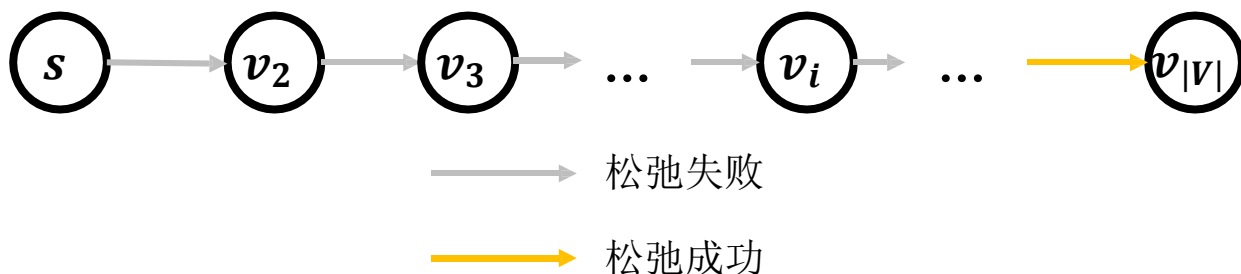
算法正确性

- 挑战1：图中存在负权边时，如何求解单源最短路径？
 - 解决方案：每轮对所有边进行松弛，持续迭代 $|V| - 1$ 轮
- 最坏情况
 - 非环路的路径 $\langle s, v_2, v_3, \dots, v_{|V|} \rangle$ 至多经过 $|V| - 1$ 条边



算法正确性

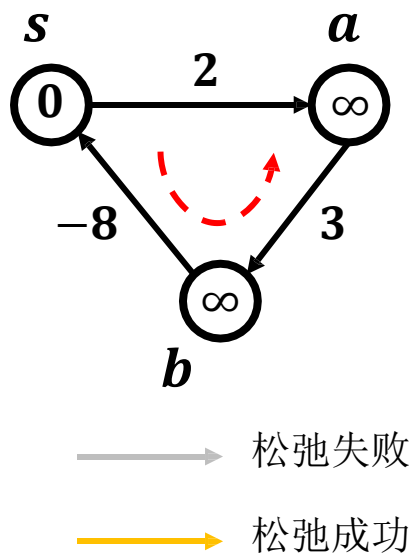
- 挑战1: 图中存在负权边时, 如何求解单源最短路径?
 - 解决方案: 每轮对所有边进行松弛, 持续迭代 $|V| - 1$ 轮
- 最坏情况
 - 非环路的路径 $\langle s, v_2, v_3, \dots, v_{|V|} \rangle$ 至多经过 $|V| - 1$ 条边



最坏情况下进行 $|V| - 1$ 轮松弛操作, 可以保证求得单源最短路径

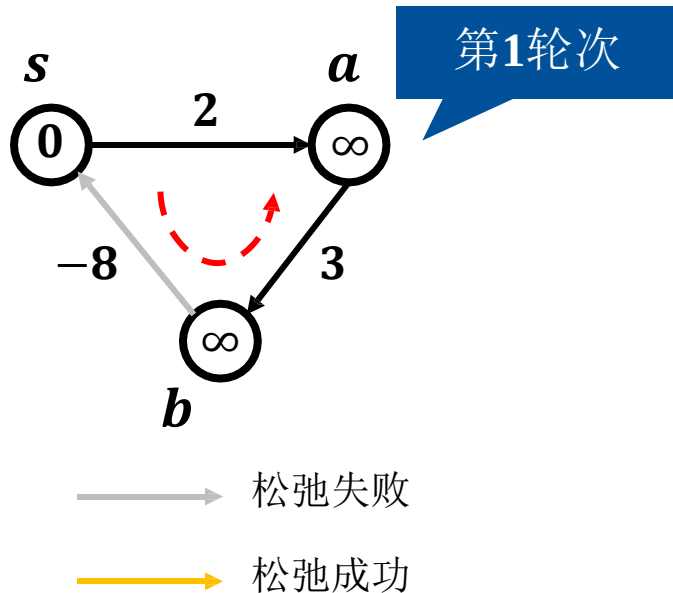
算法正确性

- 挑战2：图中存在负权边时，如何发现源点可达负环？
 - 解决方案：若第 $|V|$ 轮仍松弛成功，存在源点 s 可达的负环
- 若源点 s 可达负环，可松弛成功无限次



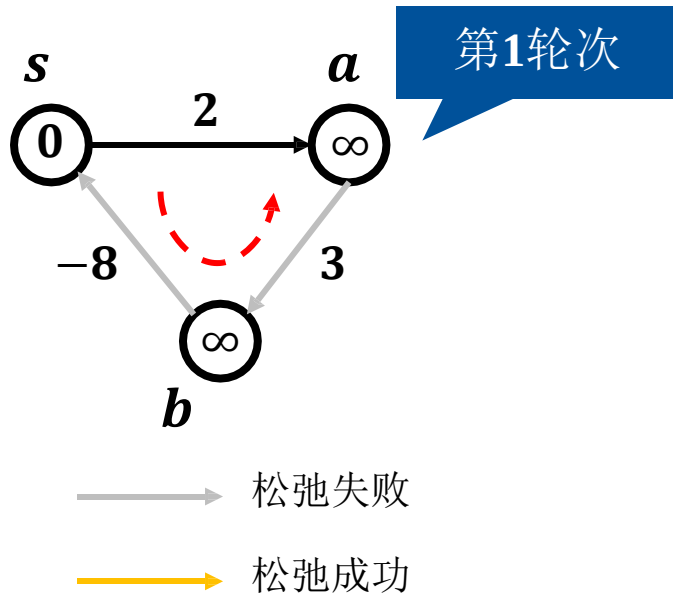
算法正确性

- 挑战2：图中存在负权边时，如何发现源点可达负环？
 - 解决方案：若第 $|V|$ 轮仍松弛成功，存在源点 s 可达的负环
- 若源点 s 可达负环，可松弛成功无限次



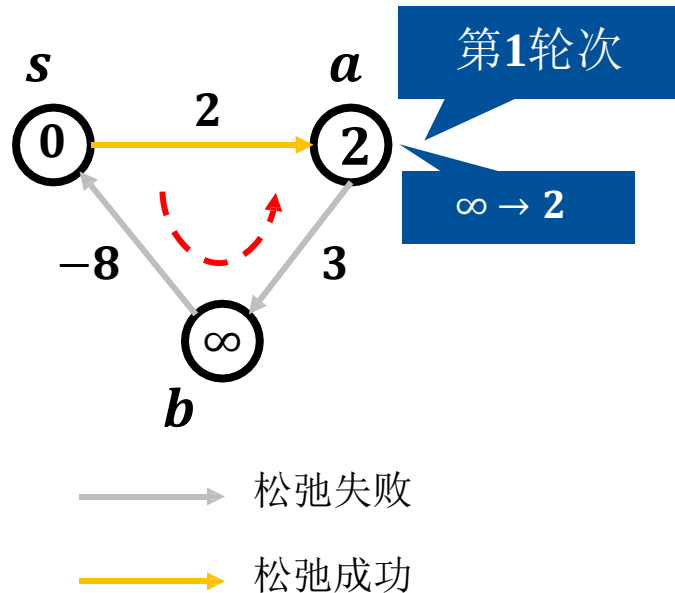
算法正确性

- 挑战2：图中存在负权边时，如何发现源点可达负环？
 - 解决方案：若第 $|V|$ 轮仍松弛成功，存在源点 s 可达的负环
- 若源点 s 可达负环，可松弛成功无限次



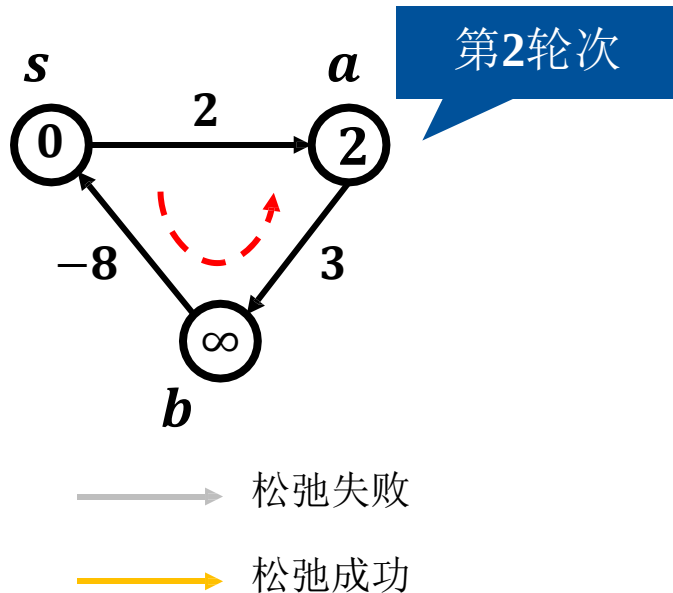
算法正确性

- 挑战2：图中存在负权边时，如何发现源点可达负环？
 - 解决方案：若第 $|V|$ 轮仍松弛成功，存在源点 s 可达的负环
- 若源点 s 可达负环，可松弛成功无限次



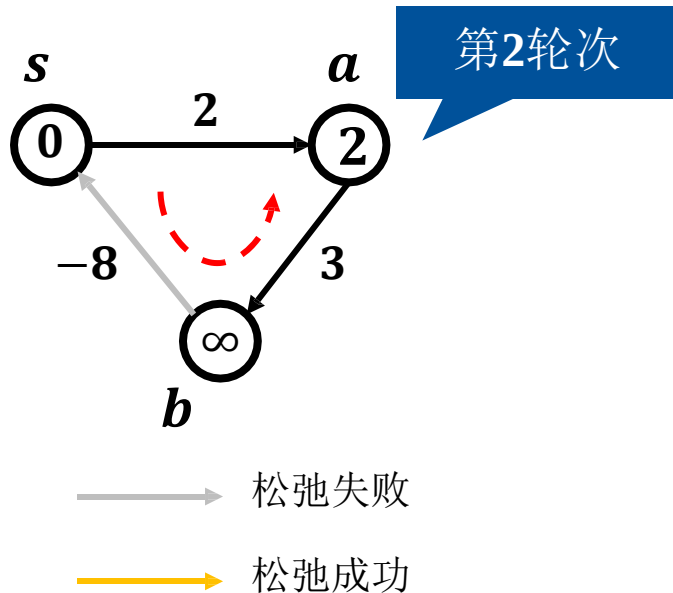
算法正确性

- 挑战2：图中存在负权边时，如何发现源点可达负环？
 - 解决方案：若第 $|V|$ 轮仍松弛成功，存在源点 s 可达的负环
- 若源点 s 可达负环，可松弛成功无限次



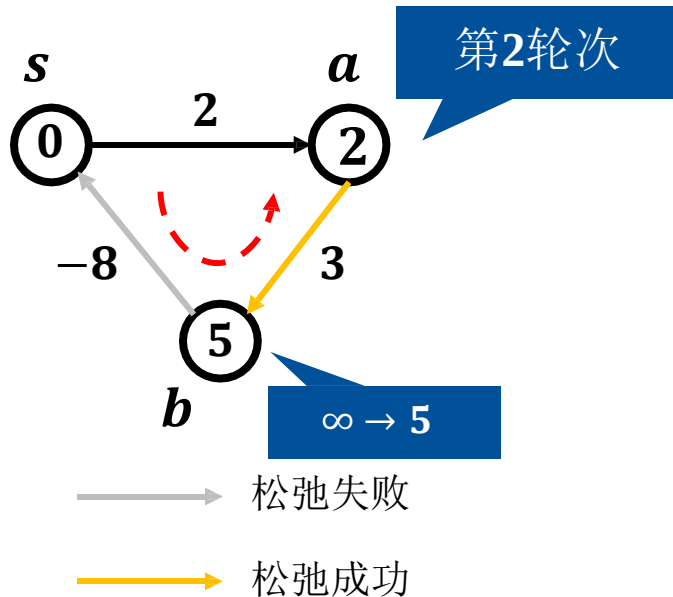
算法正确性

- 挑战2：图中存在负权边时，如何发现源点可达负环？
 - 解决方案：若第 $|V|$ 轮仍松弛成功，存在源点 s 可达的负环
- 若源点 s 可达负环，可松弛成功无限次



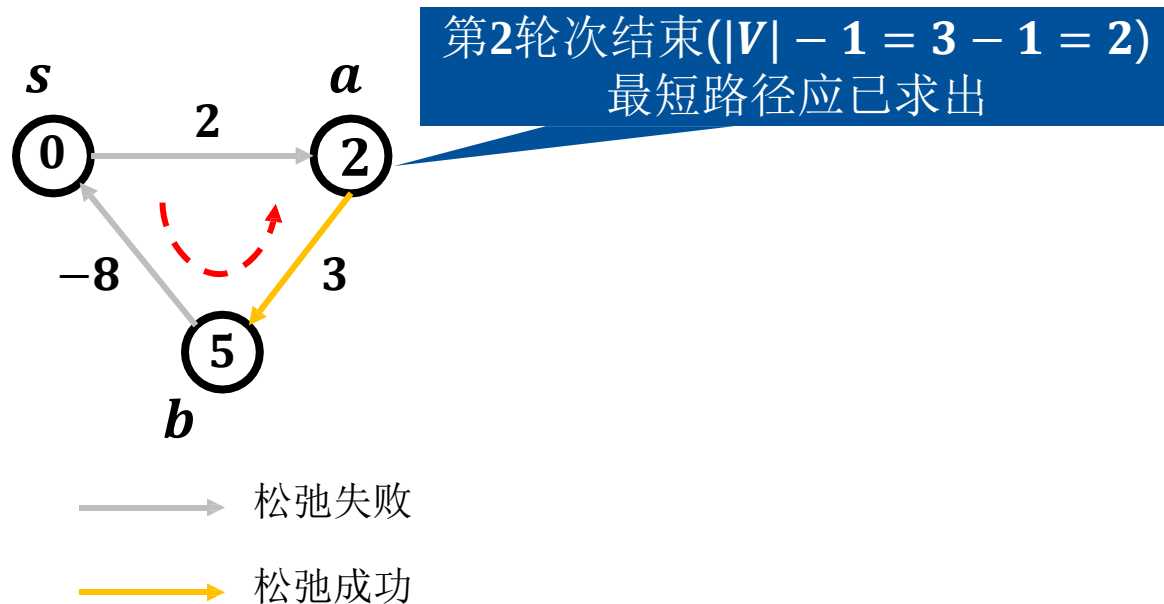
算法正确性

- 挑战2：图中存在负权边时，如何发现源点可达负环？
 - 解决方案：若第 $|V|$ 轮仍松弛成功，存在源点 s 可达的负环
- 若源点 s 可达负环，可松弛成功无限次



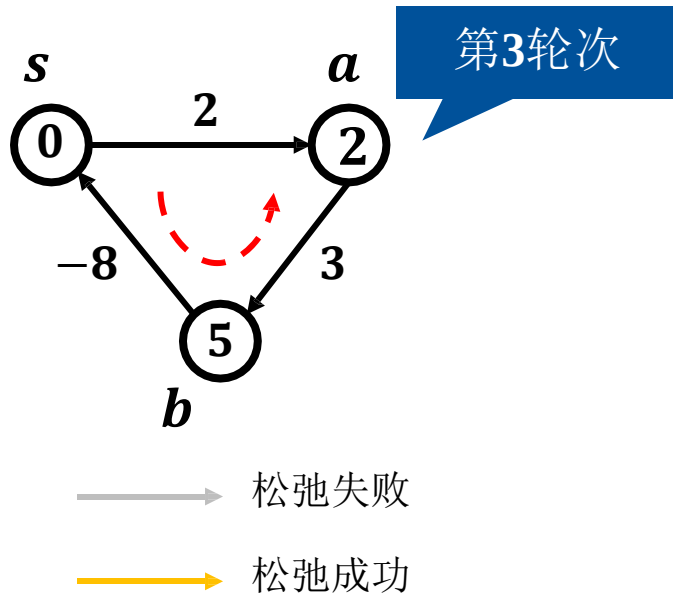
算法正确性

- 挑战2：图中存在负权边时，如何发现源点可达负环？
 - 解决方案：若第 $|V|$ 轮仍松弛成功，存在源点 s 可达的负环
- 若源点 s 可达负环，可松弛成功无限次



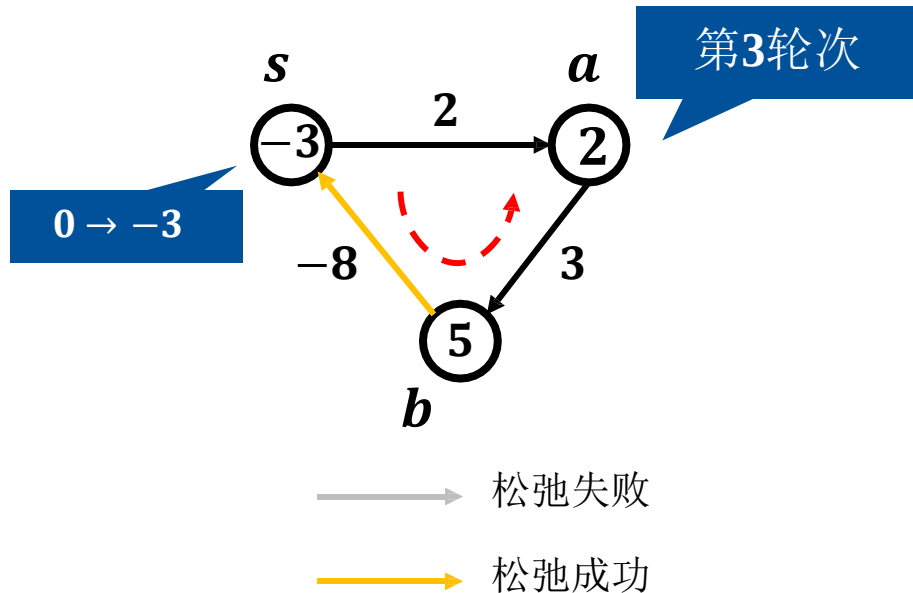
算法正确性

- 挑战2：图中存在负权边时，如何发现源点可达负环？
 - 解决方案：若第 $|V|$ 轮仍松弛成功，存在源点 s 可达的负环
- 若源点 s 可达负环，可松弛成功无限次



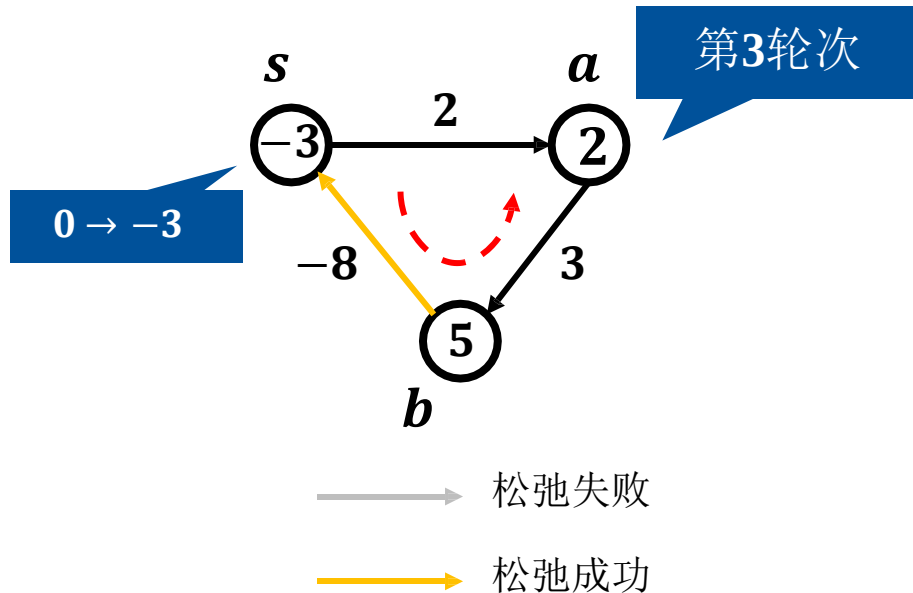
算法正确性

- 挑战2：图中存在负权边时，如何发现源点可达负环？
 - 解决方案：若第 $|V|$ 轮仍松弛成功，存在源点 s 可达的负环
- 若源点 s 可达负环，可松弛成功无限次



算法正确性

- 挑战2：图中存在负权边时，如何发现源点可达负环？
 - 解决方案：若第 $|V|$ 轮仍松弛成功，存在源点 s 可达的负环
- 若源点 s 可达负环，可松弛成功无限次



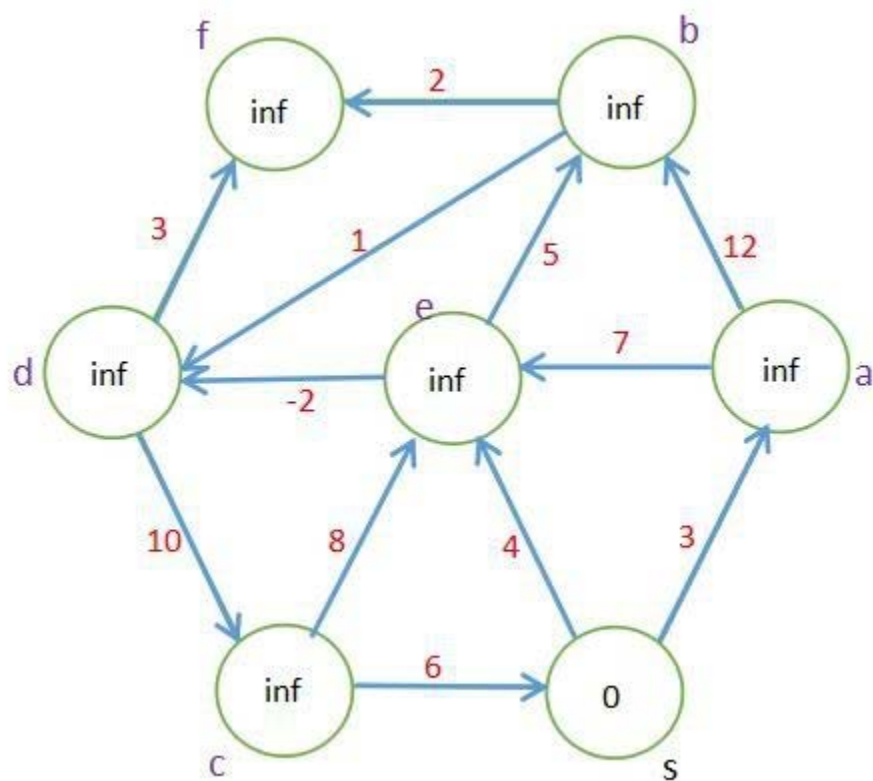
第 $|V|$ 轮仍松弛成功的原因：存在源点可达的负环



SPFA算法

SPFA算法也是一个求单源最短路径的算法，全称是Shortest Path Faster Algorithm (SPFA)。

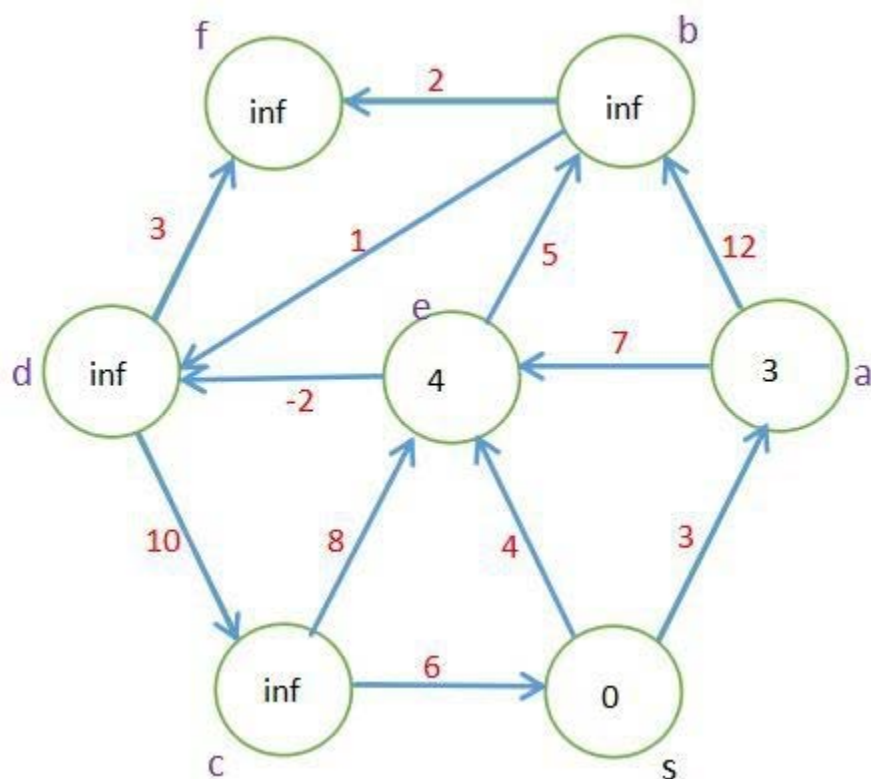
当给定的图存在负权边时，狄克斯特拉不再适合，而Bellman-Ford算法的时间复杂度又过高，此时可以采用SPFA算法。Bellman-FordSPFA算法事实上是对Bellman-Ford算法的优化。Bellman-Ford算法在每次更新时，遍历了所有的边，而也如前面看到的，每次遍历未必会真的更新最短距离。SPFA算法就是对该步骤的优化。每次只有当前的起点到源点的距离变小了，该点连通的其他点才会变小。只要一个结点的边变小了，我们就把他放进队列(其他的也行)中，只要队列中还有值，我们就继续更新，更新到的点也加入队列，如此往复，直到标记全部的点。



初始化和前面的几种最短路算法一样。把 s 入队

队头 s 队尾

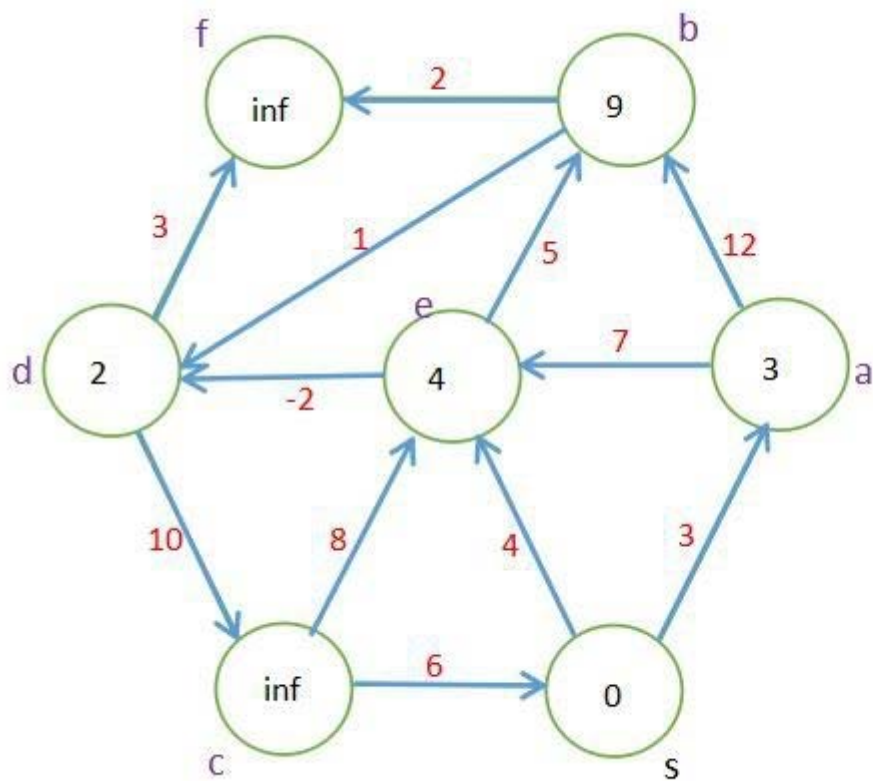
知乎 @WorrywartsL



第一轮：s 出队。遍历 s 所有的出边，到 e 和 a 的距离都能更新。e 和 a 入队，

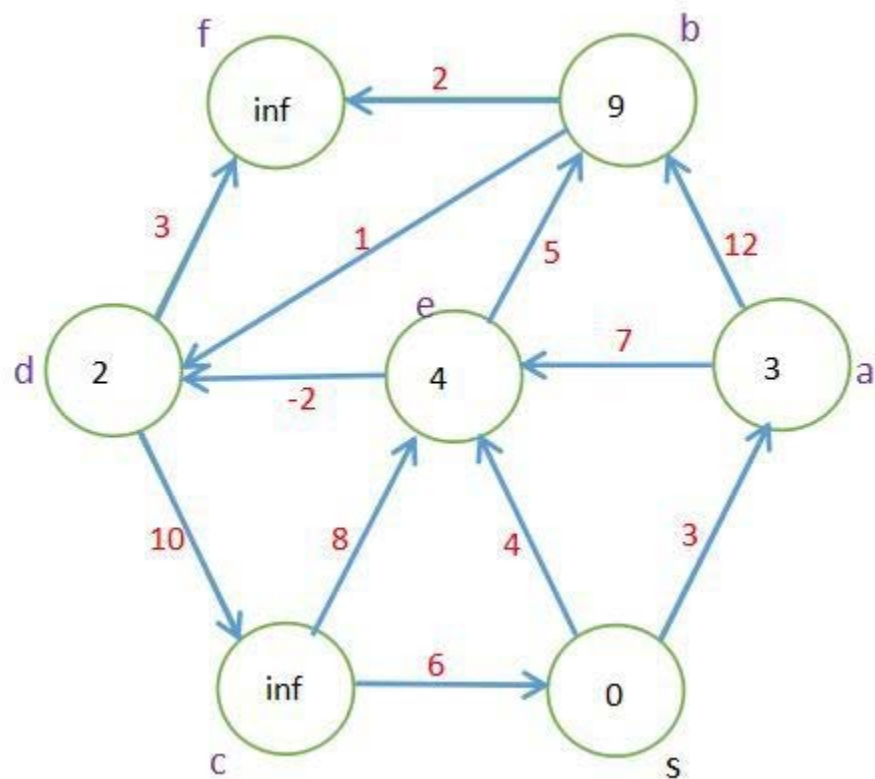
队头 e、a 队尾

知乎 @WorrywartsL



第二轮：e 出队。遍历 e 的所有出边，b 和 d 都能更新。b 和 d 入队。

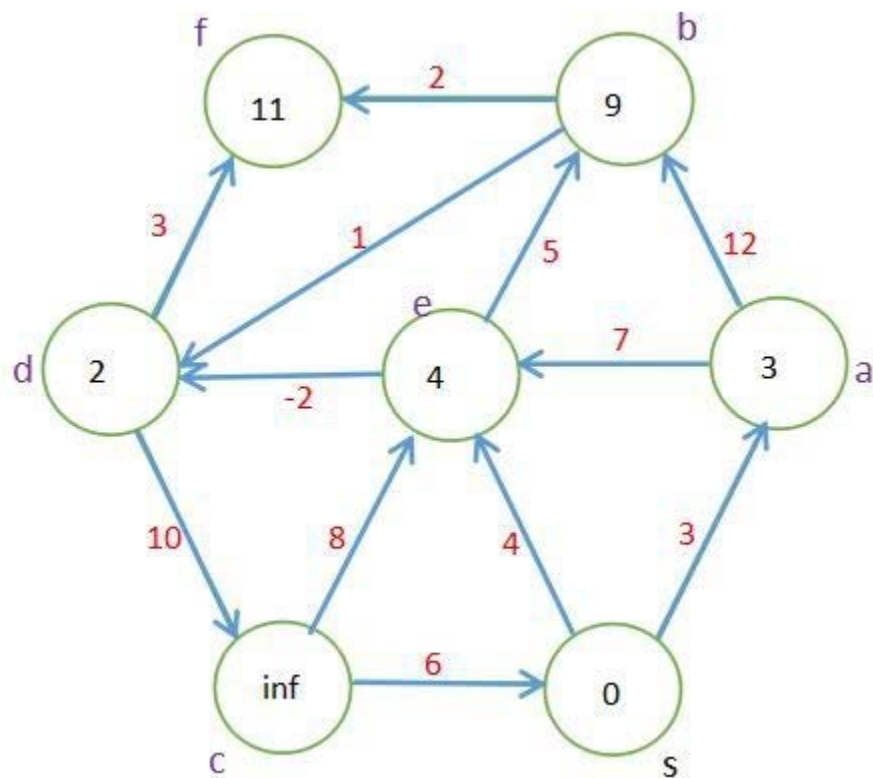
队头 a、b、d 队尾



第三轮：a 出队。遍历 a 的所有出边 e 和 b。都无法更新。

队头 b、d 队尾

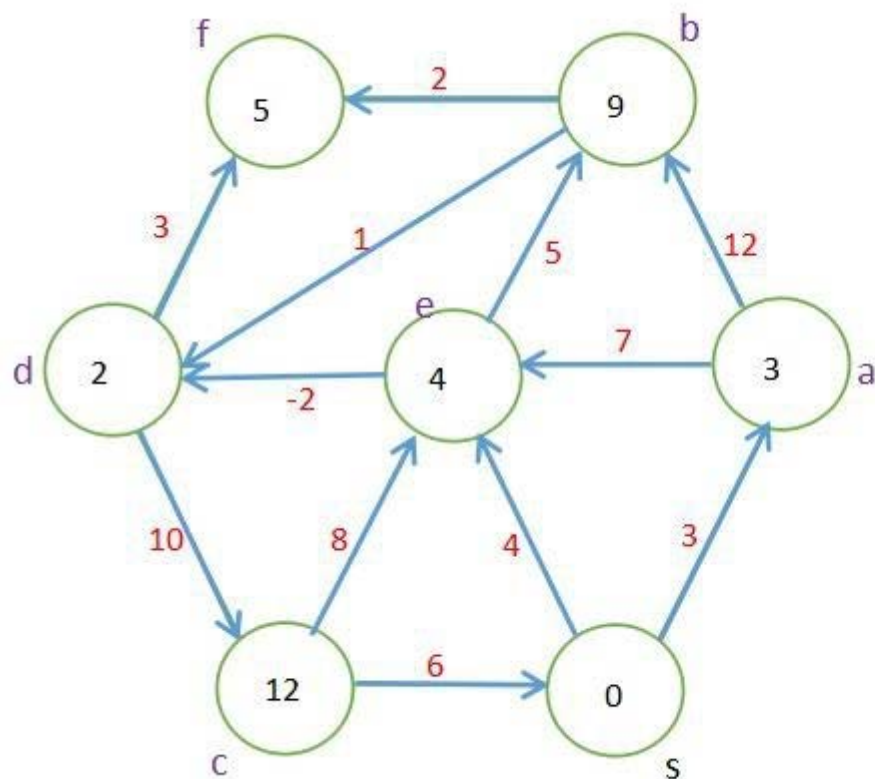
知乎 @WorrywartsL



第四轮：b 出队。遍历 b 的所有出边。可以更新 f，无法更新 d。f 入队。

队头 d、f 队尾

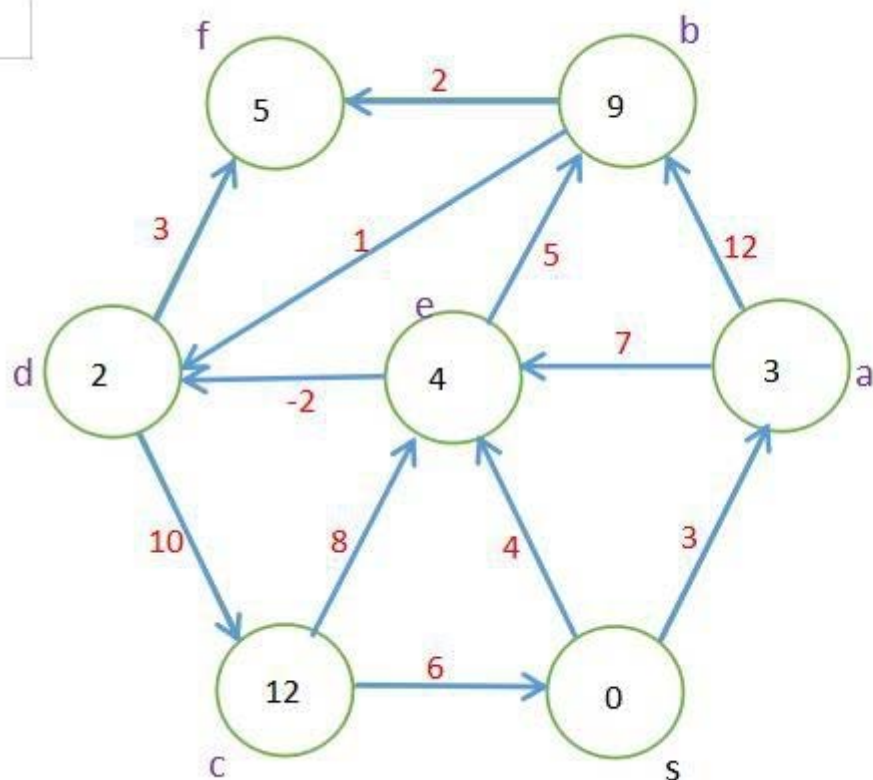
知乎 @WorrywartsL



第五轮：d 出队。遍历 d 的所有出边。可以更新 f 和 c。f 和 c 入队。

队头 f、c 队尾

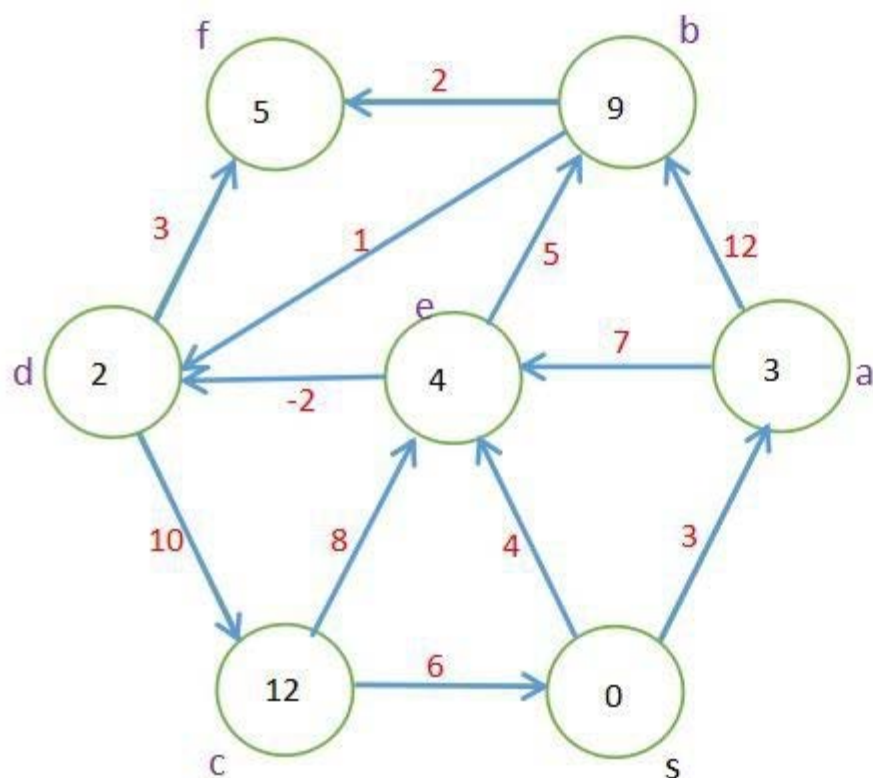
知乎 @WorrywartsL



第六轮：f 出队。遍历 f 所有的出边。没有出边。

队头 c 队尾

知乎 @WorrywartsL



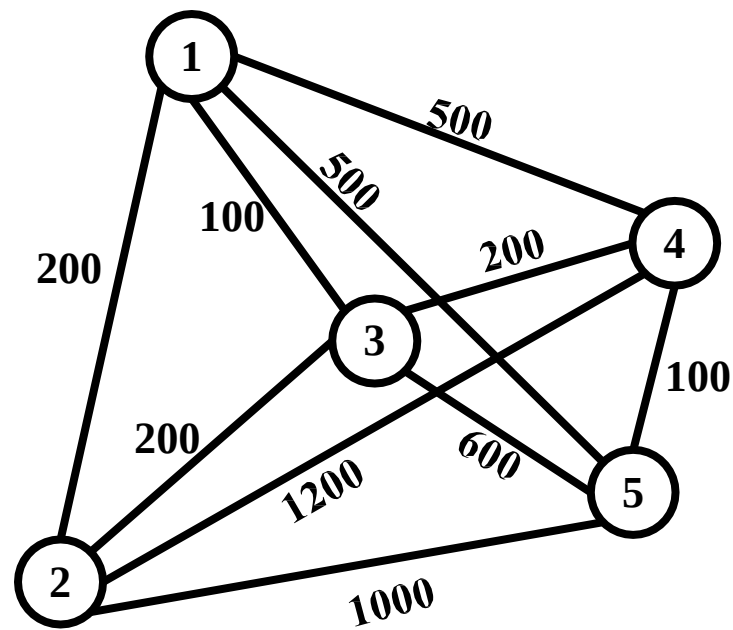
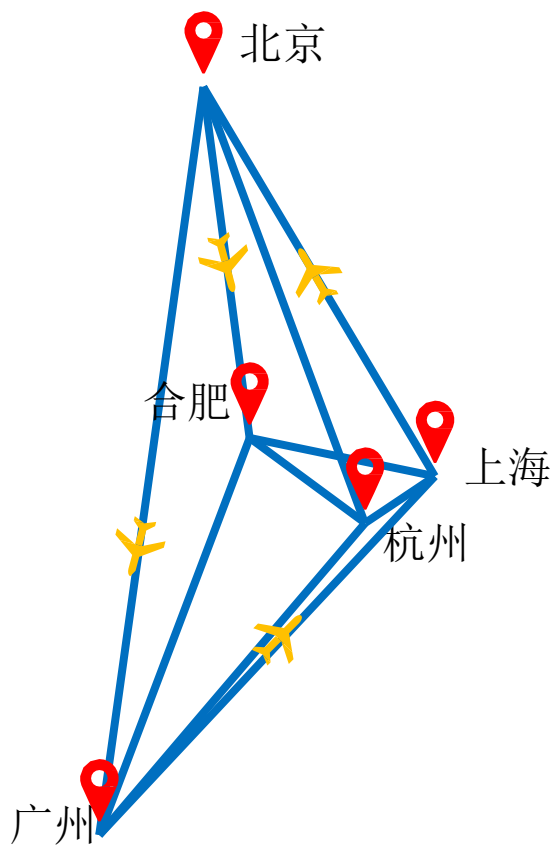
第七轮：c 出队。遍历 c 的所有出边 e 和 s。都无法更新。队空结束。

队空

知乎 @WorrywartsL

问题背景

- 航班价格



如何求出所有城市之间的最低航班价格？

问题定义

所有点对最短路径问题

All Pairs Shortest Paths

输入

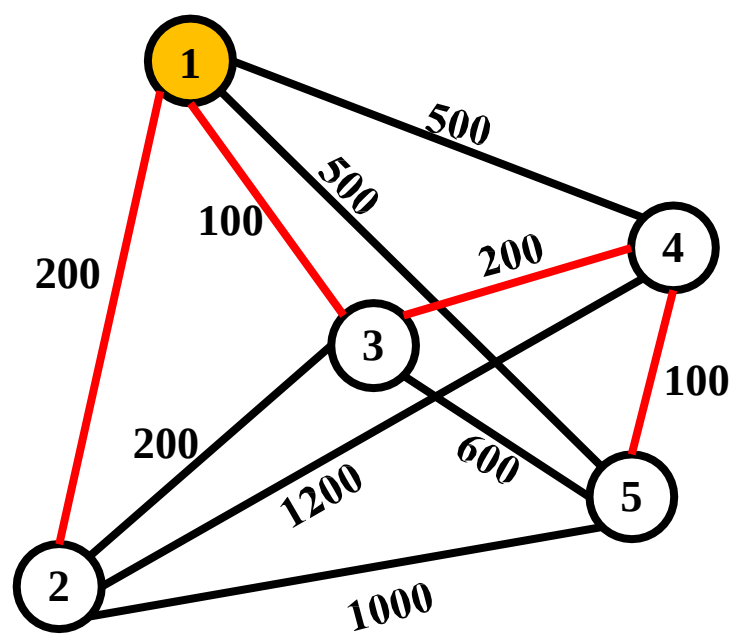
- 带权图 $G = \langle V, E, W \rangle$, W 为边权

输出

- $\forall u, v \in V$, 从 u 到 v 的最短路径

直观思路

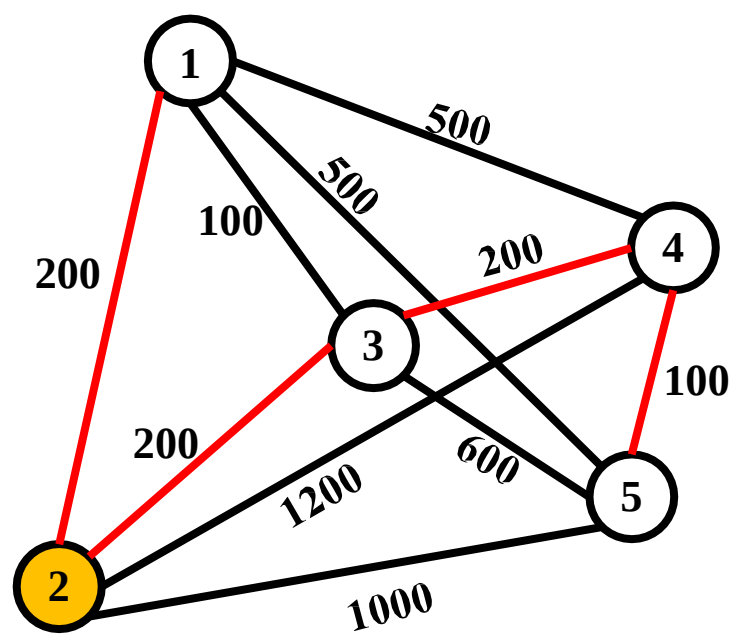
- 使用**Dijkstra**算法依次求解所有点



	1	2	3	4	5
<i>u</i>					
1	0	200	100	300	400
2					
3					
4					
5					

直观思路

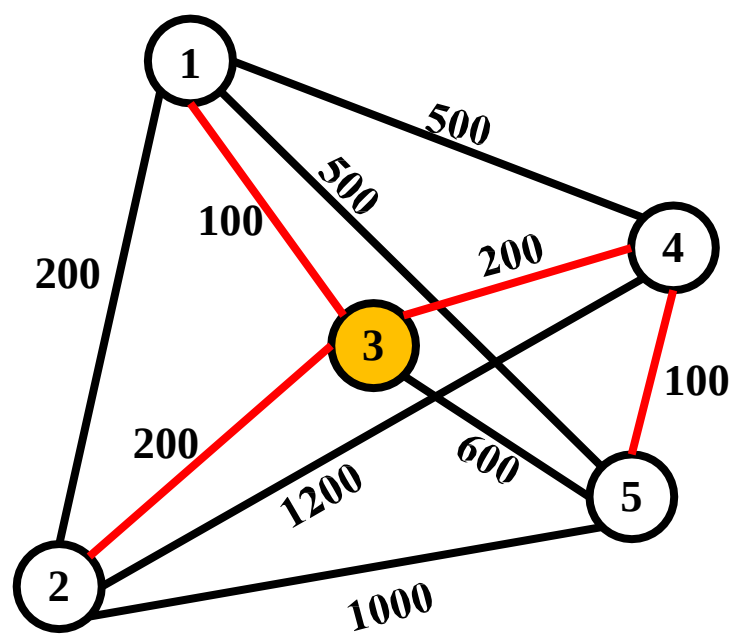
- 使用**Dijkstra**算法依次求解所有点



	1	2	3	4	5
<i>u</i>					
1	0	200	100	300	400
2	200	0	200	400	500
3					
4					
5					

直观思路

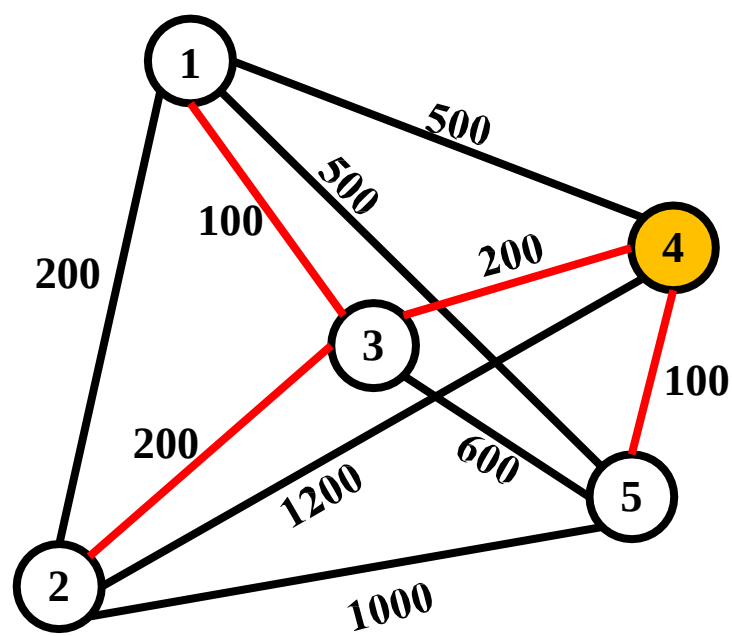
- 使用**Dijkstra**算法依次求解所有点



	1	2	3	4	5
<i>u</i>					
1	0	200	100	300	400
2	200	0	200	400	500
3	100	200	0	200	300
4					
5					

直观思路

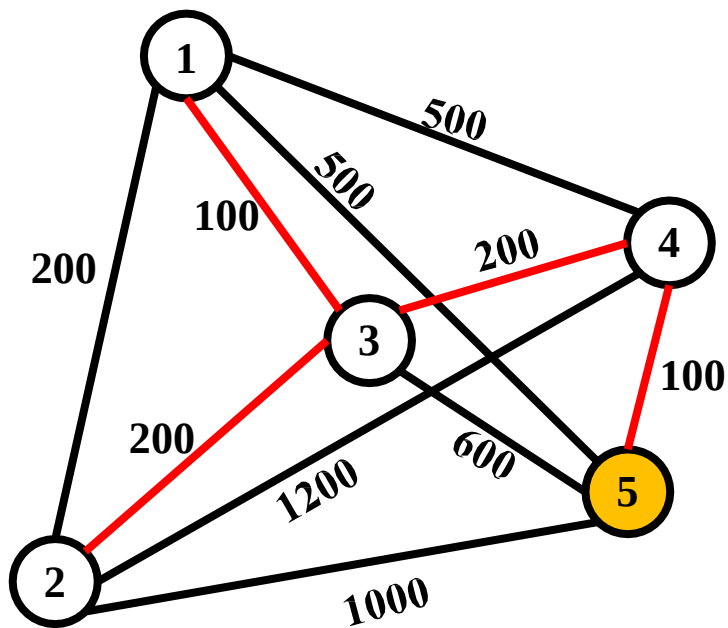
- 使用**Dijkstra**算法依次求解所有点



	1	2	3	4	5
<i>u</i>					
1	0	200	100	300	400
2	200	0	200	400	500
3	100	200	0	200	300
4	300	400	200	0	100
5					

直观思路

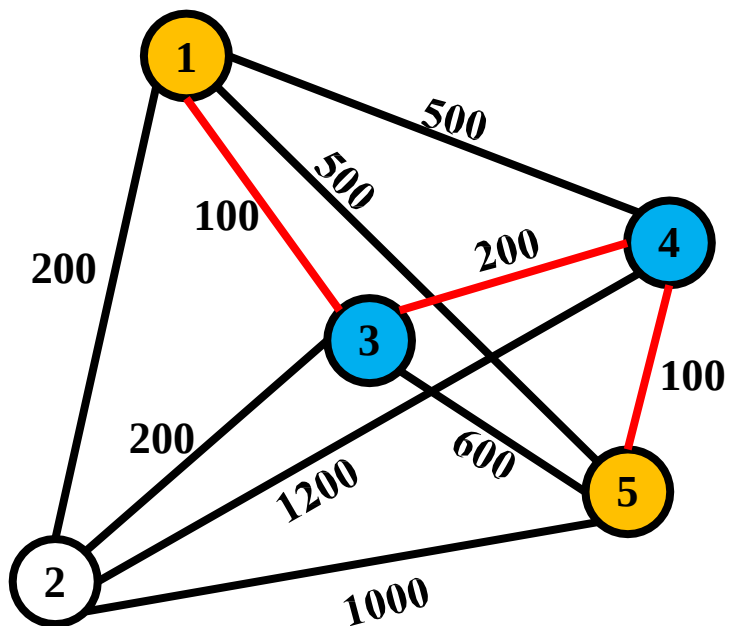
- 使用**Dijkstra**算法依次求解所有点



	1	2	3	4	5
u					
1	0	200	100	300	400
2	200	0	200	400	500
3	100	200	0	200	300
4	300	400	200	0	100
5	400	500	300	100	0

直观思路

- 使用**Dijkstra**算法依次求解所有点
- 存在重叠子问题

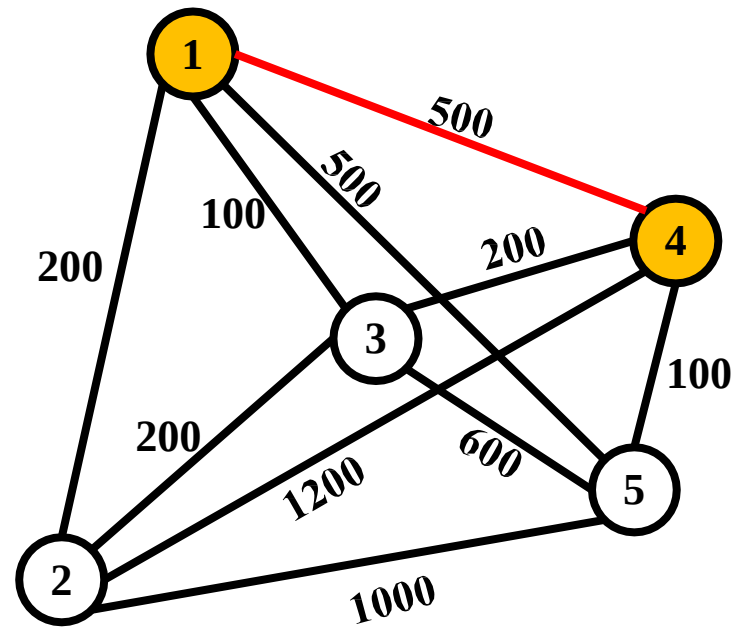


从1到5的最短路径: **1→3→4→5**

从3到5的最短路径: **3→4→5**

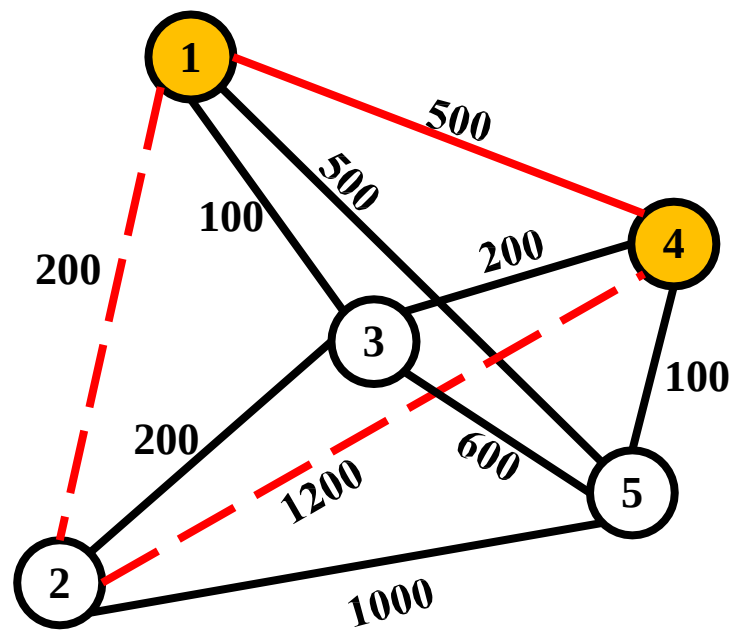
观察松弛过程

- 从1到4的路径更新
 - 可从前1个点中选择点经过：500



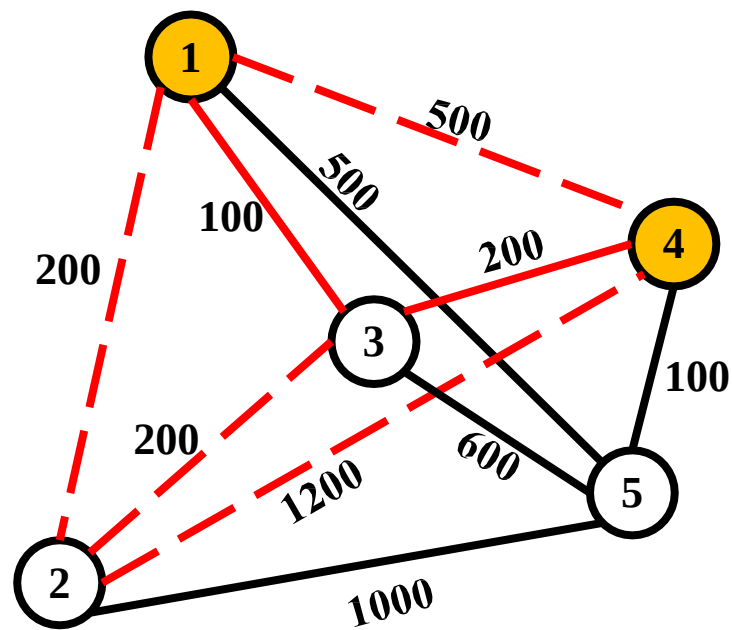
观察松弛过程

- 从1到4的路径更新
 - 可从前1个点中选择点经过：500
 - 可从前2个点中选择点经过：500



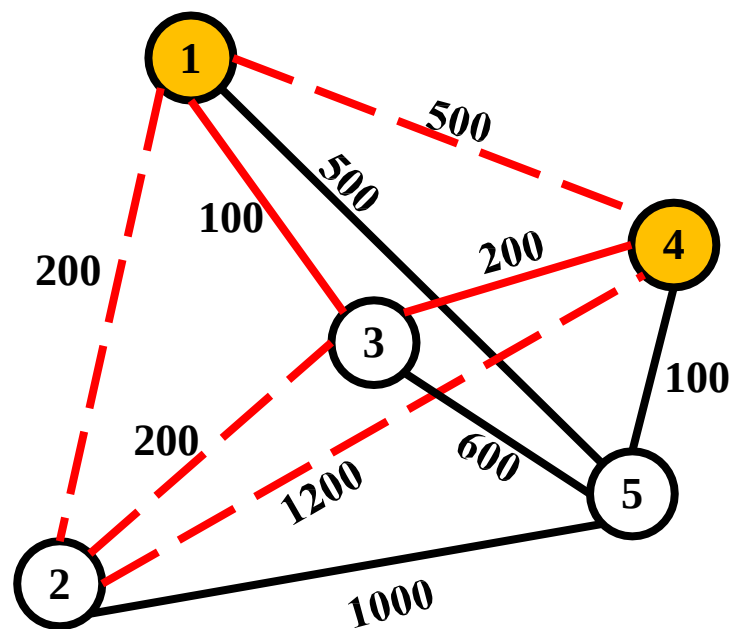
观察松弛过程

- 从1到4的路径更新
 - 可从前1个点中选择点经过：500
 - 可从前2个点中选择点经过：500
 - 可从前3个点中选择点经过：300



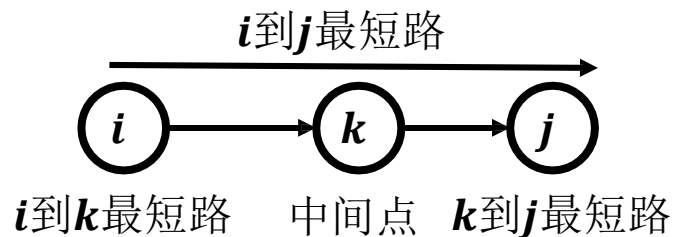
观察松弛过程

- 从1到4的路径更新
 - 可从前1个点中选择点经过：500
 - 可从前2个点中选择点经过：500
 - 可从前3个点中选择点经过：300
 - 可从前 k 个点中选择点经过：...

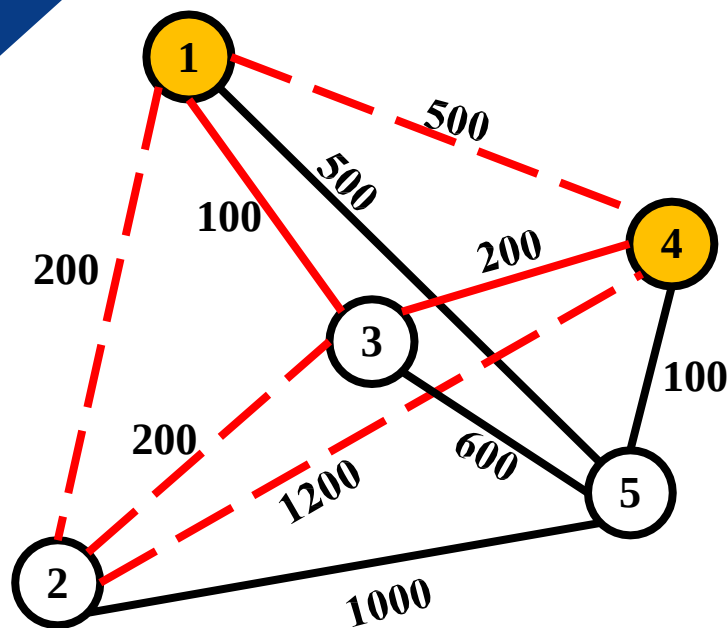


观察松弛过程

- 从1到4的路径更新
 - 可从前1个点中选择点经过：500
 - 可从前2个点中选择点经过：500
 - 可从前3个点中选择点经过：300
 - 可从前 k 个点中选择点经过：
...

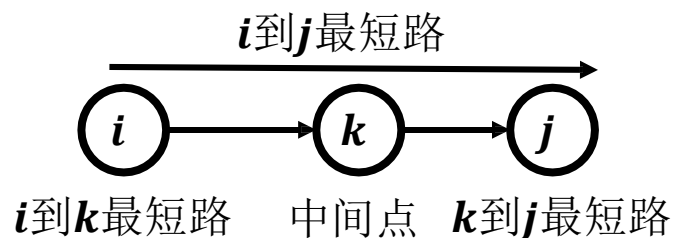


可经过的中间点越多
距离逐渐变短

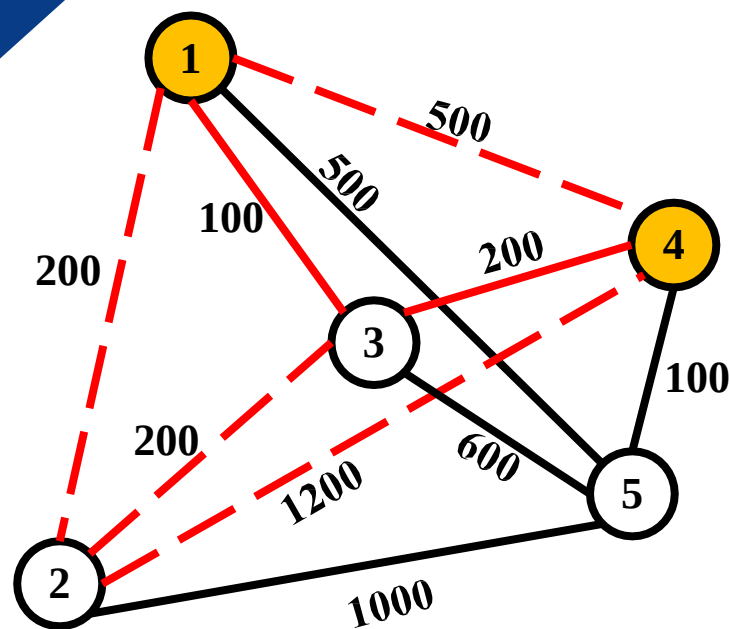


观察松弛过程

- 从1到4的路径更新
 - 可从前1个点中选择点经过：500
 - 可从前2个点中选择点经过：500
 - 可从前3个点中选择点经过：300
 - 可从前 k 个点中选择点经过：
...



可经过的中间点越多
距离逐渐变短



重叠子问题、最优子结构启发使用动态规划求解

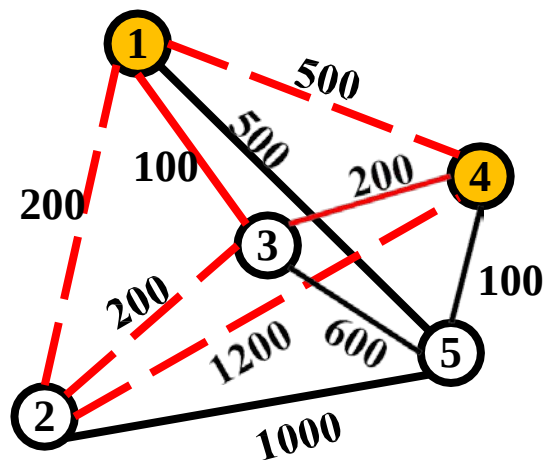
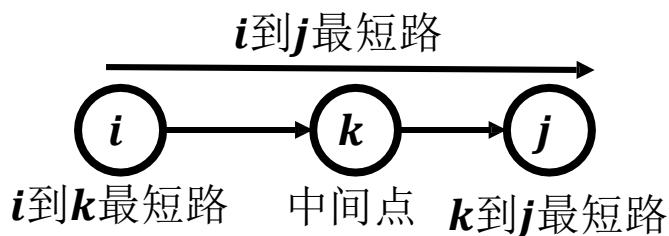
动态规划：问题结构分析

- 给出问题表示

- $D[k, i, j]$: 可从前 k 个点经过时, i 到 j 的最短距离

- 从1到4的路径更新

- 可从前1个点中选择点经过: $D[1, 1, 4] = 500$
 - 可从前2个点中选择点经过: $D[2, 1, 4] = 500$
 - 可从前3个点中选择点经过: $D[3, 1, 4] = 300$



问题结构分析

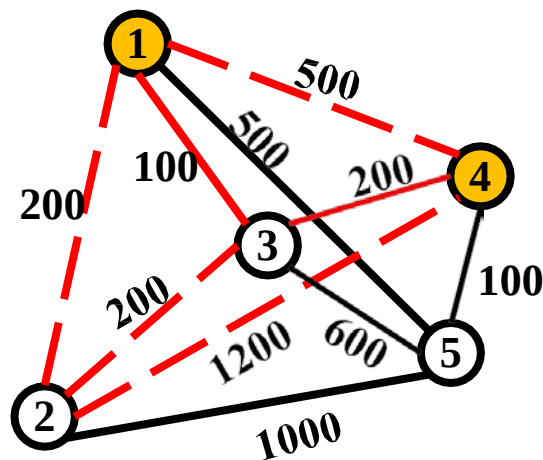
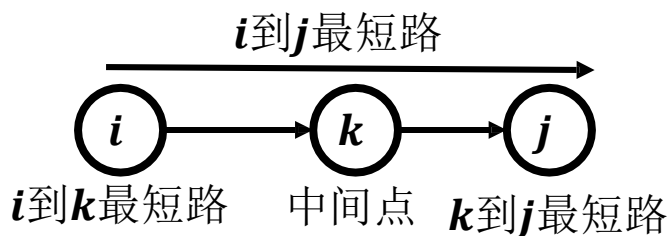
递推关系建立

自底向上计算

最优方案追踪

动态规划：问题结构分析

- 给出问题表示
 - $D[k, i, j]$: 可从前 k 个点经过时, i 到 j 的最短距离
- 从1到4的路径更新
 - 可从前1个点中选择点经过: $D[1, 1, 4] = 500$
 - 可从前2个点中选择点经过: $D[2, 1, 4] = 500$
 - 可从前3个点中选择点经过: $D[3, 1, 4] = 300$
- 明确原始问题
 - $D[|V|, i, j]$



问题结构分析

递推关系建立

自底向上计算

最优方案追踪

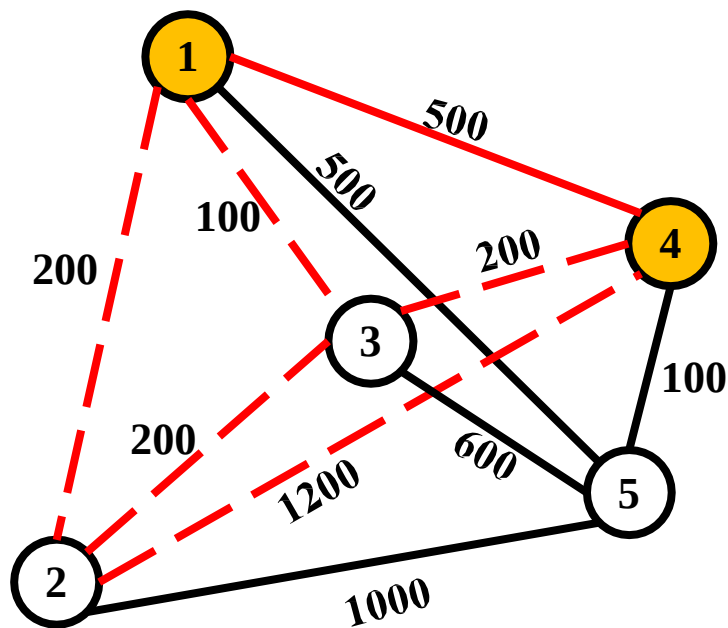
动态规划：递推关系建立

- 如果不选第 k 个点经过
 - $D[k, i, j] = D[k - 1, i, j]$

本例中

$$k = 2, i = 1, j = 4$$

$$D[2, 1, 4] = D[1, 1, 4] = 500$$



问题结构分析

递推关系建立

自底向上计算

最优方案追踪

动态规划：递推关系建立

- 如果不选第 k 个点经过

- $D[k, i, j] = D[k - 1, i, j]$

- 如果选择第 k 个点经过

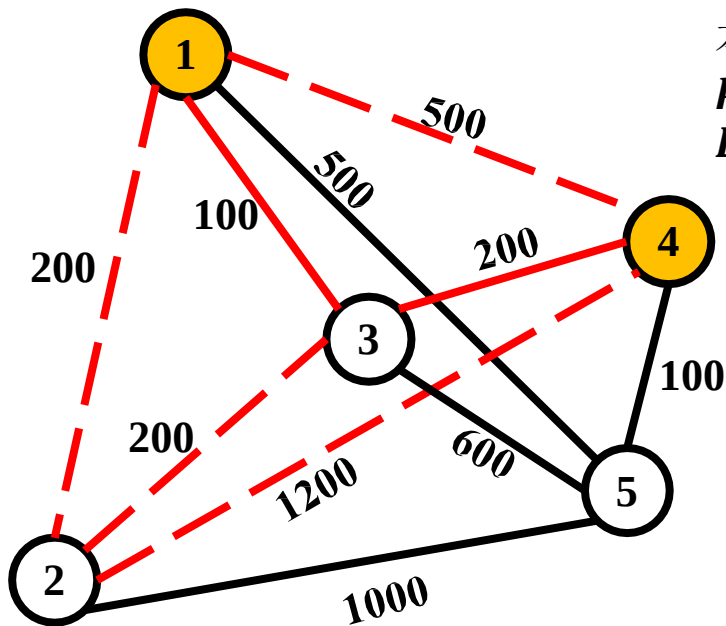
- $D[k, i, j] = D[k - 1, i, k] + D[k - 1, k, j]$

表示松弛成功

本例中

$$k = 3, i = 1, j = 4$$

$$\begin{aligned} D[3, 1, 4] &= D[2, 1, 3] + D[2, 3, 4] \\ &= 100 + 200 = 300 \end{aligned}$$



问题结构分析

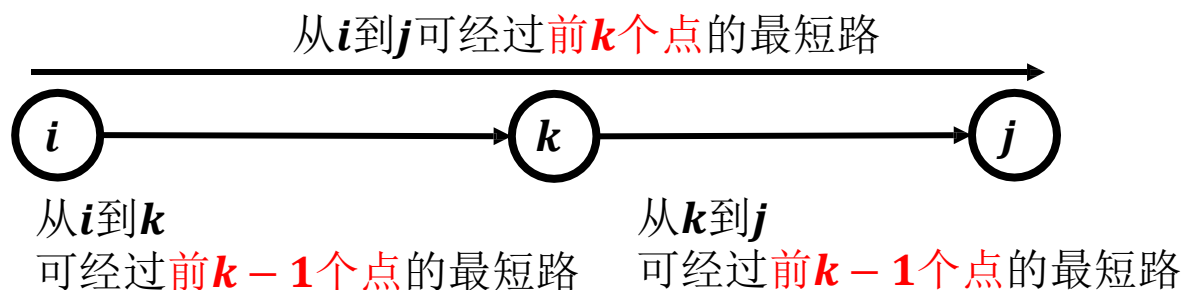
递推关系建立

自底向上计算

最优方案追踪

动态规划：递推关系建立

- 如果不选第 k 个点经过
 - $D[k, i, j] = D[k - 1, i, j]$
- 如果选择第 k 个点经过
 - $D[k, i, j] = D[k - 1, i, k] + D[k - 1, k, j]$



问题结构分析

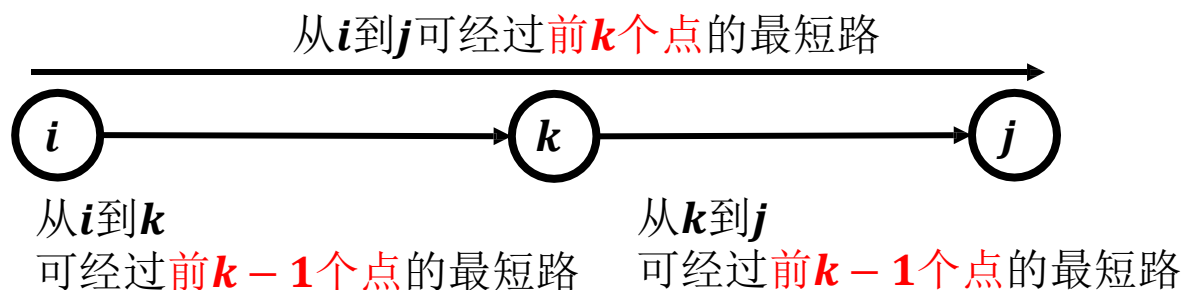
递推关系建立

自底向上计算

最优方案追踪

动态规划：递推关系建立

- 如果不选第 k 个点经过
 - $D[k, i, j] = D[k - 1, i, j]$
- 如果选择第 k 个点经过
 - $D[k, i, j] = D[k - 1, i, k] + D[k - 1, k, j]$



- $D[k, i, j] = \min\{D[k - 1, i, j], D[k - 1, i, k] + D[k - 1, k, j]\}$

问题结构分析

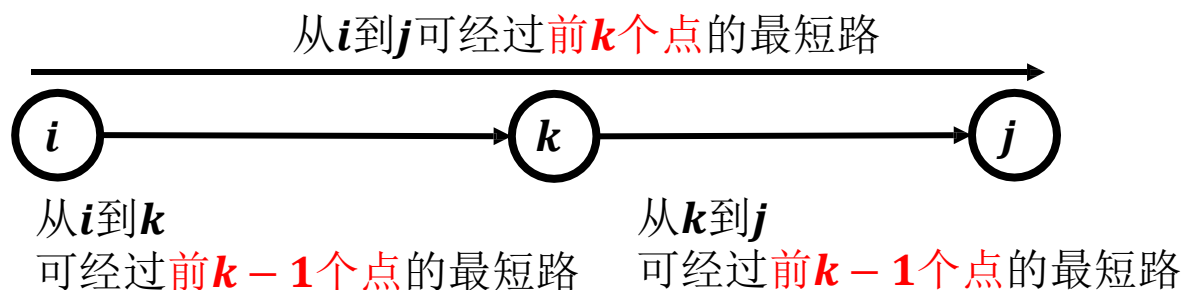
递推关系建立

自底向上计算

最优方案追踪

动态规划：递推关系建立

- 如果不选第 k 个点经过
 - $D[k, i, j] = D[k - 1, i, j]$
- 如果选择第 k 个点经过
 - $D[k, i, j] = D[k - 1, i, k] + D[k - 1, k, j]$



$$D[k, i, j] = \min \{ D[k - 1, i, j], D[k - 1, i, k] + D[k - 1, k, j] \}$$

最优子结构

问题结构分析

递推关系建立

自底向上计算

最优方案追踪

动态规划：自底向上计算（确定计算顺序）

- 初始化
 - $D[0, i, i] = 0$: 起终点重合，路径长度为0

i	j	1	2	3	4	5
1		0				
2			0			
3				0		
4					0	
5						0

问题结构分析

递推关系建立

自底向上计算

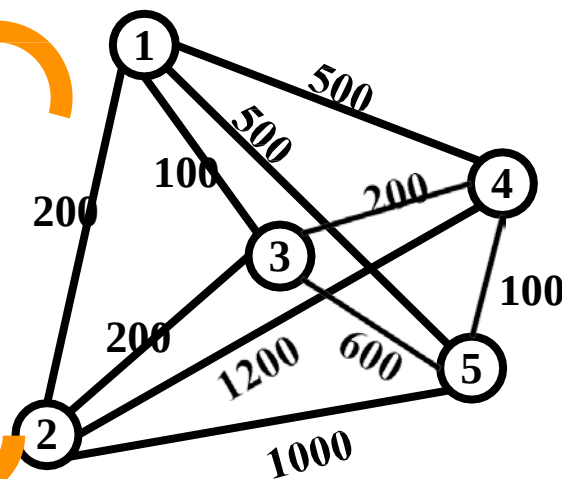
最优方案追踪

动态规划：自底向上计算（确定计算顺序）

- 初始化

- $D[0, i, i] = 0$: 起终点重合，路径长度为0
- $D[0, i, j] = e[i, j]$: 任意两点直达距离为边权

i	j	1	2	3	4	5
1		0	200	100	500	500
2		200	0	200	1200	1000
3		100	200	0	200	600
4		500	1200	200	0	100
5		500	1000	600	100	0



问题结构分析

递推关系建立

自底向上计算

最优方案追踪

动态规划：自底向上计算（确定计算顺序）

- 递推公式

- $$D[k, i, j] = \min\{D[k-1, i, j], D[k-1, i, k] + D[k-1, k, j]\}$$

最终的表格： $k = |V|$

$i \backslash j$	1	...	...	...	...
1					
...					
...			?		
...					
...					

初始化的表格： $k = 0$

$i \backslash j$	1	2	3	4	5
1	0	200	100	500	500
2	200	0	200	1200	1000
3	100	200	0	200	600
4	500	1200	200	0	100
5	500	1000	600	100	0

问题结构分析

递推关系建立

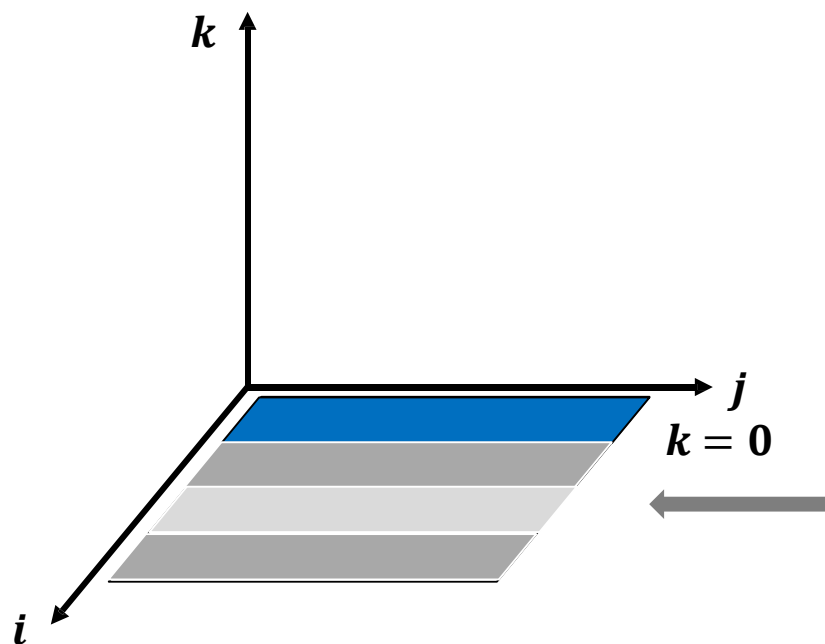
自底向上计算

最优方案追踪

动态规划：自底向上计算（确定计算顺序）

- 递推公式

- $$D[k, i, j] = \min\{D[k-1, i, j], D[k-1, i, k] + D[k-1, k, j]\}$$



最终的表格： $k = |V|$

$i \backslash j$	1	...	...	...	...
1					
...					
...			?		
...					
...					

初始化的表格： $k = 0$

$i \backslash j$	1	2	3	4	5
1	0	200	100	500	500
2	200	0	200	1200	1000
3	100	200	0	200	600
4	500	1200	200	0	100
5	500	1000	600	100	0

问题结构分析

递推关系建立

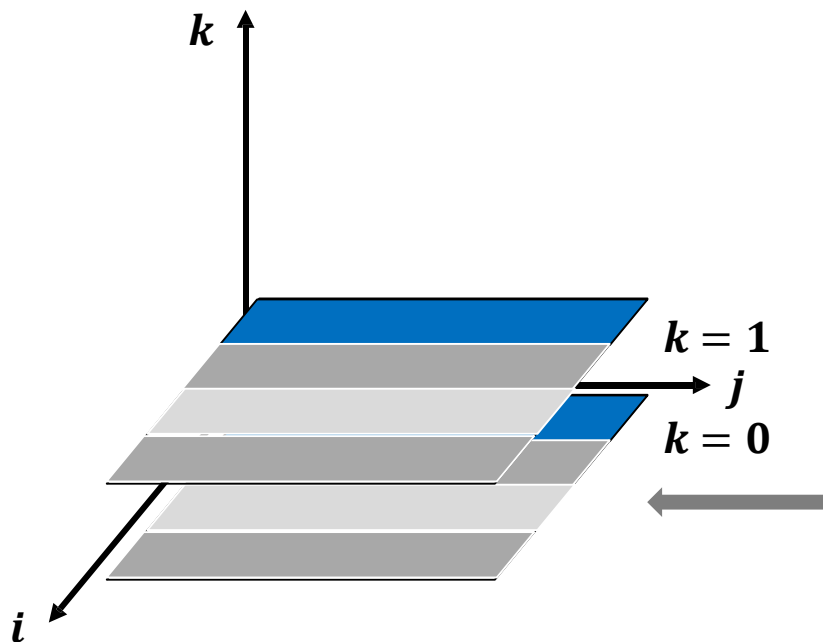
自底向上计算

最优方案追踪

动态规划：自底向上计算（确定计算顺序）

- 递推公式

- $$D[k, i, j] = \min\{D[k-1, i, j], D[k-1, i, k] + D[k-1, k, j]\}$$



最终的表格： $k = |V|$

$i \backslash j$	1	...	...	...	...
1					
...					
...			?		
...					
...					

初始化的表格： $k = 0$

$i \backslash j$	1	2	3	4	5
1	0	200	100	500	500
2	200	0	200	1200	1000
3	100	200	0	200	600
4	500	1200	200	0	100
5	500	1000	600	100	0

问题结构分析

递推关系建立

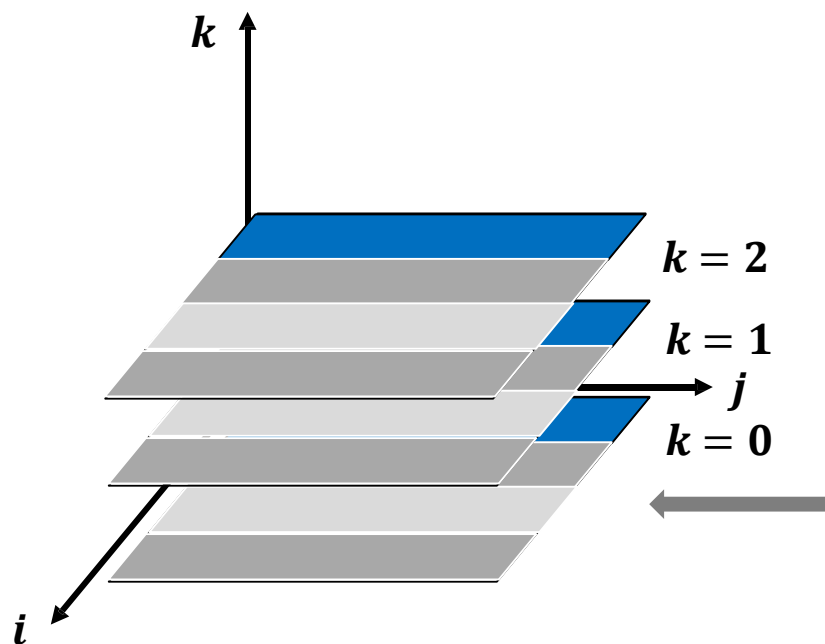
自底向上计算

最优方案追踪

动态规划：自底向上计算（确定计算顺序）

- 递推公式

- $$D[k, i, j] = \min\{D[k-1, i, j], D[k-1, i, k] + D[k-1, k, j]\}$$



最终的表格： $k = |V|$

$i \backslash j$	1	...	...	...	...
1					
...					
...					
...					
...					

初始化的表格： $k = 0$

$i \backslash j$	1	2	3	4	5
1	0	200	100	500	500
2	200	0	200	1200	1000
3	100	200	0	200	600
4	500	1200	200	0	100
5	500	1000	600	100	0

问题结构分析

递推关系建立

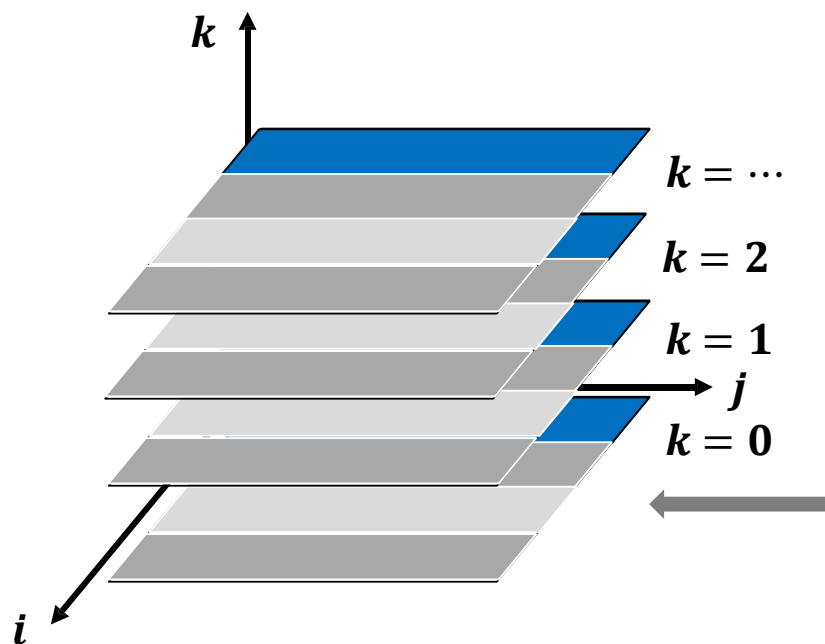
自底向上计算

最优方案追踪

动态规划：自底向上计算（确定计算顺序）

- 递推公式

- $$D[k, i, j] = \min\{D[k-1, i, j], D[k-1, i, k] + D[k-1, k, j]\}$$



最终的表格： $k = |V|$

$i \backslash j$	1	...	...	...	...
1					
...					
...			?		
...					
...					

初始化的表格： $k = 0$

$i \backslash j$	1	2	3	4	5
1	0	200	100	500	500
2	200	0	200	1200	1000
3	100	200	0	200	600
4	500	1200	200	0	100
5	500	1000	600	100	0

问题结构分析

递推关系建立

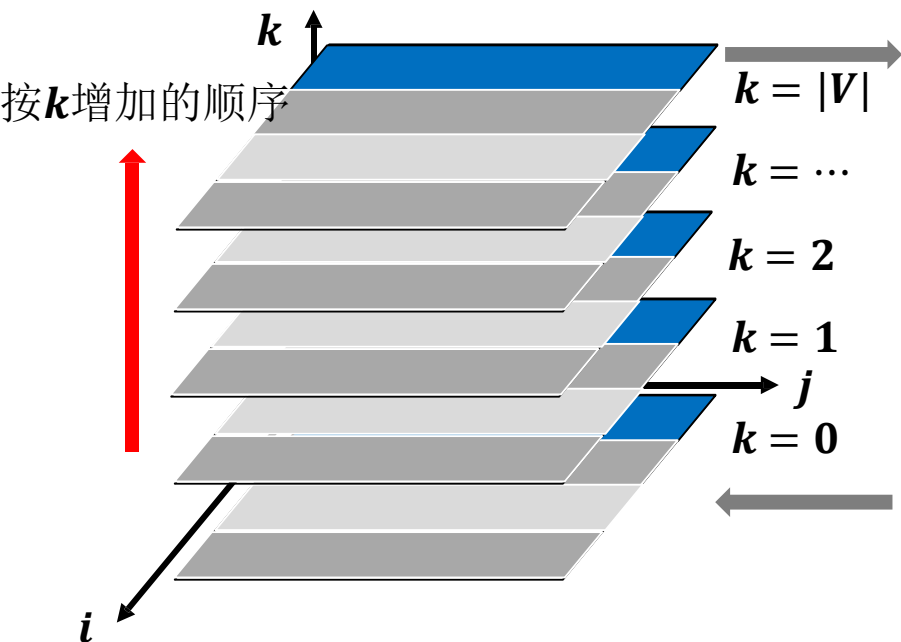
自底向上计算

最优方案追踪

动态规划：自底向上计算（确定计算顺序）

- 递推公式

- $$D[k, i, j] = \min\{D[k-1, i, j], D[k-1, i, k] + D[k-1, k, j]\}$$



最终的表格： $k = |V|$

$i \backslash j$	1	...	...	...	...
1					
...			?		
...					
...					

初始化的表格： $k = 0$

$i \backslash j$	1	2	3	4	5
1	0	200	100	500	500
2	200	0	200	1200	1000
3	100	200	0	200	600
4	500	1200	200	0	100
5	500	1000	600	100	0

问题结构分析

递推关系建立

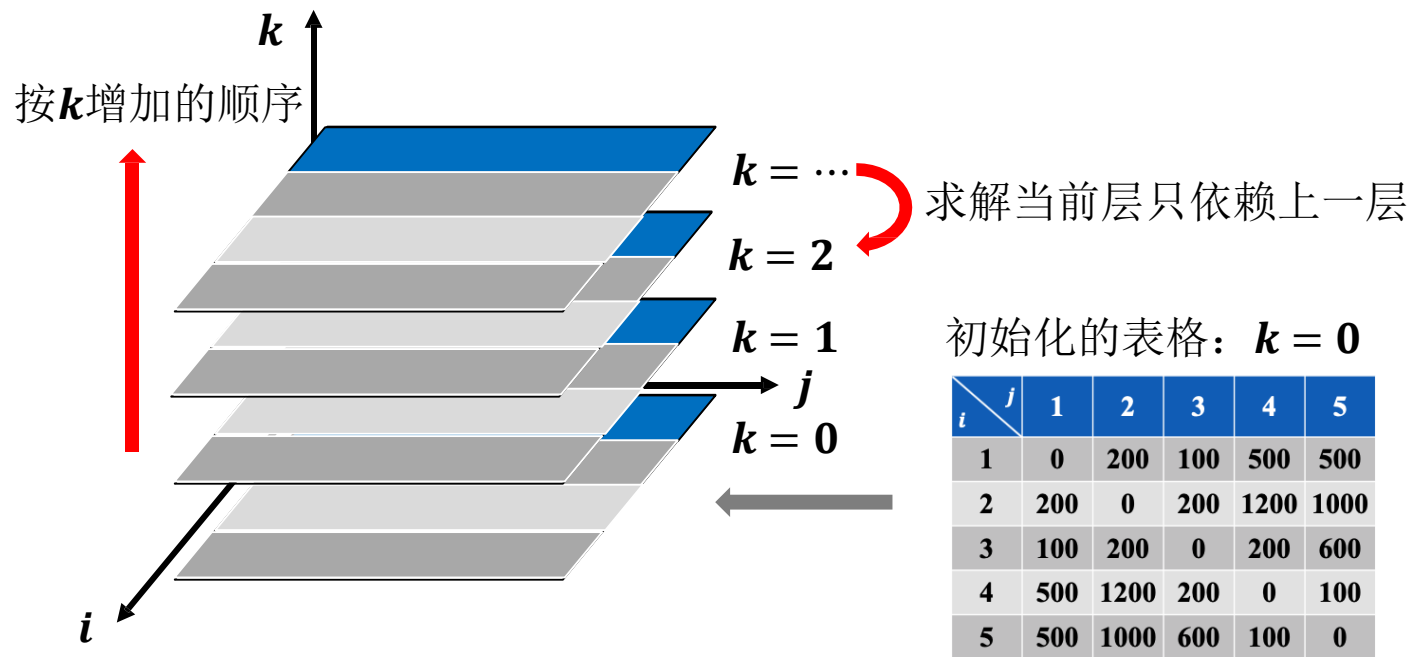
自底向上计算

最优方案追踪

动态规划：自底向上计算（确定计算顺序）

- 递推公式

- $$D[k, i, j] = \min\{D[k-1, i, j], D[k-1, i, k] + D[k-1, k, j]\}$$



问题结构分析

递推关系建立

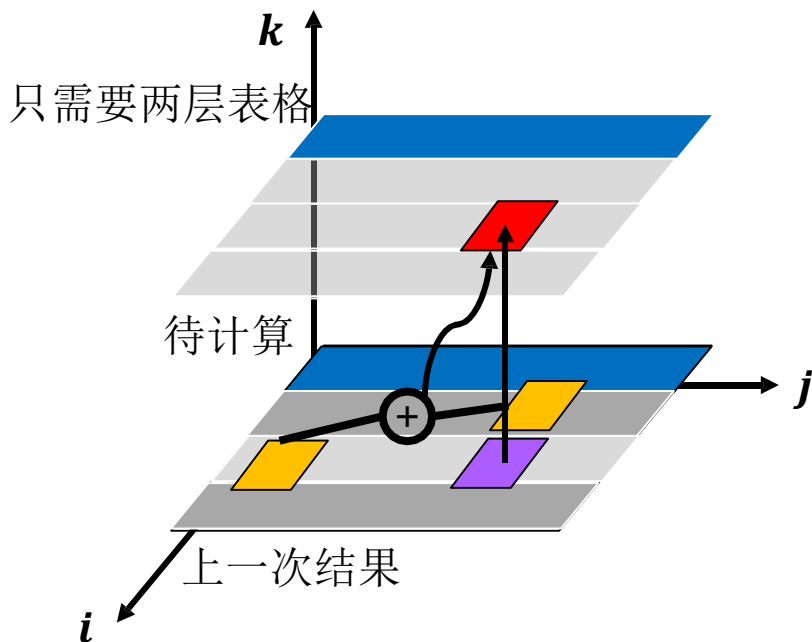
自底向上计算

最优方案追踪

动态规划：自底向上计算（确定计算顺序）

- 递推公式

- $$D[k, i, j] = \min\{D[k-1, i, j], D[k-1, i, k] + D[k-1, k, j]\}$$



问题结构分析

递推关系建立

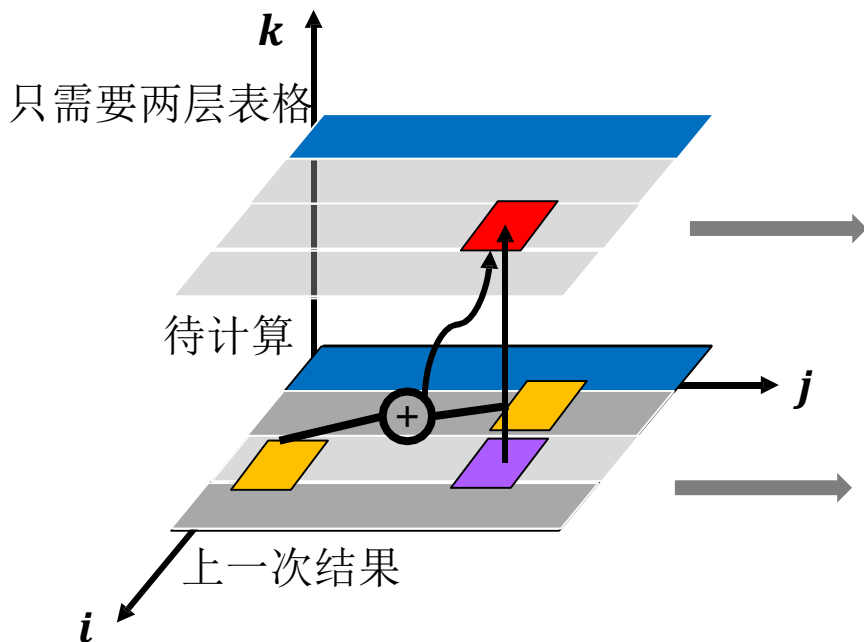
自底向上计算

最优方案追踪

动态规划：自底向上计算（确定计算顺序）

- 递推公式

- $$D[k, i, j] = \min\{D[k-1, i, j], D[k-1, i, k] + D[k-1, k, j]\}$$



i	j	1	...	k	...	...
		1	...	k	...	...
1						
...						
k						
...						
...						
i	j	1	...	k	...	...
1						
...						
k						
...						
...						

问题结构分析

递推关系建立

自底向上计算

最优方案追踪

动态规划：自底向上计算（确定计算顺序）

- 递推公式

- $$D[k, i, j] = \min\{D[k-1, i, j], \quad 0 + D[k-1, i, j] = D[k-1, i, j], \\ D[k-1, i, k] + D[k-1, k, j]\}$$

- 若 $k = i$ 或 $k = j$

$$D[k, i, j] = D[k-1, i, j]$$

值相同，可以直接覆盖

$i \backslash j$	1	...	k	...	...
1					
...					
k					
...					
...					
$i \backslash j$	1	...	k	...	...
1					
...					
k					
...					
...					

问题结构分析

递推关系建立

自底向上计算

最优方案追踪

动态规划：自底向上计算（确定计算顺序）

- 递推公式

- $$D[k, i, j] = \min\{D[k-1, i, j], \quad 0 + D[k-1, i, j] = D[k-1, i, j], \quad D[k-1, i, k] + D[k-1, k, j]\}$$

- 若 $k = i$ 或 $k = j$

$$D[k, i, j] = D[k-1, i, j]$$

值相同，可以直接覆盖

- 若 $k \neq i$ 且 $k \neq j$

$D[k-1, i, j]$ 和 $D[k-1, i, k], D[k-1, k, j]$

不是相同子问题

求出 $D[k, i, j]$ 后， $D[k-1, i, j]$ 不再被使用
可直接覆盖

$i \backslash j$	1	...	k	...	...
1					
...					
k					
...					
...					
$i \backslash j$	1	...	k	...	...
1					
...					
k					
...					
...					

问题结构分析

递推关系建立

自底向上计算

最优方案追踪

动态规划：自底向上计算（确定计算顺序）

- 递推公式

- $D[k, i, j] = \min\{D[k-1, i, j], \quad 0 + D[k-1, i, j] = D[k-1, i, j], \quad D[k-1, i, k] + D[k-1, k, j]\}$

- 若 $k = i$ 或 $k = j$

$$D[k, i, j] = D[k-1, i, j]$$

值相同，可以直接覆盖

- 若 $k \neq i$ 且 $k \neq j$

$D[k-1, i, j]$ 和 $D[k-1, i, k], D[k-1, k, j]$

不是相同子问题

求出 $D[k, i, j]$ 后， $D[k-1, i, j]$ 不再被使用
可直接覆盖

求出新值可直接在原位置覆盖
只需存储一层表格

$i \backslash j$	1	...	k	...	...
1					
...					
k					
...					
...					
$i \backslash j$	1	...	k	...	...
1					
...					
k					
...					
...					

问题结构分析

递推关系建立

自底向上计算

最优方案追踪

动态规划：自底向上计算（确定计算顺序）

- 递推公式

- $$D_k[i, j] = \min\{D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j]\}$$

求出新值可直接在原位置覆盖
只需存储一层表格

问题结构分析

递推关系建立

自底向上计算

最优方案追踪

动态规划：最优方案追踪

- 递推公式
 - $D_k[i, j] = \min\{D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j]\}$
- 追踪数组 Rec ，记录经过的中间点
 - $D_k[i, j] = D_{k-1}[i, j]$: 0 表示没有中间点

Rec

$i \quad j$	1	...	j	...	V
1					
...					
i			0		
...					
V					

问题结构分析

递推关系建立

自底向上计算

最优方案追踪

动态规划：最优方案追踪

- 递推公式

- $D_k[i, j] = \min\{D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j]\}$

- 追踪数组 Rec ，记录经过的中间点

- $D_k[i, j] = D_{k-1}[i, j]$: 0 表示没有中间点

- $D_k[i, j] = D_{k-1}[i, k] + D_{k-1}[k, j]$: k 表示经过中间点 k

松弛时使用的点

Rec

$i \backslash j$	1	...	j	...	V
1					
...					
i			k		
...					
V					

问题结构分析

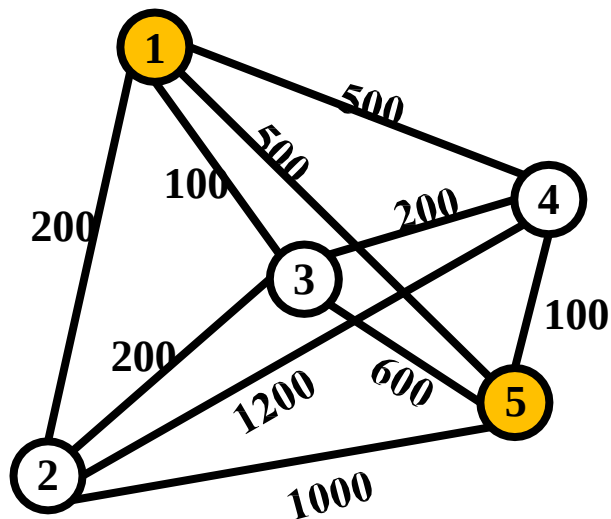
递推关系建立

自底向上计算

最优方案追踪

动态规划：最优方案追踪

- 根据数组 **Rec**，输出最短路径



Rec

<i>i</i> \ <i>j</i>	1	2	3	4	5
1			0		3
2					
3				0	4
4					0
5					

问题结构分析

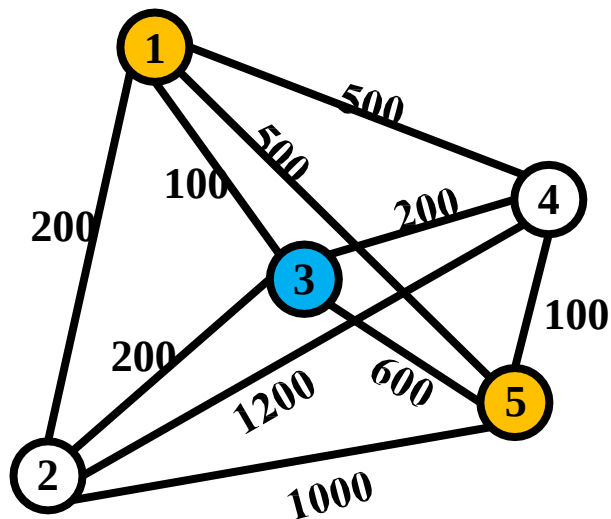
递推关系建立

自底向上计算

最优方案追踪

动态规划：最优方案追踪

- 根据数组 **Rec**，输出最短路径



Rec

<i>i</i> \ <i>j</i>	1	2	3	4	5
1			0		3
2					
3				0	4
4					0
5					

问题结构分析

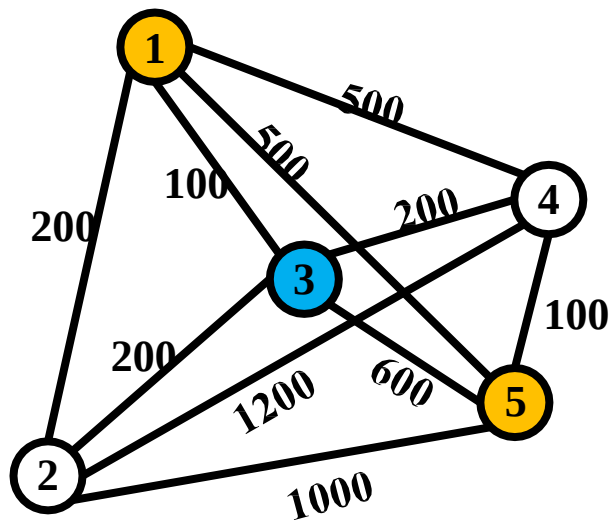
递推关系建立

自底向上计算

最优方案追踪

动态规划：最优方案追踪

- 根据数组 **Rec**，输出最短路径



Rec

<i>i</i> \ <i>j</i>	1	2	3	4	5
1			0		3
2					
3				0	4
4					0
5					

问题结构分析

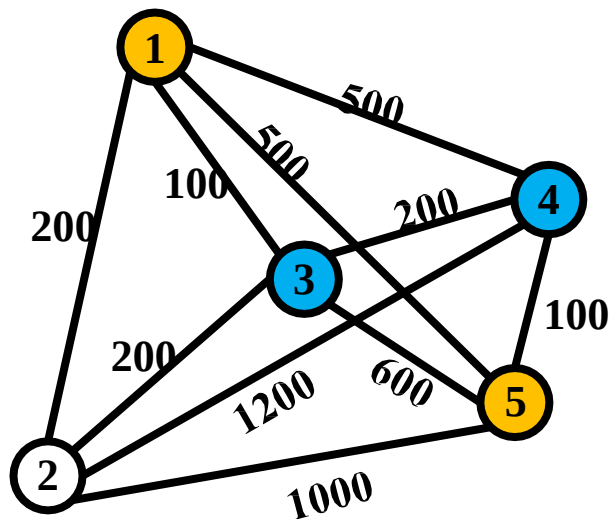
递推关系建立

自底向上计算

最优方案追踪

动态规划：最优方案追踪

- 根据数组 **Rec**，输出最短路径



Rec

<i>i</i> \ <i>j</i>	1	2	3	4	5
1			0		3
2					
3				0	
4					
5					

问题结构分析

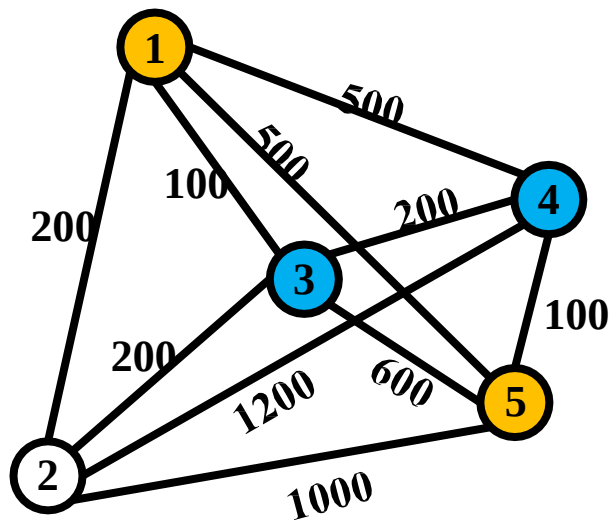
递推关系建立

自底向上计算

最优方案追踪

动态规划：最优方案追踪

- 根据数组 **Rec**，输出最短路径



Rec

$i \backslash j$	1	2	3	4	5
1			0		3
2					
3				0	4
4					0
5					

问题结构分析

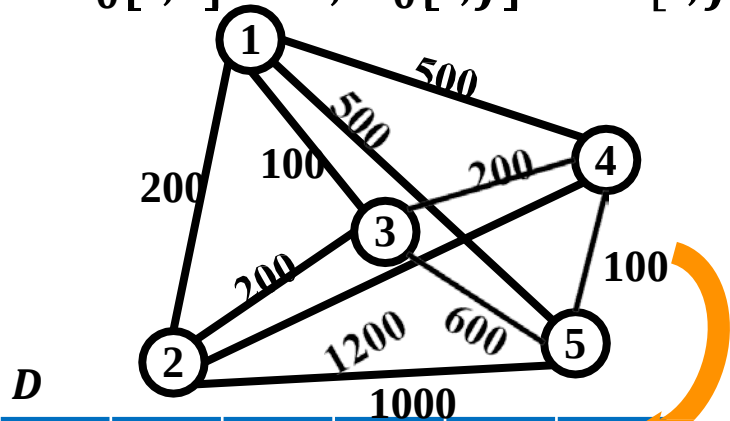
递推关系建立

自底向上计算

最优方案追踪

算法实例

$D_0[i,i] = 0, D_0[i,j] = W[i,j]$



D

<i>i</i> \ <i>j</i>	1	2	3	4	5
1	0	200	100	500	500
2	200	0	200	1200	1000
3	100	200	0	200	600
4	500	1200	200	0	100
5	500	1000	600	100	0

$k = 0$

所有点对都没有经过其他点

Rec

<i>i</i> \ <i>j</i>	1	2	3	4	5
1	0	0	0	0	0
2	0	0	0	0	0
3	0	0	0	0	0
4	0	0	0	0	0
5	0	0	0	0	0

算法实例

- $D_k[i, j] = \min\{D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j]\}$

D

<i>i</i> \ <i>j</i>	1	2	3	4	5
1	0	200	100	500	500
2	200	0	200	1200	1000
3	100	200	0	200	600
4	500	1200	200	0	100
5	500	1000	600	100	0

Rec

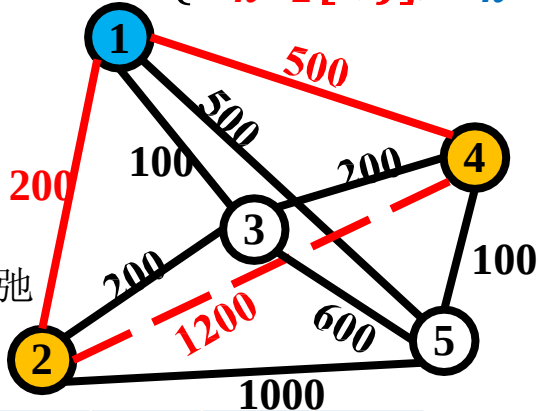
<i>i</i> \ <i>j</i>	1	2	3	4	5
1	0	0	0	0	0
2	0	0	0	0	0
3	0	0	0	0	0
4	0	0	0	0	0
5	0	0	0	0	0

$k = 1$

算法实例

$$D_k[i,j] = \min\{D_{k-1}[i,j], D_{k-1}[i,k] + D_{k-1}[k,j]\}$$

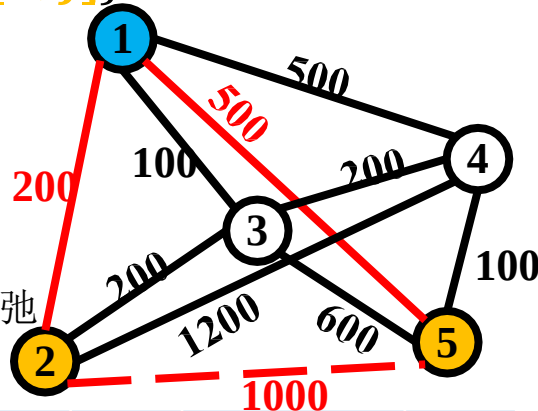
从2到4路径得到松弛



D

<i>i</i> \ <i>j</i>	1	2	3	4	5
1	0	200	100	500	500
2	200	0	200	700	700
3	100	200	0	200	600
4	500	1200	200	0	100
5	500	1000	600	100	0

从2到5路径得到松弛



Rec

<i>i</i> \ <i>j</i>	1	2	3	4	5
1	0	0	0	0	0
2	0	0	0	1	1
3	0	0	0	0	0
4	0	0	0	0	0
5	0	0	0	0	0

k = 1

算法实例

- $$D_k[i, j] = \min\{D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j]\}$$

$i \backslash j$	1	2	3	4	5
1	0	200	100	500	500
2	200	0	200	700	700
3	100	200	0	200	600
4	500	1200	200	0	100
5	500	1000	600	100	0

$k = 1$

$i \backslash j$	1	2	3	4	5
1	0	0	0	0	0
2	0	0	0	1	1
3	0	0	0	0	0
4	0	0	0	0	0
5	0	0	0	0	0

算法实例

- $D_k[i, j] = \min\{D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j]\}$

$i \backslash j$	1	2	3	4	5
1	0	200	100	500	500
2	200	0	200	700	700
3	100	200	0	200	600
4	500	700	200	0	100
5	500	1000	600	100	0

$k = 1$

$i \backslash j$	1	2	3	4	5
1	0	0	0	0	0
2	0	0	0	1	1
3	0	0	0	0	0
4	0	1	0	0	0
5	0	0	0	0	0

算法实例

- $D_k[i, j] = \min\{D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j]\}$

$i \backslash j$	1	2	3	4	5
1	0	200	100	500	500
2	200	0	200	700	700
3	100	200	0	200	600
4	500	700	200	0	100
5	500	700	600	100	0

$k = 1$

$i \backslash j$	1	2	3	4	5
1	0	0	0	0	0
2	0	0	0	1	1
3	0	0	0	0	0
4	0	1	0	0	0
5	0	0	0	0	0

算法实例

- $D_k[i, j] = \min\{D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j]\}$

$i \backslash j$	1	2	3	4	5
1	0	200	100	500	500
2	200	0	200	700	700
3	100	200	0	200	600
4	500	700	200	0	100
5	500	700	600	100	0

$k = 2$

$i \backslash j$	1	2	3	4	5
1	0	0	0	0	0
2	0	0	0	1	1
3	0	0	0	0	0
4	0	1	0	0	0
5	0	0	0	0	0

算法实例

- $D_k[i, j] = \min\{D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j]\}$

$i \backslash j$	1	2	3	4	5
1	0	200	100	500	500
2	200	0	200	700	700
3	100	200	0	200	600
4	500	700	200	0	100
5	500	700	600	100	0

$k = 2$

$i \backslash j$	1	2	3	4	5
1	0	0	0	0	0
2	0	0	0	1	1
3	0	0	0	0	0
4	0	1	0	0	0
5	0	0	0	0	0

算法实例

- $D_k[i, j] = \min\{D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j]\}$

$i \backslash j$	1	2	3	4	5
1	0	200	100	500	500
2	200	0	200	700	700
3	100	200	0	200	600
4	500	700	200	0	100
5	500	700	600	100	0

$k = 2$

$i \backslash j$	1	2	3	4	5
1	0	0	0	0	0
2	0	0	0	1	1
3	0	0	0	0	0
4	0	1	0	0	0
5	0	0	0	0	0

算法实例

- $D_k[i, j] = \min\{D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j]\}$

$i \backslash j$	1	2	3	4	5
1	0	200	100	500	500
2	200	0	200	700	700
3	100	200	0	200	600
4	500	700	200	0	100
5	500	700	600	100	0

$k = 2$

$i \backslash j$	1	2	3	4	5
1	0	0	0	0	0
2	0	0	0	1	1
3	0	0	0	0	0
4	0	1	0	0	0
5	0	0	0	0	0

算法实例

- $D_k[i, j] = \min\{D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j]\}$

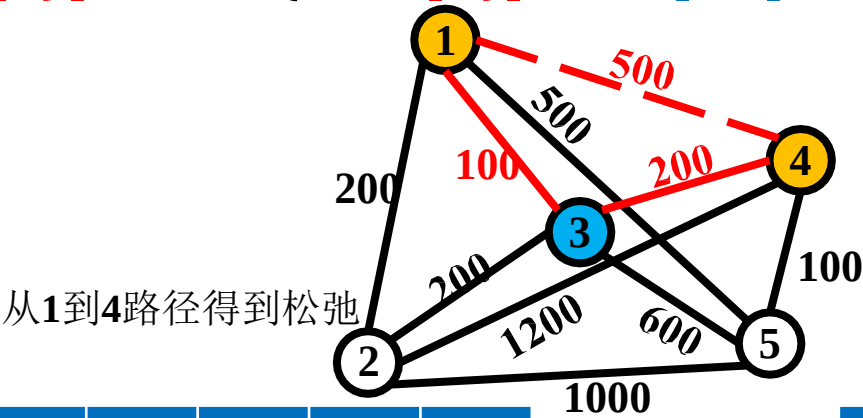
$i \backslash j$	1	2	3	4	5
1	0	200	100	500	500
2	200	0	200	700	700
3	100	200	0	200	600
4	500	700	200	0	100
5	500	700	600	100	0

$k = 2$

$i \backslash j$	1	2	3	4	5
1	0	0	0	0	0
2	0	0	0	1	1
3	0	0	0	0	0
4	0	1	0	0	0
5	0	0	0	0	0

算法实例

$D_k[i, j] = \min\{D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j]\}$



$i \backslash j$	1	2	3	4	5
1	0	200	100	300	500
2	200	0	200	700	700
3	100	200	0	200	600
4	500	700	200	0	100
5	500	700	600	100	0

$k = 3$

$i \backslash j$	1	2	3	4	5
1	0	0	0	3	0
2	0	0	0	1	1
3	0	0	0	0	0
4	0	1	0	0	0
5	0	0	0	0	0

算法实例

- $$D_k[i, j] = \min\{D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j]\}$$

$i \backslash j$	1	2	3	4	5
1	0	200	100	300	500
2	200	0	200	400	700
3	100	200	0	200	600
4	500	700	200	0	100
5	500	700	600	100	0

$k = 3$

$i \backslash j$	1	2	3	4	5
1	0	0	0	3	0
2	0	0	0	3	1
3	0	0	0	0	0
4	0	1	0	0	0
5	0	0	0	0	0

算法实例

- $D_k[i, j] = \min\{D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j]\}$

$i \backslash j$	1	2	3	4	5
1	0	200	100	300	500
2	200	0	200	400	700
3	100	200	0	200	600
4	500	700	200	0	100
5	500	700	600	100	0

$k = 3$

$i \backslash j$	1	2	3	4	5
1	0	0	0	3	0
2	0	0	0	3	1
3	0	0	0	0	0
4	0	1	0	0	0
5	0	0	0	0	0

算法实例

$$\bullet \quad D_k[i, j] = \min\{D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j]\}$$

$i \backslash j$	1	2	3	4	5
1	0	200	100	300	500
2	200	0	200	400	700
3	100	200	0	200	600
4	300	400	200	0	100
5	500	700	600	100	0

$k = 3$

$i \backslash j$	1	2	3	4	5
1	0	0	0	3	0
2	0	0	0	3	1
3	0	0	0	0	0
4	3	3	0	0	0
5	0	0	0	0	0

算法实例

- $$D_k[i, j] = \min\{D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j]\}$$

$i \backslash j$	1	2	3	4	5
1	0	200	100	300	500
2	200	0	200	400	700
3	100	200	0	200	600
4	300	400	200	0	100
5	500	700	600	100	0

$k = 3$

$i \backslash j$	1	2	3	4	5
1	0	0	0	3	0
2	0	0	0	3	1
3	0	0	0	0	0
4	3	3	0	0	0
5	0	0	0	0	0

算法实例

- $D_k[i, j] = \min\{D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j]\}$

$i \backslash j$	1	2	3	4	5
1	0	200	100	300	400
2	200	0	200	400	700
3	100	200	0	200	600
4	300	400	200	0	100
5	500	700	600	100	0

$k = 4$

$i \backslash j$	1	2	3	4	5
1	0	0	0	3	4
2	0	0	0	3	1
3	0	0	0	0	0
4	3	3	0	0	0
5	0	0	0	0	0

算法实例

- $D_k[i, j] = \min\{D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j]\}$

$i \backslash j$	1	2	3	4	5
1	0	200	100	300	400
2	200	0	200	400	500
3	100	200	0	200	600
4	300	400	200	0	100
5	500	700	600	100	0

$k = 4$

$i \backslash j$	1	2	3	4	5
1	0	0	0	3	4
2	0	0	0	3	4
3	0	0	0	0	0
4	3	3	0	0	0
5	0	0	0	0	0

算法实例

- $D_k[i, j] = \min\{D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j]\}$

$i \backslash j$	1	2	3	4	5
1	0	200	100	300	400
2	200	0	200	400	500
3	100	200	0	200	300
4	300	400	200	0	100
5	500	700	600	100	0

$k = 4$

$i \backslash j$	1	2	3	4	5
1	0	0	0	3	4
2	0	0	0	3	4
3	0	0	0	0	4
4	3	3	0	0	0
5	0	0	0	0	0

算法实例

- $D_k[i, j] = \min\{D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j]\}$

$i \backslash j$	1	2	3	4	5
1	0	200	100	300	400
2	200	0	200	400	500
3	100	200	0	200	300
4	300	400	200	0	100
5	500	700	600	100	0

$k = 4$

$i \backslash j$	1	2	3	4	5
1	0	0	0	3	4
2	0	0	0	3	4
3	0	0	0	0	4
4	3	3	0	0	0
5	0	0	0	0	0

算法实例

- $$D_k[i, j] = \min\{D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j]\}$$

$i \backslash j$	1	2	3	4	5
1	0	200	100	300	400
2	200	0	200	400	500
3	100	200	0	200	300
4	300	400	200	0	100
5	400	500	300	100	0

$k = 4$

$i \backslash j$	1	2	3	4	5
1	0	0	0	3	4
2	0	0	0	3	4
3	0	0	0	0	4
4	3	3	0	0	0
5	4	4	4	0	0

算法实例

- $D_k[i, j] = \min\{D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j]\}$

$i \backslash j$	1	2	3	4	5
1	0	200	100	300	400
2	200	0	200	400	500
3	100	200	0	200	300
4	300	400	200	0	100
5	400	500	300	100	0

$k = 5$

$i \backslash j$	1	2	3	4	5
1	0	0	0	3	4
2	0	0	0	3	4
3	0	0	0	0	4
4	3	3	0	0	0
5	4	4	4	0	0

算法实例

- $D_k[i, j] = \min\{D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j]\}$

$i \backslash j$	1	2	3	4	5
1	0	200	100	300	400
2	200	0	200	400	500
3	100	200	0	200	300
4	300	400	200	0	100
5	400	500	300	100	0

$k = 5$

$i \backslash j$	1	2	3	4	5
1	0	0	0	3	4
2	0	0	0	3	4
3	0	0	0	0	4
4	3	3	0	0	0
5	4	4	4	0	0

算法实例

- $D_k[i, j] = \min\{D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j]\}$

$i \backslash j$	1	2	3	4	5
1	0	200	100	300	400
2	200	0	200	400	500
3	100	200	0	200	300
4	300	400	200	0	100
5	400	500	300	100	0

$k = 5$

$i \backslash j$	1	2	3	4	5
1	0	0	0	3	4
2	0	0	0	3	4
3	0	0	0	0	4
4	3	3	0	0	0
5	4	4	4	0	0

算法实例

- $D_k[i, j] = \min\{D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j]\}$

$i \backslash j$	1	2	3	4	5
1	0	200	100	300	400
2	200	0	200	400	500
3	100	200	0	200	300
4	300	400	200	0	100
5	400	500	300	100	0

$k = 5$

$i \backslash j$	1	2	3	4	5
1	0	0	0	3	4
2	0	0	0	3	4
3	0	0	0	0	4
4	3	3	0	0	0
5	4	4	4	0	0

算法实例

- $$D_k[i, j] = \min\{D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j]\}$$

$i \backslash j$	1	2	3	4	5
1	0	200	100	300	400
2	200	0	200	400	500
3	100	200	0	200	300
4	300	400	200	0	100
5	400	500	300	100	0

$k = 5$

$i \backslash j$	1	2	3	4	5
1	0	0	0	3	4
2	0	0	0	3	4
3	0	0	0	0	4
4	3	3	0	0	0
5	4	4	4	0	0

算法实例

- $D_k[i, j] = \min\{D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j]\}$
- 查询从1到5的最短路

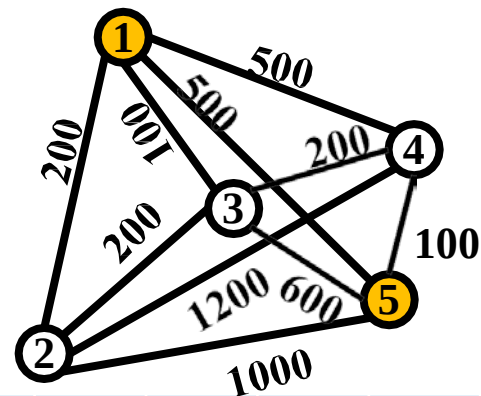
D

$i \backslash j$	1	2	3	4	5
1	0	200	100	300	400
2	200	0	200	400	500
3	100	200	0	200	300
4	300	400	200	0	100
5	400	500	300	100	0

Rec

$i \backslash j$	1	2	3	4	5
1	0	0	0	3	4
2	0	0	0	3	4
3	0	0	0	0	4
4	3	3	0	0	0
5	4	4	4	0	0

$k = 5$



算法实例

- $D_k[i, j] = \min\{D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j]\}$
- 查询从1到5的最短路

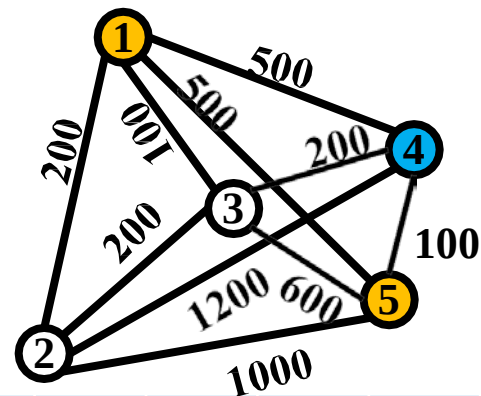
D

$i \backslash j$	1	2	3	4	5
1	0	200	100	300	400
2	200	0	200	400	500
3	100	200	0	200	300
4	300	400	200	0	100
5	400	500	300	100	0

Rec

$i \backslash j$	1	2	3	4	5
1	0	0	0	3	4
2	0	0	0	3	4
3	0	0	0	0	4
4	3	3	0	0	0
5	4	4	4	0	0

$k = 5$



算法实例

- $D_k[i, j] = \min\{D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j]\}$
- 查询从1到5的最短路

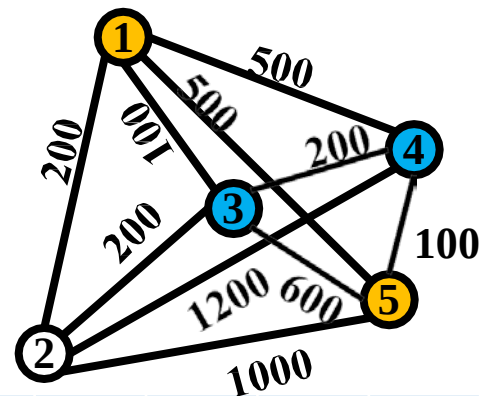
D

$i \backslash j$	1	2	3	4	5
1	0	200	100	300	400
2	200	0	200	400	500
3	100	200	0	200	300
4	300	400	200	0	100
5	400	500	300	100	0

Rec

$i \backslash j$	1	2	3	4	5
1	0	0	0	3	4
2	0	0	0	3	4
3	0	0	0	0	4
4	3	3	0	0	0
5	4	4	4	0	0

$k = 5$



算法实例

- $D_k[i, j] = \min\{D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j]\}$
- 查询从1到5的最短路

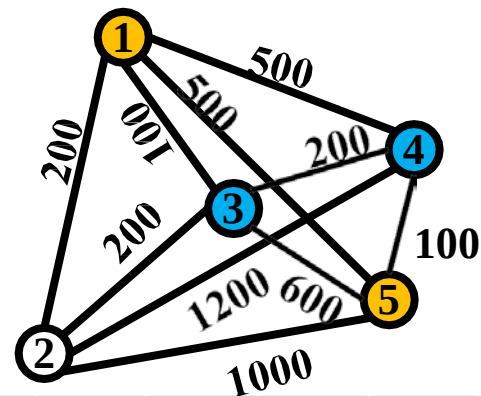
D

$i \backslash j$	1	2	3	4	5
1	0	200	100	300	400
2	200	0	200	400	500
3	100	200	0	200	300
4	300	400	200	0	100
5	400	500	300	100	0

Rec

$i \backslash j$	1	2	3	4	5
1	0	0	0	3	4
2	0	0	0	3	4
3	0	0	0	0	4
4	3	3	0	0	0
5	4	4	4	0	0

$k = 5$



算法实例

- $D_k[i, j] = \min\{D_{k-1}[i, j], D_{k-1}[i, k] + D_{k-1}[k, j]\}$
- 查询从1到5的最短路

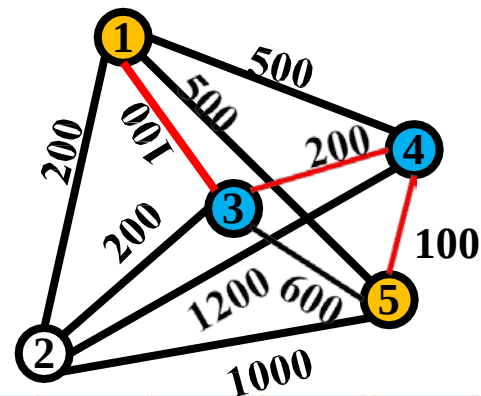
D

$i \backslash j$	1	2	3	4	5
1	0	200	100	300	400
2	200	0	200	400	500
3	100	200	0	200	300
4	300	400	200	0	100
5	400	500	300	100	0

Rec

$i \backslash j$	1	2	3	4	5
1	0	0	0	3	4
2	0	0	0	3	4
3	0	0	0	0	4
4	3	3	0	0	0
5	4	4	4	0	0

$k = 5$



Floyd-Warshall算法伪代码

- All-Pairs-Shortest-Paths(G)

输入: 图 $G = \langle V, E, W \rangle$

输出: 任意两点最短路径

新建二维数组 $D[1..|V|, 1..|V|]$, $Rec[1..|V|, 1..|V|]$

```
for  $i \leftarrow 1$  to  $|V|$  do
  for  $j \leftarrow 1$  to  $|V|$  do
     $Rec[i, j] \leftarrow 0$ 
    if  $i = j$  then
       $D[i, j] \leftarrow 0$ 
    end
    else
       $D[i, j] \leftarrow W[i, j]$ 
    end
  end
end
end
```

初始化

伪代码

- **All-Pairs-Shortest-Paths(G)**

输入: 图 $G = \langle V, E, W \rangle$

输出: 任意两点最短路径

新建二维数组 $D[1..|V|, 1..|V|]$, $Rec[1..|V|, 1..|V|]$

for $i \leftarrow 1$ to $|V|$ do

 for $j \leftarrow 1$ to $|V|$ do

$Rec[i, j] \leftarrow 0$

 if $i = j$ then

$D[i, j] \leftarrow 0$

 end

 else

$D[i, j] \leftarrow W[i, j]$

 end

 end

end

起终点相同，距离为0

起终点不同，距离为边权

伪代码

- **All-Pairs-Shortest-Paths(G)**

```
for  $k \leftarrow 1$  to  $|V|$  do
    for  $i \leftarrow 1$  to  $|V|$  do
        for  $j \leftarrow 1$  to  $|V|$  do
            if  $D[i, j] > D[i, k] + D[k, j]$  then
                 $D[i, j] \leftarrow D[i, k] + D[k, j]$ 
                 $Rec[i, j] \leftarrow k$ 
            end
        end
    end
end
return  $D, Rec$ 
```

按照 k 增大的顺序

伪代码

- **All-Pairs-Shortest-Paths(G)**

```
for  $k \leftarrow 1$  to  $|V|$  do
  for  $i \leftarrow 1$  to  $|V|$  do
    for  $j \leftarrow 1$  to  $|V|$  do
      if  $D[i, j] > D[i, k] + D[k, j]$  then
         $D[i, j] \leftarrow D[i, k] + D[k, j]$ 
         $Rec[i, j] \leftarrow k$ 
      end
    end
  end
end
return  $D, Rec$ 
```

松弛操作

伪代码

- **Find-Path(Rec, u, v)**

输入: 备忘数组 Rec , 起点 u , 终点 v

输出: 最短路径(逆序)

```
if  $Rec[u, v] = 0$  then  
    print  $v$   
    return  
end
```

$k \leftarrow Rec[u, v]$

Find-Path(Rec, u, k)

Find-Path(Rec, k, v)

没有中间点，直接输出

伪代码

- **Find-Path(Rec, u, v)**

输入: 备忘数组 Rec , 起点 u , 终点 v

输出: 最短路径(逆序)

if $Rec[u, v] = 0$ then

 | print v

 | return

end

$k \leftarrow Rec[u, v]$

Find-Path(Rec, u, k)

Find-Path(Rec, k, v)

有中间点，递归查找

时间复杂度

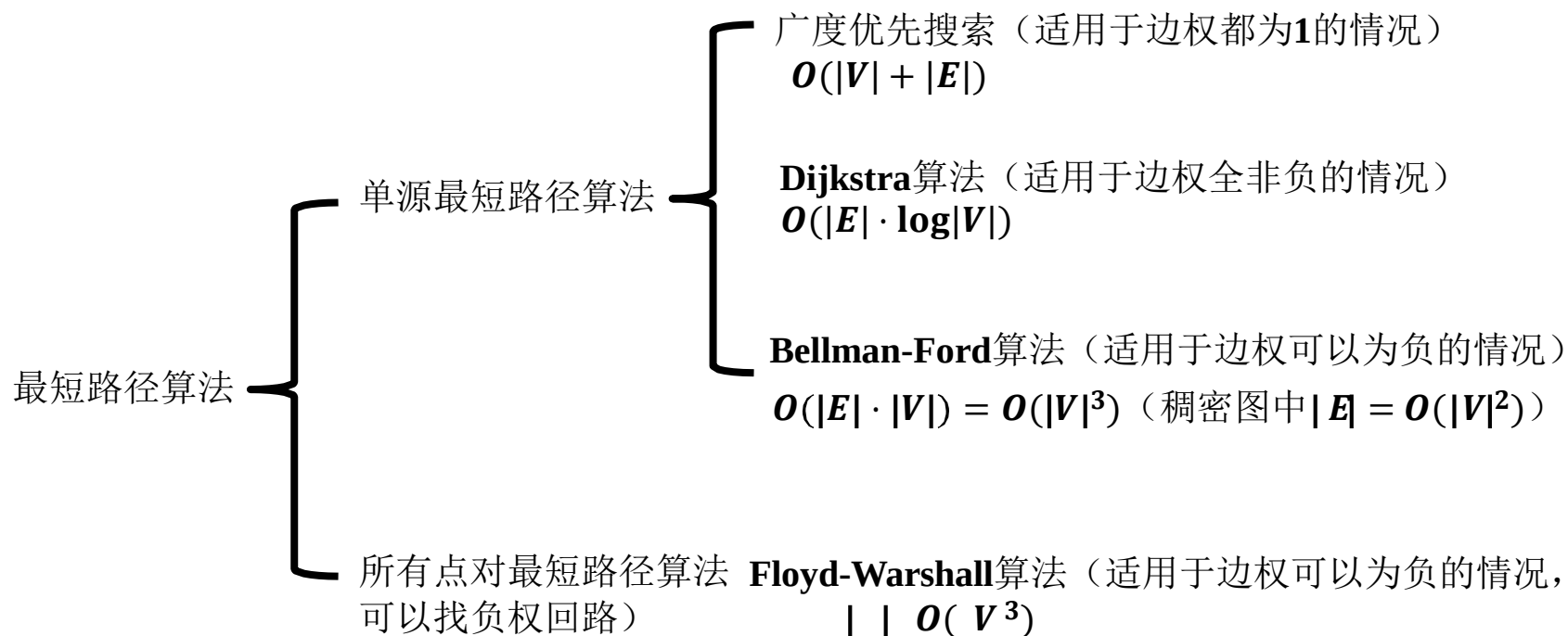
- **All-Pairs-Shortest-Paths(G)**

```
for  $k \leftarrow 1$  to  $|V|$  do
  for  $i \leftarrow 1$  to  $|V|$  do
    for  $j \leftarrow 1$  to  $|V|$  do
      if  $D[i, j] > D[i, k] + D[k, j]$  then
         $D[i, j] \leftarrow D[i, k] + D[k, j]$ 
         $Rec[i, j] \leftarrow k$ 
      end
    end
  end
end
return  $D, Rec$ 
```

时间复杂度: $O(|V|^3)$

$O(|V|)$ $O(|V|^2)$ $O(|V|^3)$

最短路径算法小结



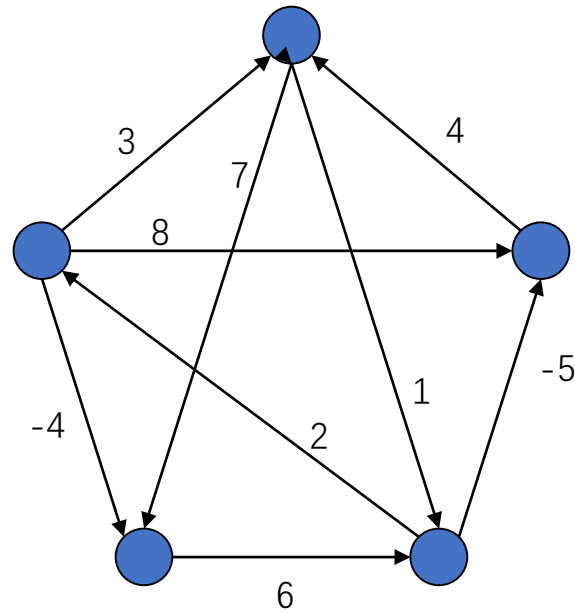
Johnson's algorithm

- Johnson's算法可用于计算All pairs shortest path问题。
- 在边的数量不多的时候，如 $|E|=O(|V|\log|V|)$ 时，能有比Warshall-Floyd演算法较佳的效能。
- 其输入需求是利用Adjacency list表示的图。

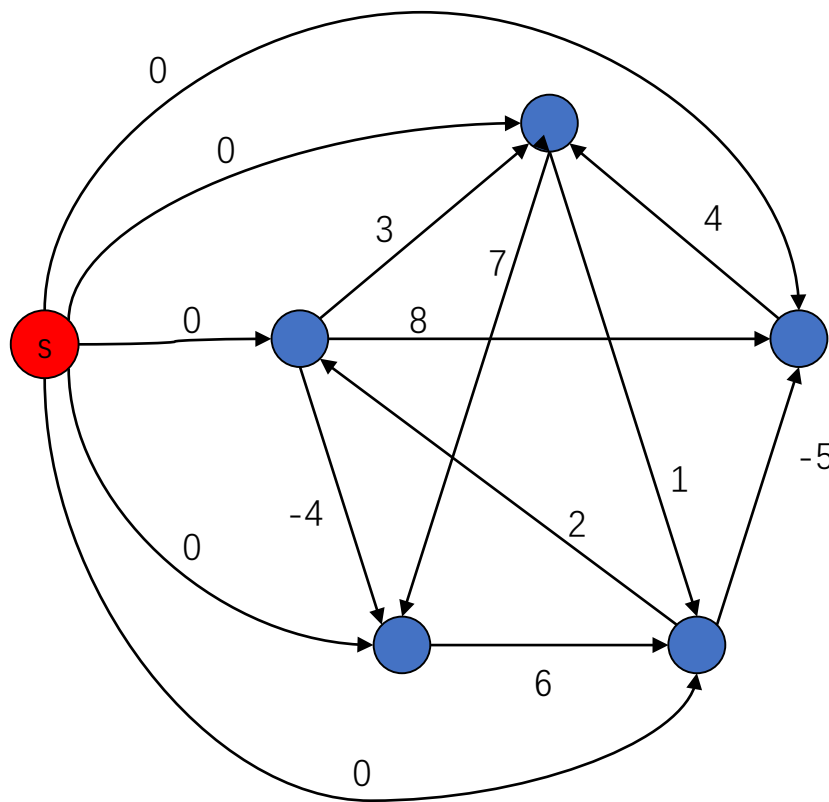
Johnson's algorithm

- Johnson's 算法利用reweighing来除去负边，使得该图可以套用Dijkstra算法，来达到较高的效能。
- Reweighing是将每个点 v 设定一个高度 $h(v)$ ，并且调整边的weight function $w(u,v)$ 成为 $w'(u,v)=w(u,v)+h(u)-h(v)$ 。
- 令 $\delta'(u,v)$ 如此调整之后的最短距离，则原先的最短距离 $\delta(u,v)=\delta'(u,v)-h(u)+h(v)$ 。

Johnson's algorithm 范例

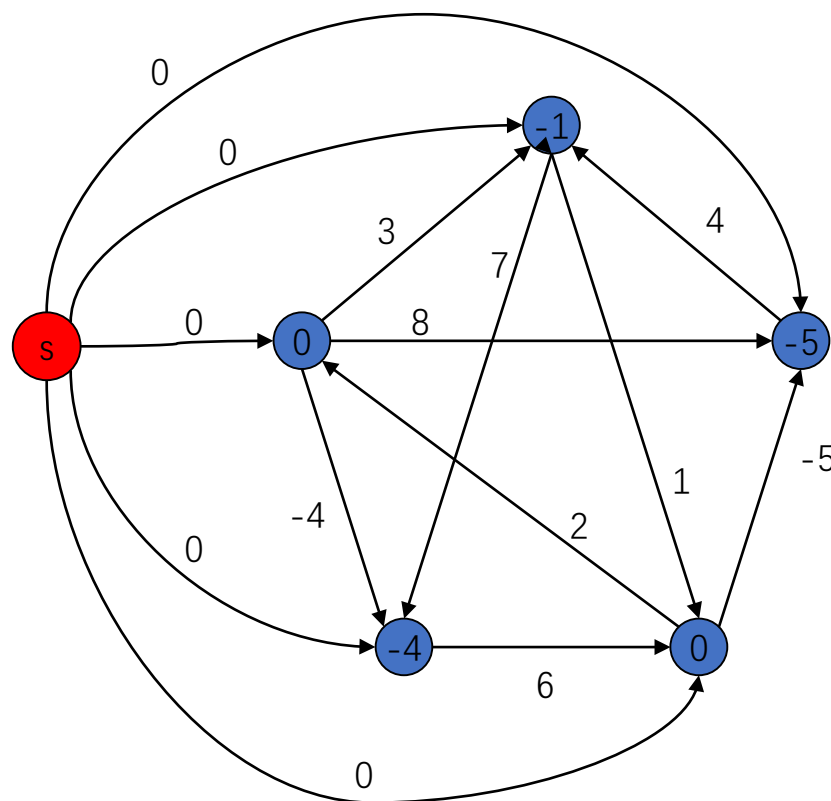


Johnson's algorithm 范例



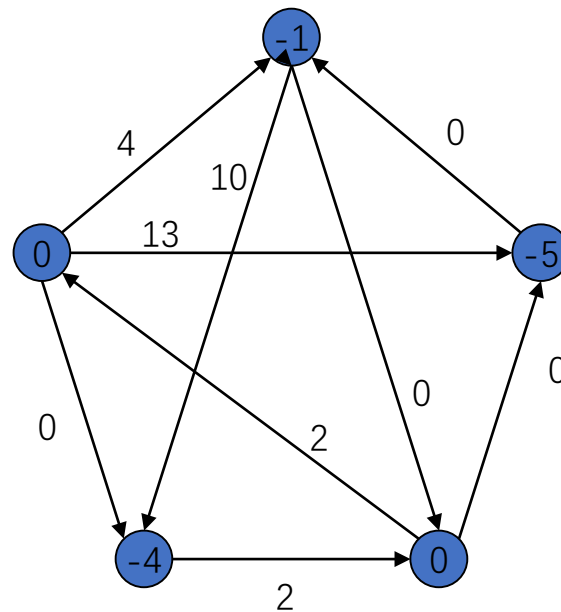
加入一个点s，
以及自s拉一条
weight为0的边
到每一点。

Johnson's algorithm 范例



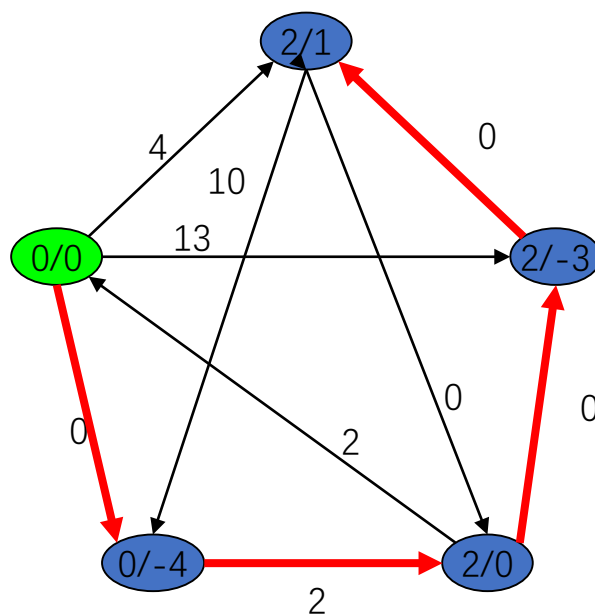
执行Bellman-Ford算法，得到自s出发每一点的最短距离。

Johnson's algorithm 范例



做完reweighting

Johnson's algorithm 范例

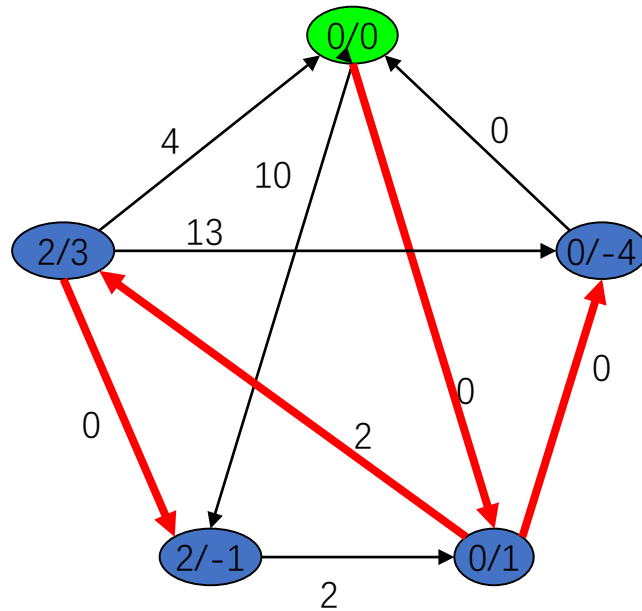


红线部分是Shortest-paths tree。

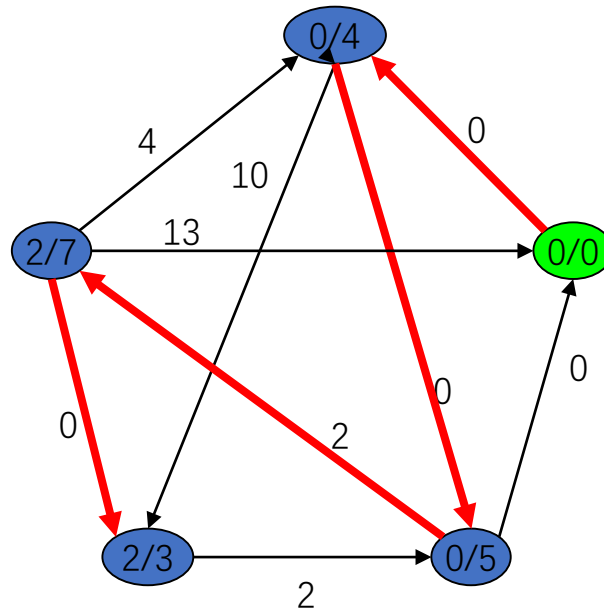
点的数字a/b代表自出发点(绿色点)出发，到达该点的最短路径

(Reweighting后的图/原图)。

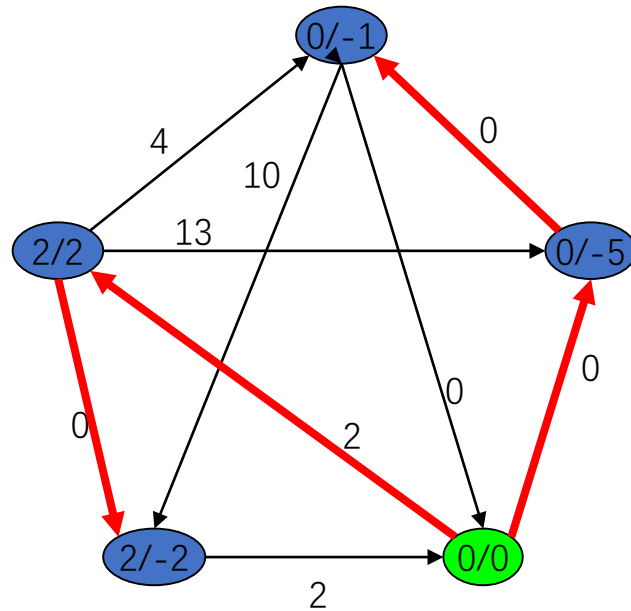
Johnson's algorithm 范例



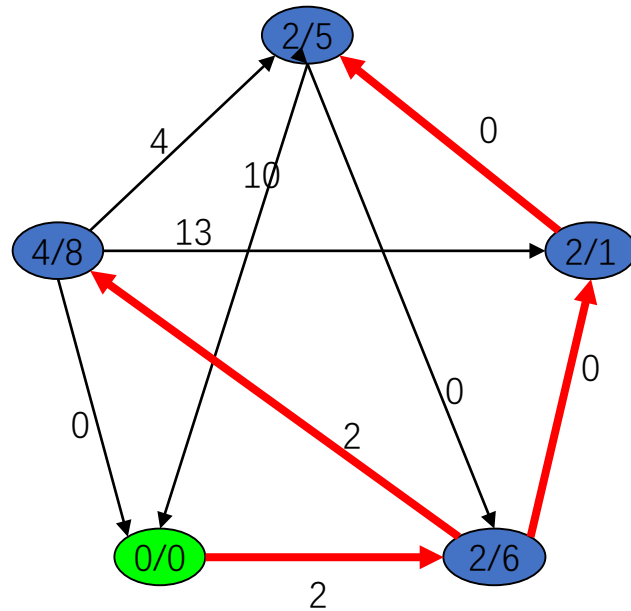
Johnson's algorithm 范例



Johnson's algorithm 范例



Johnson's algorithm 范例



Johnson's algorithm

```
Johnson(G)
{ compute  $G'$ , where  $V[G'] = V[G] \cup \{s\}$  and
     $E[G'] = E[G] \cup \{(s, v) : v \in V[G]\}$ 
  if Bellman-Ford( $G', w, s$ ) = False
    then print " $\exists$  a neg-weight cycle"
  else for each vertex  $v \in V[G']$ 
    set  $h(v) = \delta(s, v)$  computed by Bellman-Ford algo.
    for each edge  $(u, v) \in E[G']$ 
       $w'(u, v) = w(u, v) + h(u) - h(v)$ 
    for each vertex  $u \in V[G]$ 
      run Dijkstra( $G, w', u$ ) to compute  $\delta'(u, v)$ 
    for each  $v \in V[G]$ 
       $d_{uv} = \delta'(u, v) - h(u) + h(v)$ 
  return D
}
```

Johnson's algorithm复杂度分析

- 执行一次Bellman-Ford。 $O(|V||E|)$ 。
- 执行 $|V|$ 次Dijkstra。
 - 使用Fibonacci heap，总计 $O(|V|^2 \log |V| + |V||E|)$ 。
 - 使用Binary heap，总计 $O(|V||E| \log |V|)$ 。
- 当 $|E|$ 足够小，比Warshall-Floyd快。